N° d'ordre NNT : xxx

THESE de DOCTORAT DE L'INSA LYON,
membre de l'Université de Lyon

Ecole Doctorale N° 512
École doctorale Informatique et Mathématiques de Lyon

Spécialité de doctorat :
Informatique

Soutenue par :

Aurélien Cécille

Structure-Aware Self-Supervised Monocular Depth Estimation

Devant le jury composé de :

David PICARD	Directeur de recherche	ENPC - IP Paris	Rapporteur
Philippe XU	Professeur	ENSTA - IP Paris	Rapporteur
Liming CHEN	Professeur	Centrale Lyon	Examinateur
Véronique CHERFAOUI	Professeure	UTC, Compiègne	Examinatrice
Céline TEULIÈRE	Maîtresse de Conférences	Université Clermont Auvergne	Examinatrice
Franck DAVOINE	Directeur de recherche	CNRS	Co-directeur de thèse
Stefan DUFFNER	Professeur	INSA Lyon	Co-directeur de thèse
Rémi AGIER	Docteur	Visual Behavior	Invité
Thibault NEVEU	Ingénieur	Visual Behavior, Sophialys	Invité

Département FEDORA – INSA Lyon - Ecoles Doctorales

SIGLE	ECOLE DOCTORALE	NOM ET COORDONNEES DU RESPONSABLE
ED 206 CHIMIE	**CHIMIE DE LYON** **https://www.edchimie-lyon.fr** Sec. : Renée EL MELHEM Bât. Blaise PASCAL 7, Avenue Jean Capelle 69621 Villeurbanne CEDEX **secretariat@edchimie-lyon.fr**	**M. Stéphane DANIELE** C2P2-CPE LYON-UMR 5265 Bâtiment F308, BP 2077 43 Boulevard du 11 novembre 1918 69616 Villeurbanne CEDEX **directeur@edchimie-lyon.fr**
ED 341 E2M2	**ÉVOLUTION, ÉCOSYSTÈME, MICROBIOLOGIE, MODÉLISATION** **http://e2m2.universite-lyon.fr** Sec. : Bénédicte LANZA Bâtiment Atrium, 43, Boulevard du 11 Novembre 1918 69622 Villeurbanne CEDEX Tél : 04.72.44.83.62 **secretariat.e2m2@univ-lyon1.fr**	**Mme Sandrine CHARLES** Université Claude Bernard Lyon 1 UFR Biosciences, Bâtiment Mendel 43, boulevard du 11 Novembre 1918 69622 Villeurbanne CEDEX **e2m2.codir@listes.univ-lyon1.fr**
ED 205 EDISS	**INTERDISCIPLINAIRE SCIENCES-SANTÉ** **http://ediss.universite-lyon.fr** Sec. : Bénédicte LANZA Bâtiment Atrium, 43, Boulevard du 11 Novembre 1918 69622 Villeurbanne CEDEX Tél : 04.72.44.83.62 **secretariat.ediss@univ-lyon1.fr**	**M. Jérôme LAMARTINE** Université Claude Bernard Lyon 1 Equipe Intégrité Fonctionnelle du Tissu Cutané LBTI - UMR5305 CNRS IBCP - 7 passage du Vercors, 69367 Lyon CEDEX 07 Tel : 04.72.72.26.66 **jerome.lamartine@univ-lyon1.fr**
ED 34 EDML	**MATÉRIAUX DE LYON** **http://ed34.universite-lyon.fr** Sec. : Yann DE ORDENANA Tél : 04.72.18.62.44 **yann.de-ordenana@ec-lyon.fr**	**M. Stéphane BENAYOUN** Ecole Centrale de Lyon Laboratoire LTDS 36 avenue Guy de Collongue 69134 Ecully CEDEX Tél : 04.72.18.64.37 **stephane.benayoun@ec-lyon.fr**
ED 160 EEA	**ÉLECTRONIQUE, ÉLECTROTECHNIQUE, AUTOMATIQUE** **https://edeea.universite-lyon.fr** Sec. : Philomène TRECOURT Bâtiment Charlotte PERRIAND 69621 Villeurbanne CEDEX Tél : 04.72.43.71.70 **secretariat.edeea@insa-lyon.fr**	**M. Philippe DELACHARTRE** INSA LYON Laboratoire CREATIS Bâtiment Blaise Pascal, 7 avenue Jean Capelle 69621 Villeurbanne CEDEX Tél : 04.72.43.88.63 **philippe.delachartre@insa-lyon.fr**
ED 512 INFOMATHS	**INFORMATIQUE ET MATHÉMATIQUES** **http://edinfomaths.universite-lyon.fr** Sec. : Renée EL MELHEM Bât. Blaise PASCAL 7, Avenue Jean Capelle 69621 Villeurbanne CEDEX **infomaths@univ-lyon1.fr**	**M. Dragos IFTIMIE** Université Claude Bernard Lyon 1 Bâtiment Braconnier 21 Avenue Claude BERNARD 69 622 Villeurbanne CEDEX Tél : 04.72.44.79.58 **direction.infomaths@listes.univ-lyon1.fr**
ED 162 MEGA	**MÉCANIQUE, ÉNERGÉTIQUE, GÉNIE CIVIL, ACOUSTIQUE** **http://edmega.universite-lyon.fr** Sec. : Philomène TRECOURT Bâtiment Charlotte PERRIAND 69621 Villeurbanne CEDEX Tél : 04.72.43.71.70 **mega@insa-lyon.fr**	**M. Etienne PARIZET** INSA Lyon Laboratoire LVA Bâtiment St. Exupéry 25 bis av. Jean Capelle 69621 Villeurbanne CEDEX **etienne.parizet@insa-lyon.fr**
ED 483 ScSo	**ScSo**[1] **https://edsciencessociales.universite-lyon.fr** Sec. : Mélina FAVETON Tél : 04.78.69.72.76 Université Lyon 2 Campus Berges du Rhône, Bureau BEL 309 18, Quai Claude BERNARD 69365 LYON CEDEX 07 **melina.faveton@univ-lyon2.fr**	**M. Bruno MILLY** Univ. Lyon 2 Campus Berges du Rhône 18, quai Claude Bernard 69365 LYON CEDEX 07 Bureau BEL 319 **bruno.milly@univ-lyon2.fr**

[1] ScSo : Histoire, Géographie, Aménagement, Urbanisme, Archéologie, Science politique, Sociologie, Anthropologie

Abstract

This thesis studies self-supervised monocular depth estimation, with a particular focus on side camera mirrors used in trucks and buses. In this setting, large amounts of unlabeled video are available, while depth annotations are costly to acquire. Self-supervised learning therefore provides an attractive training paradigm by exploiting geometric consistency between consecutive images instead of relying on ground-truth depth maps. However, despite strong progress in recent years, standard self-supervised monocular methods still face important limitations for practical deployment: they do not recover metric scale reliably, and they produce blurred depth discontinuities at object boundaries.

The first contribution of this thesis addresses scale ambiguity. This self-supervised method injects ground information into the training process by using known camera parameters to derive an analytic depth prior assuming a flat ground. It is then combined with a learned attention mechanism that identifies where this prior is trustworthy in the image. A dedicated loss formulation couples ground selection and scale recovery, allowing the network to learn metrically meaningful depth without requiring depth supervision or ground mask annotations. Experiments on KITTI show competitive metric depth performance among self-supervised approaches, while additional evaluations demonstrate stronger robustness to camera pose changes and improved zero-shot transfer to unseen cameras and datasets such as DDAD.

The second contribution addresses the loss of sharpness at object boundaries. We propose to use a depth formulation in which each pixel in the depth map is represented by a two-component gaussian mixture rather than by a single scalar value. This representation explicitly models the coexistence of foreground and background hypotheses at occlusion edges. The predicted distributions are propagated through reprojection, interpolation, and photometric loss computation, making the full self-supervised pipeline compatible with multimodal depth prediction. The resulting model produces sharper discontinuities, reduces floating artifacts in reconstructed point clouds, and provides uncertainty estimates that are meaningful for downstream applications. We also introduce an edge entropy measure to quantify boundary sharpness more directly than conventional global depth metrics.

Taken together, these contributions show that incorporating explicit structure into self-supervised learning improves monocular depth estimation along dimensions that standard photometric objectives capture poorly. Ground geometry provides a practical route to metric scale when such a prior is available, while mixture-based depth representations better reflect the ambiguity present at occlusion boundaries.

Résumé

Cette thèse porte sur l'estimation auto-supervisée de profondeur monoculaire, avec un intérêt particulier pour les rétroviseurs par caméra utilisés sur les camions et les bus. Dans ce contexte, de grandes quantités de vidéos non annotées sont disponibles, alors que l'acquisition d'annotations de profondeur est coûteuse. L'auto-supervision constitue donc un cadre d'apprentissage adapté, en exploitant la cohérence géométrique entre images successives plutôt que des annotations de profondeur. Malgré les progrès importants réalisés ces dernières années, les méthodes monoculaires auto-supervisées présentent encore des limitations majeures pour un usage en conditions réelles : elles ne restituent pas l'échelle métrique de manière fiable, et elles produisent des transitions de profondeur floues aux frontières des objets.

Une première contribution de cette thèse traite le problème de l'ambiguïté d'échelle. Cette méthode auto-supervisée introduit une contrainte géométrique avec le sol à partir des paramètres connus de la caméra. Un a priori analytique de profondeur est calculé en partant du principe que le sol est plat, ce qui est combiné à un mécanisme d'attention apprenant à déterminer dans quelles régions de l'image cet a priori est fiable. Une formulation de perte spécifique couple la sélection des zones de sol et la récupération de l'échelle, ce qui permet au réseau d'apprendre une profondeur métrique cohérente sans supervision directe ou annotations de segmentation du sol. Les expériences menées sur KITTI montrent des performances compétitives en profondeur métrique parmi les approches auto-supervisées. Des évaluations complémentaires mettent en évidence une meilleure robustesse aux variations de pose de la caméra ainsi qu'un meilleur transfert zéro-shot vers des caméras et des jeux de données non vus durant l'apprentissage comme DDAD.

Une seconde contribution concerne la perte de netteté aux contours des objets. Nous proposons une formulation auto-supervisée dans laquelle chaque pixel de la carte de profondeur est représenté par un mélange gaussien à deux composantes plutôt que par une valeur scalaire unique. Cette représentation modélise explicitement la coexistence d'hypothèses de premier plan et de second plan au niveau des frontières d'occultation. Les distributions prédites sont propagées à travers la reprojection, l'interpolation et le calcul de la fonction de perte photométrique, ce qui rend l'ensemble de la chaîne auto-supervisée compatible avec une prédiction de profondeur multimodale. Le modèle obtenu produit des discontinuités plus nettes, réduit les artefacts flottants dans les nuages de points reconstruits et fournit des estimations d'incertitude utiles pour l'application finale. Une mesure d'entropie au niveau des bordures est également introduite afin d'évaluer notre contribution plus directement que les métrique de profondeur globales.

Dans son ensemble, ce travail montre que l'intégration explicite de structure dans l'apprentissage auto-supervisé améliore l'estimation de profondeur monoculaire sur des aspects que les objectifs photométriques standards saisissent mal. La géométrie du sol fournit un ancrage pratique pour retrouver une échelle métrique, tandis qu'une représentation de profondeur par mélange rend mieux compte de l'ambiguïté présente aux frontières d'occultation.

Contents

1 Introduction 1
- 1.1 Depth Estimation from Cameras . . . 2
- 1.2 Self-Supervised Depth Estimation . . . 2
- 1.3 Wider applications . . . 3
- 1.4 Thesis Organization . . . 4

2 Monocular Depth Estimation 7
- 2.1 Depth Estimation . . . 7
- 2.2 Pinhole Camera Model . . . 12
- 2.3 Neural Networks . . . 18
- 2.4 Datasets and Sensors . . . 23
- 2.5 Conclusion . . . 28

3 Self-Supervised Depth and Pose Estimation 31
- 3.1 Multi-view Geometry . . . 31
- 3.2 Learning Depth and Pose . . . 35
- 3.3 Loss Functions . . . 42
- 3.4 Conclusion . . . 48

4 Monocular Metric Depth Estimation 51
- 4.1 The Scale Ambiguity Problem . . . 52
- 4.2 Scale Clues . . . 54
- 4.3 Ground Constraint for Self-Supervised Metric Depth . . . 62
- 4.4 Conclusion . . . 71

5 Boundary Sharpness in Depth Estimation 73
- 5.1 Related Work . . . 74
- 5.2 Methodology . . . 78
- 5.3 Experiments . . . 85
- 5.4 Conclusion . . . 90

6 Conclusions and Perspectives 91
- 6.1 Summary of Contributions . . . 91
- 6.2 Relation to Current Research Trends . . . 92
- 6.3 Future Directions . . . 93
- 6.4 Conclusion . . . 94

A Mathematical Derivations and Visualizations 95
- A.1 Reprojection Equation with Disparity . . . 95
- A.2 Error Variance Approximation . . . 96
- A.3 Additional Visualization . . . 98

Chapter 1

Introduction

This thesis was conducted in the context of an industrial partnership with Visual Behavior. During this time, the company was developing a visual Advanced Driver Assistance Systems (ADAS) for trucks and buses. The product at the center of this work is a Camera Monitor System (CMS): a system that replaces traditional vehicle mirrors with cameras and displays as visible in Figure 1.1. Deployed on trucks and buses, CMS offers wider fields of view than mirrors, eliminating blind spots, improving safety for drivers and vulnerable road users alike. Beyond their role as mirror replacements, the cameras in these systems can serve as inputs to intelligent software for tasks such as vehicle and vulnerable road user (VRU) detection, obstacle avoidance, parking assistance.

The presence of cameras at multiple positions around the vehicle, each generating continuous video during operation, raises a natural question: can we extract three-dimensional understanding of the scene directly from these images ? Depth estimation, the task of determining how far objects are from the camera, is central to this goal. If accurate depth can be obtained from the CMS cameras alone, it enables safety-related features such as obstacle detection, free-space estimation, and distance-based warnings without requiring additional sensors. This chapter introduces the technical context and challenges that motivated this thesis, and outlines the contributions we developed to address them.

(a) CMS camera mounted on the side of a bus. (b) CMS display inside the bus.

Figure 1.1: Camera Monitor System (CMS) for trucks and buses.

1.1 Depth Estimation from Cameras

Several sensing technologies exist for recovering 3D information from a scene. LiDAR systems measure distances using laser pulses, providing accurate metric depth but at significant cost. Stereo camera systems triangulate depth from two viewpoints, offering a more affordable alternative at the expense of careful calibration and a need for overlapping cameras. Structured light and time-of-flight sensors provide active depth estimation but face limitations in outdoor environments and at longer ranges. Monocular depth estimation, the task of extracting depth from a single camera image, is the most constrained of these approaches but also the most practical for a system like CMS, where cameras are already present and adding further sensors is undesirable.

Since CMS cameras are already installed for the driver, they are available at no additional hardware cost for perception tasks. A single camera is sufficient to attempt depth estimation, avoiding the integration complexity and expense of equipping each viewpoint with dedicated depth sensors. The trade-off is that extracting depth from a single image is difficult: a 2D image is the projection of a 3D scene, and infinitely many 3D configurations can produce the same image. Recovering depth therefore requires exploiting prior knowledge, learned regularities, or geometric constraints.

Accurate depth from CMS cameras has direct safety applications. Knowing the distance to a cyclist, rather than merely detecting their presence, enables the system to assess whether turning is safe. Depth is also needed to estimate collision time for emergency braking, to identify navigable free-space for maneuvering, and to detect obstacles entering safety-critical zones. A particularly concrete application in the CMS setting is the automatic detection of the rear end of a truck or bus trailer. Unlike the cab, trailers vary in length and carry no camera of their own; the system must infer the trailer's rear position from the side CMS cameras. This information can be used to automatically provide the driver with a zoomed view of the space behind the trailer, which is especially critical during reversing maneuvers. Robustly tracking the trailer end as the vehicle moves further requires reasoning about uncertainty: an uncertainty-aware physical tracker fuses successive depth estimates into a coherent trajectory, and for this to work the depth predictions must carry reliable uncertainty information alongside their point estimates. These applications require not just relative depth ordering (which object is closer) but metric depth (how far in meters), and they require geometric accuracy for precise obstacle detection. As we will see, these two requirements, metric scale and boundary precision, are precisely where our contributions focus.

1.2 Self-Supervised Depth Estimation

A practical advantage specific to the CMS setting is the volume of data these systems produce. During normal operation, each camera generates continuous video covering diverse driving conditions, road layouts, and environments. This data is abundant but unlabeled: no depth ground truth is associated with it. Obtaining labeled depth data, for instance through LiDAR, is costly and limits the diversity of training sets. The temporal structure of these video sequences, however, provides a form of implicit supervision. As the vehicle moves, consecutive frames observe the same scene from different viewpoints, and this view variation encodes geometric information about scene depth. Self-supervised learning methods exploit this property, using the geometric consistency between frames as a training signal in place of explicit depth labels.

Self-supervised depth estimation builds on classical Structure from Motion (SfM) principles, reformulated within differentiable neural network frameworks. The central idea is that photometric consistency across views can serve as a training signal without requiring any manual annotation. If a network correctly predicts the depth of a scene and the camera motion between two frames, it can synthesize how one frame should appear from the viewpoint of the other. Comparing this synthesized view to the actual observed frame produces a loss: errors in the predicted depth or motion result in photometric inconsistencies that the network learns to minimize.

Since self-supervised depth estimation has matured considerably over the past years, with a series of works addressing challenges such as dynamic objects, occlusion handling, and multi-scale training, it brings performance closer to supervised baselines. However, self-supervised methods still inherit a fundamental limitation from monocular geometry: scale ambiguity. A small object nearby and a large object far away produce identical image projections, and a monocular system trained without external scale information cannot distinguish between them. Networks trained this way learn to predict depth maps with correct relative proportions, but the absolute metric scale remains undetermined. A second limitation concerns object boundaries. Pixels along occlusion edges observe both foreground and background, and models tend to blur depth transitions at these locations, producing artifacts in the predicted depth maps. These two challenges, scale ambiguity and boundary blur, are the focus of the contributions presented in this thesis.

1.3 Wider applications

Although this thesis is motivated by the specific constraints of Camera Monitor Systems for heavy vehicles, the underlying research questions are not limited to that setting. More broadly, the work addresses how monocular systems can recover depth that is both metrically meaningful and geometrically precise from large amounts of unlabeled video. These requirements arise in many perception systems beyond the original industrial context.

The most immediate extension is to autonomous driving research. Passenger cars, delivery vehicles, agricultural machines, and other road-going platforms face the same fundamental need to infer scene geometry from cameras in order to support planning and safety functions. In these settings, both contributions of the thesis are relevant. Metric depth is required whenever decisions depend on physical distance, for example to estimate stopping margins, assess whether a maneuver is feasible, or reason about free-space around the vehicle. At the same time, sharp depth boundaries are important to localize obstacles precisely, recover clean object shapes, and avoid artifacts in the reconstructed 3D scene. The methods developed here therefore extend naturally to road-going autonomous systems beyond trucks and buses.

The same observation extends naturally to mobile robotics. Many robots operate close to the ground in structured or semi-structured environments, including warehouse robots, service robots, delivery robots, and field robots. For such platforms, depth estimation supports localization, obstacle avoidance, traversability analysis, and short-horizon navigation. Here again, metric scale is useful for reliable spatial reasoning, while sharp boundaries improve the geometric quality of the scene representation near obstacles and object contours. This is particularly important for navigation, where boundary artifacts can create false obstacles in the reconstructed scene and lead the robot to behave unnecessarily conservatively. Since these robots are grounded, the geometric assumptions used in Chapter 4 are often meaningful, but the boundary-aware modeling developed in

Chapter 5 directly addresses this issue and is equally relevant for downstream navigation and reconstruction.

The boundary-sharpness contribution also extends to aerial platforms such as drones. In that setting, the assumption of a stable ground plane beneath the camera is often too weak or too variable to provide a reliable metric prior, so the scale-recovery contribution is less directly applicable. However, the same issue of boundary artifacts creating spurious structures is also problematic there, especially around thin scene elements such as cables, vegetation, or building edges. The contribution of Chapter 5 is therefore relevant beyond grounded platforms even when the assumptions of Chapter 4 do not hold.

Taken together, these observations place the thesis at the intersection of automotive perception, robotics, and general 3D scene understanding. The industrial CMS application provides the motivating use case, but the methods speak to a broader problem: how to make monocular self-supervised depth estimation sufficiently informative for real systems, both in terms of absolute scale when geometric cues allow it, and in terms of structural fidelity at object boundaries across a wide range of platforms.

1.4 Thesis Organization

The remainder of the thesis is organized as follows. Chapter 2 establishes the foundations of monocular depth estimation, introducing depth representations ranging from dense maps to point clouds, surveying depth sensing modalities including LiDAR and stereo systems, formalizing the pinhole camera model and projection geometry, and examining the monocular depth cues that neural networks must implicitly learn. The chapter further presents neural network architectures for depth prediction, discusses depth output representations and their implications, and defines the evaluation metrics used to assess depth estimation quality.

Chapter 3 develops the self-supervised learning paradigm that forms the basis for our contributions. After introducing the multi-view geometric principles inherited from classical Structure from Motion, the chapter presents the photometric consistency framework that enables depth learning from video alone. It formalizes the reprojection pipeline through which depth predictions and camera poses synthesize novel views, details the loss functions including photometric terms and edge-aware smoothness regularization, and explains training strategies such as auto-masking for handling dynamic objects and multi-scale prediction refinement.

With these foundations in place, Chapters 4 and 5 present the core research contributions of this thesis. Chapter 4 addresses the scale ambiguity problem that prevents standard self-supervised methods from producing metric depth. After analyzing the sources of scale ambiguity in both classical Structure from Motion and neural depth estimation, and reviewing prior approaches including velocity-based scaling and supervised ground priors, the chapter introduces a ground constraint method for metric self-supervised monocular depth. The method integrates ground plane constraints directly into self-supervised training through novel loss functions that encode geometric consistency with flat ground geometry derived from camera parameters. An attention mechanism adaptively localizes ground regions, providing interpretability and robustness to varying camera extrinsics. Extensive experiments on KITTI demonstrate improvements in metric depth accuracy, while zero-shot transfer to DDAD validates generalization across diverse camera configurations.

Chapter 5 tackles the boundary sharpness problem wherein standard depth predictions blur discontinuities at object edges. After characterizing the artifacts that arise from single-value depth representations at boundaries, and reviewing prior work on confidence estimation, the chapter introduces our contribution for sharper object boundaries. This method represents per-pixel depth as a mixture distribution, explicitly modeling the multimodal depth observed by edge pixels that simultaneously view foreground and background. The chapter develops the mathematical framework for propagating these distributions through the self-supervised pipeline, including reprojection geometry, photometric sampling, and loss computation. Experiments demonstrate substantial improvements in point cloud quality at boundaries, quantified through a novel edge entropy metric, alongside qualitative comparisons showing the elimination of floating artifacts and the potential for uncertainty propagation to downstream systems.

Chapter 6 concludes the thesis by reflecting on the contributions and their implications. The chapter synthesizes insights from both our methods, discusses their combined impact on enabling metric-accurate, boundary-sharp depth estimation within scalable self-supervised learning frameworks, and outlines promising directions for future work.

Chapter 2

Monocular Depth Estimation

Before explaining self-supervised learning and structural constraints, it is necessary to establish the formal foundations of depth estimation from a single image. While Chapter 1 introduced the practical motivations within the context of vehicle perception, this chapter focuses on the geometric and computational mechanics of monocular depth estimation.

The transition from a three-dimensional physical space to a two-dimensional image plane fundamentally destroys depth information, meaning that any single image can theoretically represent an infinite number of scaled 3D configurations [25, 45]. Solving this mathematical ambiguity depends entirely on exploiting priors like patterns, textures, objects, and configurations that naturally occur in the visual world [105]. Today, deep learning architectures [25, 100] provide the primary mechanism for extracting and mapping these priors into dense pixel-wise depth predictions.

This chapter details the core concepts and terminology used throughout this thesis. The first section formalizes the depth estimation process: we characterize the visual cues that implicitly enable monocular depth perception and define the standard pinhole camera model that governs perspective projection. The second section examines the deep learning methodology, covering the standard neural network architectures designed for dense prediction and the mathematical metrics used to evaluate them. Finally, the third section review the physical sensors often used to generate ground truth and the standard driving datasets that supply the necessary data to train and benchmark deep learning models.

2.1 Depth Estimation

2.1.1 Depth Map and Point Cloud

Depth estimation is the process of determining the distance of objects in a scene from the camera. While depth can be obtained from various sensors, in this work we specifically focus on estimating depth from a single RGB image, a task known as *monocular depth estimation* [57, 85]. This is typically achieved by predicting a depth value for each pixel in the image, resulting in a *dense depth map* [122]. Figure 2.1 shows a typical example, where colors encode the distance of each scene point from the camera.

Given a depth map and knowledge of the camera's internal parameters (discussed in Section 2.2), it is possible to reconstruct the three-dimensional structure of the scene by projecting each pixel into 3D space. This produces a *point cloud*: a set of points in 3D space representing the visible surfaces of objects [80, 104, 148]. The example in

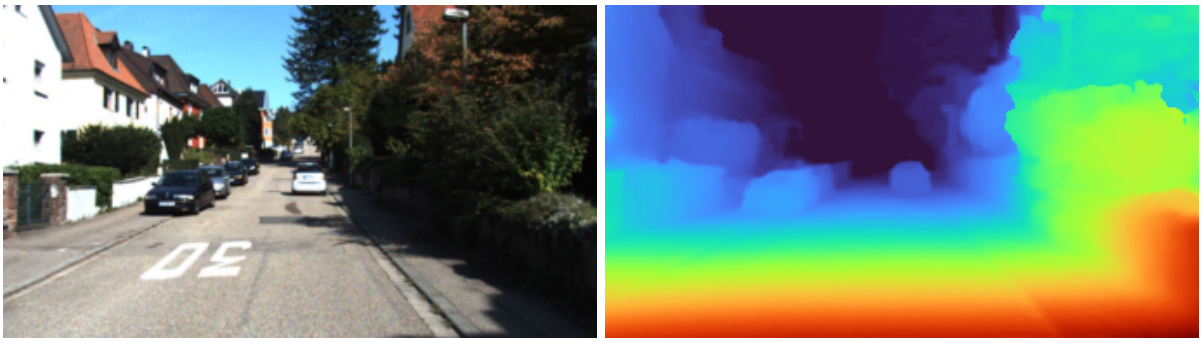


Figure 2.1: Example of an image and its corresponding depth map. The depth map encodes the distance of each pixel from the camera, with warmer colors indicating closer objects and cooler colors indicating farther objects.

Figure 2.2 shows the resulting 3D structure for a single image-depth pair. Point clouds provide a rich geometric representation of the environment that is widely used in robotics, autonomous driving, and augmented reality applications. The mathematical details of this back-projection are presented in Section 2.2.

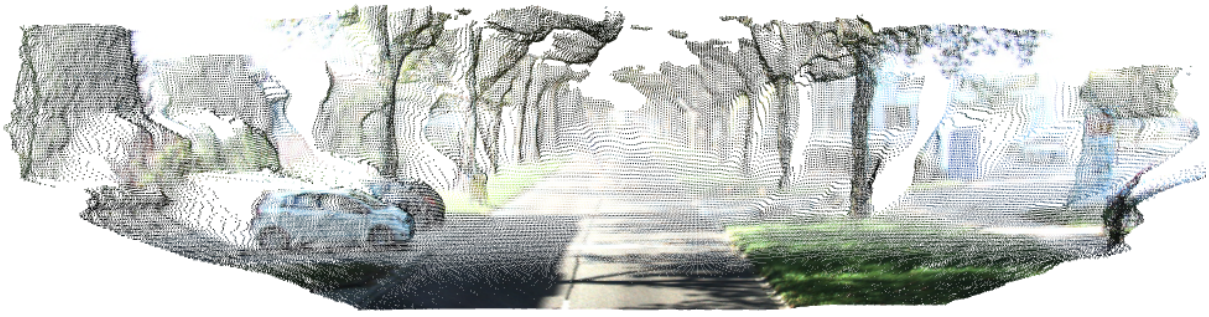


Figure 2.2: Example of a point cloud obtained from an image and its corresponding depth map. Notice the empty space behind trees and cars, which correspond to areas not visible in the image

2.1.2 Visual Monocular Depth Cues

Since this thesis focuses on the problem of estimating depth from a single RGB image, it is crucial to understand the visual cues that can be used to infer depth. Monocular depth estimation is fundamentally an ill-posed problem: a single two-dimensional image is merely a planar projection of a three-dimensional scene. Because the perspective projection collapses the depth dimension, an infinite number of three-dimensional configurations can theoretically produce the exact same two-dimensional projection. Resolving this inherent ambiguity requires the model to rely on learned priors, contextual semantics, and geometric visual cues rather than direct geometric triangulation.

While humans primarily rely on binocular vision for depth perception at close range, we are also remarkably capable of perceiving depth from a single image or when viewing distant scenes where stereo cues are ineffective. This ability depends on a set of *monocular depth cues* present in a single viewpoint that provide information about spatial layout [19, 36, 49, 92]. Understanding these cues is essential, as they form the implicit knowledge that

modern neural networks must learn to perform monocular depth estimation [105]. In the following, we review the main monocular cues and discuss their implications.

Before examining specific cues, it is important to distinguish between different types of depth information [19]. *Ordinal depth* refers to the relative ordering of objects in depth (e.g., object A is closer than object B), without specifying by how much [150]. *Relative depth* provides quantitative information about depth relationships, such as ratios or differences (e.g., object A is twice as far as object B). Finally, *absolute* or *metric depth* specifies the actual distance in physical units such as meters [143]. Most monocular cues provide only ordinal or relative depth information while absolute depth typically requires additional knowledge such as familiar object sizes or camera parameters.

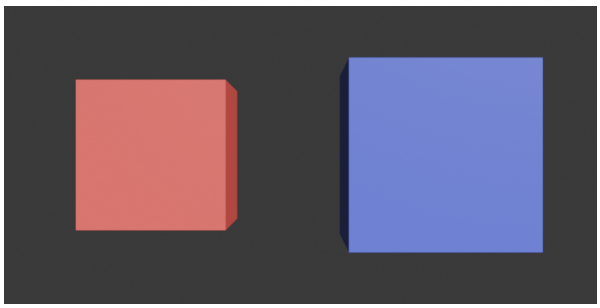
Relative and familiar size

In general, the apparent size of an object in the image is inversely related to its distance from the camera. If there are two objects in an image, the larger one is usually interpreted as being closer. This intuitive relation relies on *relative size* [3,27]. By itself, however, this cue is fundamentally weak because the size in the image depends jointly on distance and on the object's true physical dimensions. Figure 2.3a illustrates this ambiguity: without additional context, the large blue cube appears closer than the small red cube. This stems from an implicit assumption by the human visual system that both objects share roughly the same physical size.

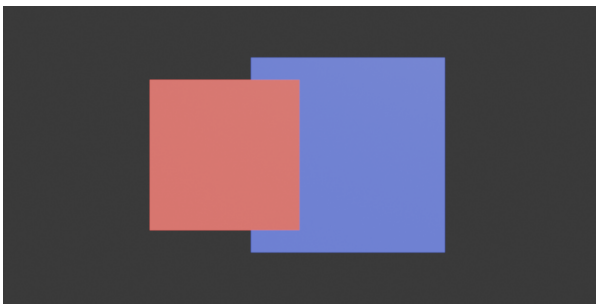
Conversely, if the observer knows an object's typical physical size, such as a car, a pedestrian, or a traffic sign, this observation transitions into a *familiar size* cue and can support stronger, and in some cases approximately absolute, distance judgments.

Occlusion

When one object partially blocks the view of another, the occluding object is perceived as closer. In Figure 2.3b, the red cube partially hides the blue one, which immediately fixes their depth ordering [48]. Occlusion is one of the strongest depth cues, as it provides unambiguous ordinal depth information: the occluded object must be behind the occluder. However, it gives no information about the actual distance between the two objects even in a relative sense. Despite this limitation it remains a crucial cue for scene understanding.



(a) Apparent size cue: the large blue cube appears closer than the small red cube under the assumption of similar physical size.



(b) Occlusion cue: the red cube partially blocks the blue one, indicating that the red cube is closer.

Figure 2.3: Monocular depth cues based on object size and occlusion.

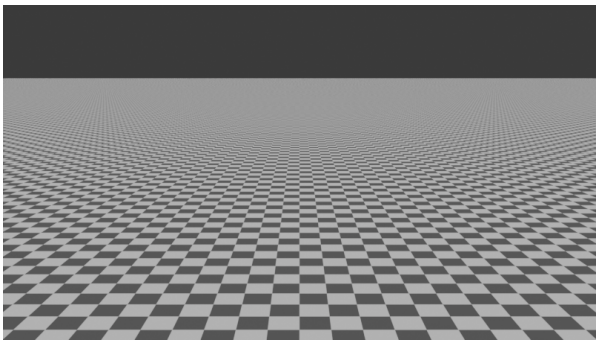
Linear perspective

Parallel lines in the world appear to converge toward a common vanishing point in the image [18]. This phenomenon, known as linear perspective, is a direct consequence of projective geometry and provides strong cues about scene depth and spatial layout. The railway tracks in Figure 2.4a are a canonical example: their apparent convergence conveys a clear sense of depth recession. Vanishing points and lines also encode information about the scene's 3D structure and camera orientation, making them useful for both depth estimation and camera calibration tasks.

Texture gradient

As a textured surface recedes into the distance, the density of texture elements increases in the image while their individual size decreases [82]. Figure 2.4b makes this effect easy to see on a ground plane, where the checker pattern gets denser with distance. This systematic variation, known as the texture gradient, provides a continuous depth cue across surfaces. It is particularly informative for ground planes (e.g., grass fields, tiled floors), where the texture gradient clearly indicates the surface extending away from the camera.

(a) Linear perspective cue: parallel tracks converge toward a vanishing point.



(b) Texture gradient cue: the ground plane texture becomes denser with distance.

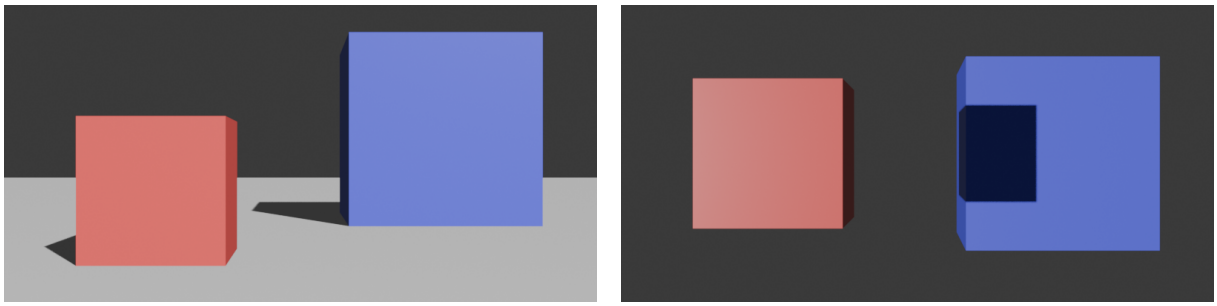
Figure 2.4: Monocular depth cues based on linear perspective and texture gradient.

Height in the visual field

For objects resting on a ground plane, those placed higher in the image (closer to the horizon) are farther away [90]. Figure 2.5a shows this directly: the cube nearer the horizon is read as farther than the one lower in the image. This cue is closely related to perspective projection and is particularly effective for estimating the depth of objects on flat surfaces such as roads or floors.

Cast Shadows

When an object casts a shadow on an other, the shadow provides a strong cue about the ordinal depth relationship between the two objects [83]. In the setup of Figure 2.5b, the shadow makes it clear that the red cube lies closer to the light source than the blue one. Additionally, the position and size of the shadow can provide information about the relative positions of the objects and the light source, which can be used to infer depth relationships in complex scenes.



(a) Height-in-field cue: the cube with heightest contact point with the ground appears farther away.

(b) Cast shadow cue: the red cube casts a shadow on the blue cube, indicating that the red cube is closer to the light source.

Figure 2.5: Monocular depth cues based on cast shadows and height in the visual field.

Atmospheric perspective

Objects at greater distances appear hazier, less saturated, and shifted toward blue due to light scattering by particles in the atmosphere [87]. In Figure 2.6, the distant mountains are visibly hazier and bluer than the foreground trees. This effect, known as atmospheric or aerial perspective, provides a monocular depth cue that is especially prominent in outdoor scenes with large depth ranges like in mountains photography. The degree of haze can be used to infer relative depth, with more distant objects appearing more obscured. However, this cue is highly dependent on environmental conditions and may not always be reliable.

Figure 2.6: The mountains in the background appear hazier and bluer than the trees in the foreground, illustrating the atmospheric perspective cue.

Together, these clues provide a rich set of visual features that can be used to infer depth from a single image. But it is often said that monocular depth estimation is an ill-posed problem [2]. This is because there is no geometric constraint that directly links the 2D image to the 3D world, and the same image can be generated by many different 3D scenes as in Figure 2.3a. Moreover, it is important to note that monocular depth estimation is inherently *scale-ambiguous*: without additional information (such as known object sizes or camera parameters), it is not always possible to determine absolute distances. A photograph of a miniature model can appear identical to a photograph of a full-scale scene. This fundamental ambiguity, which we will revisit in later chapters, has important implications for the design and evaluation of monocular depth estimation methods.

2.2 Pinhole Camera Model

The pinhole camera model is a simple yet effective way to model the way cameras work and has been presented in mutiple works [44, 117, 118, 144]. It is based on the idea of a pinhole, which is a small hole that allows light to pass through a surface and project an image onto a plane as seen in Figure 2.7.

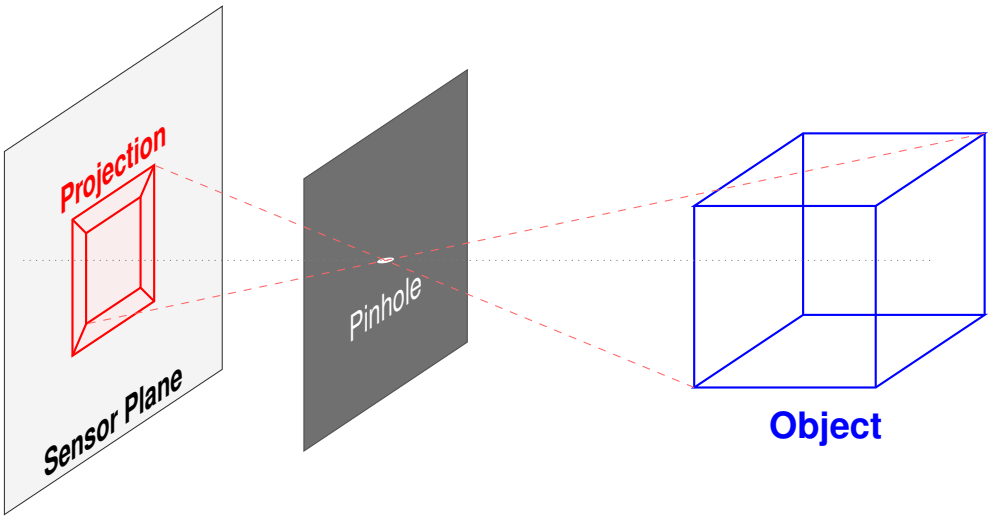


Figure 2.7: 3D-to-2D perspective projection. A 3D object (blue cube) in the world is projected onto a 2D image plane (sensor) through the pinhole. The projection demonstrates how 3D points map to 2D image coordinates following the pinhole camera model, with closer objects projecting larger and farther objects projecting smaller.

For cameras, this plane is the sensor, which converts light into an electrical signal that can be processed by a computer. This pinhole is an idealized model where a single point in the scene always results in a single light ray that hits the sensor. It results in a perfectly sharp image whereas, if multiple light rays could hit the same point on the sensor, it would result in a blurry image. We can also make the simplified analogy with the human eye as in Figure 2.8, where the pinhole is represented by the pupil and the sensor by the retina.

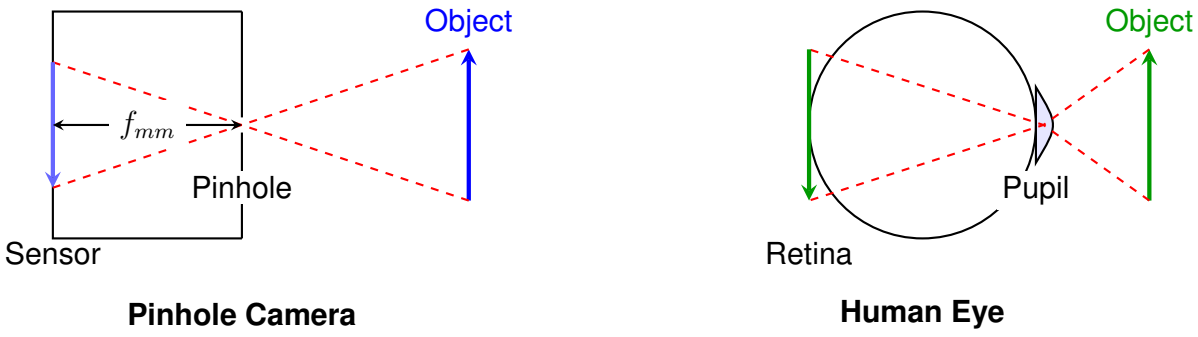


Figure 2.8: Pinhole camera model compared with human eye structure. Light rays converge through the pinhole (or pupil) to form an inverted image on the sensor (or retina).

Notice that in this model there is no lens, which is a common element in modern cameras. Lenses are used to focus light onto the image sensor and to control the field

of view. When the field of view is relatively small, a complex lens system can often be approximated well by a simple pinhole camera model, provided that lens distortions remain limited. In such cases, the dominant effect of the lens is mainly reflected in the effective focal length of the system. However, for lenses with larger fields of view (such as wide-angle or fisheye lenses), the lens introduces optical aberrations and radial distortions that bend the light rays [134]. In such cases, the standard pinhole camera model is insufficient, and more complex distortion models (such as the Brown-Conrady model) must be applied to un-distort the image before the pinhole projection equations can be used.

2.2.1 Camera Parameters

The principal parameter in this model is the focal length, which is the distance between the pinhole and the image sensor (see Figure 2.8). The focal length determines the field of view of the camera, with shorter focal lengths resulting in wider fields of view and longer focal lengths resulting in narrower fields of view. The focal length can be physically measured in millimeters (mm) but in computer vision it is usually represented in pixels, which is the focal length divided by the size of a pixel on the image sensor.

$$f_{\text{pixels}} = \frac{f_{\text{mm}}}{\text{pixel_size}_{\text{mm/px}}} \tag{2.1}$$

In addition to the focal length, the pinhole camera model also includes the principal point, which is the point on the image sensor where the optical axis intersects the sensor plane. The optical axis (or principal axis or principal ray) being the line passing through the pinhole and perpendicular to the image plane like in Figure 2.9. The principal point is usually located at the center of the image sensor, but it can be offset due to manufacturing tolerances or misalignment of the camera components. The principal point is usually represented by its coordinates in pixels, denoted as (c_x, c_y).

Putting this together, we can represent the camera intrinsics using a matrix, known as the camera intrinsic matrix or simply the camera matrix:

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \tag{2.2}$$

where f_x and f_y are the focal lengths in pixels along the x and y axes, respectively. In most cases, the pixels are approximatively square, so $f_x \approx f_y$.

This intrinsic matrix K is crucial as it allows us to convert between 3D coordinates in the camera frame to 2D coordinates in the image plane. We denote a 3D point in the camera frame as $P = (X, Y, Z) \in \mathbb{R}^3$, its 2D pixel coordinates as $\mathbf{p} = (u, v) \in \mathbb{R}^2$ and its homogeneous image coordinates as $\tilde{\mathbf{p}} = (u, v, 1) \in \mathbb{R}^3$. The projection in homogeneous form is given by:

$$Z\,\tilde{\mathbf{p}} = KP \quad \Leftrightarrow \quad Z \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} \tag{2.3}$$

Dividing both sides by Z and expanding the matrix multiplication yields the explicit pixel coordinates, which we formalize as a projection function $\pi : \mathbb{R}^3 \to \mathbb{R}^2$ (defined for

$Z > 0$):

$$\mathbf{p} = \pi(P, K) = \begin{bmatrix} f_x \frac{X}{Z} + c_x \\ f_y \frac{Y}{Z} + c_y \end{bmatrix} \tag{2.4}$$

This reveals the core mechanism of perspective projection: the image coordinates depend on the ratios X/Z and Y/Z, which collapses the 3D scene onto a 2D plane.

To simplify the analysis of this projection, it is often useful to consider a *normalized image plane*, as illustrated in Figure 2.9. This virtual plane is located at unit distance ($Z = 1$) from the camera center. A 3D point $P = (X, Y, Z)$ projects onto this plane as a normalized point which we will denote as $\tilde{\mathbf{p}}_n = (x_n, y_n, 1)$.

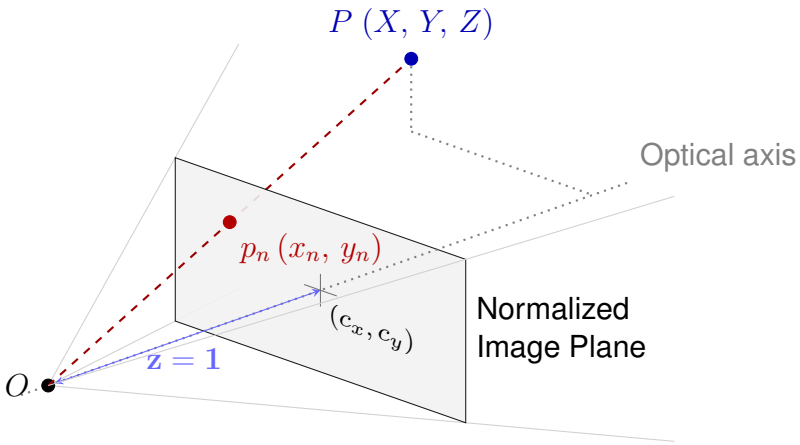


Figure 2.9: Virtual normalized image plane used in computer vision. Instead of drawing the physically inverted sensor image, we place a plane at $z = 1$ in front of the camera and represent projections as upright normalized coordinates, with $x_n = X/Z$ and $y_n = Y/Z$.

In homogeneous coordinates, casting the 3D point into these normalized coordinates explicitly amounts to dividing by depth Z:

$$\tilde{\mathbf{p}}_n = \frac{1}{Z} P = \begin{bmatrix} X/Z \\ Y/Z \\ 1 \end{bmatrix} = \begin{bmatrix} x_n \\ y_n \\ 1 \end{bmatrix} \tag{2.5}$$

By decoupling the non-linear perspective division from the camera intrinsics, the camera matrix K acts merely as a standard linear operator that maps normalized homogeneous coordinates into homogeneous pixel coordinates $\tilde{\mathbf{p}}$:

$$\tilde{\mathbf{p}} = K \tilde{\mathbf{p}}_n \tag{2.6}$$

This formulation highlights that the normalized homogeneous point $\tilde{\mathbf{p}}_n$ conceptually represents a ray intersecting the plane $Z = 1$, while K simply handles the affine transformation (scaling by focal length and translating by the principal point) into the final pixel coordinates.

An important consequence of this perspective effect is the relationship between object size and depth. Consider an object of physical width W located at distance Z from the camera. Its projected width w in the image is:

$$w = f_x \frac{W}{Z} \tag{2.7}$$

This inverse relationship between the object size in the image and depth is one of the fundamental monocular depth cues discussed in the previous section. Figure 2.10 illustrates the same idea with mutiple objects of identical physical size placed at different depths.

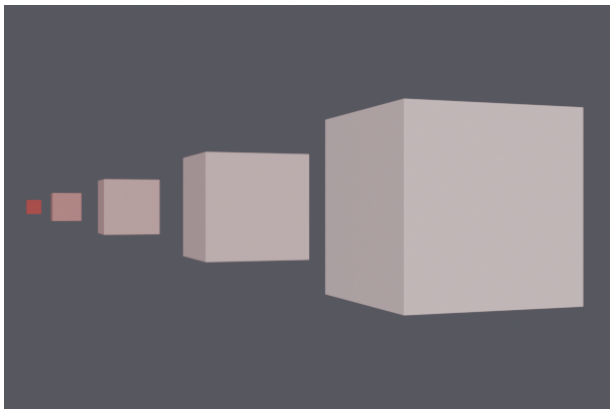
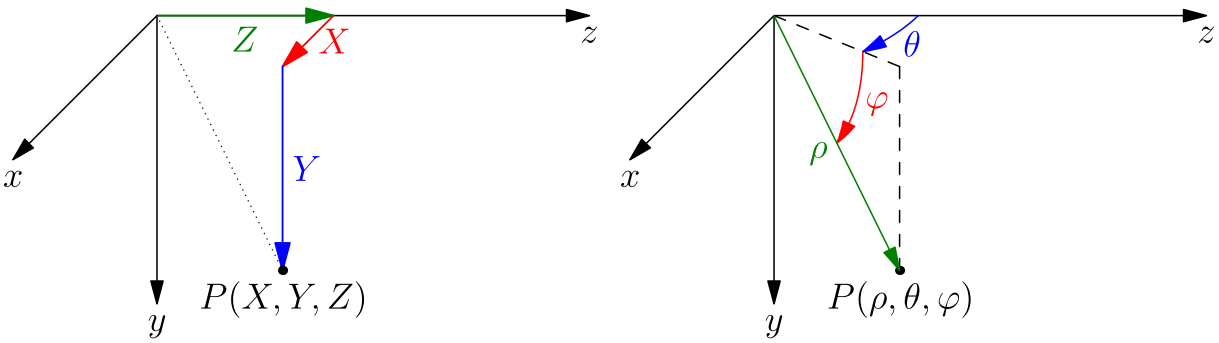


Figure 2.10: Effect of perspective projection on objects of the same size in the image. As depth increases, objects get proportionally smaller.

Cartesian and spherical parameterizations



(a) Cartesian coordinates with depth Z.

(b) Spherical coordinates with range ρ.

Figure 2.11: Comparison of two ways to describe a 3D point in the camera frame: Cartesian coordinates (X, Y, Z) explicitly expose optical axis depth Z while spherical coordinates (ρ, θ, φ) encode radial distance and direction.

Throughout this manuscript, we represent 3D points in the camera frame with Cartesian coordinates $P = (X, Y, Z)$, which is the standard choice for pinhole projection and for composing transformations an visible in Figure 2.11a. In this parameterization, the depth is directly given by Z, and perspective projection is naturally written with the ratios X/Z and Y/Z.

The alternative of Figure 2.11b is to parameterize the same point with spherical coordinates $P = (\rho, \theta, \varphi)$, where ρ is the radial distance to the camera center, θ is the azimuth, and φ is the elevation angle. The conversion is:

$$\rho = \sqrt{X^2 + Y^2 + Z^2}, \qquad \theta = \operatorname{atan2}(X, Z), \qquad \varphi = \operatorname{atan2}\left(Y, \sqrt{X^2 + Z^2}\right) \tag{2.8}$$

and inversely:

$$X = \rho \cos\varphi \sin\theta, \qquad Y = \rho \sin\varphi, \qquad Z = \rho \cos\varphi \cos\theta \tag{2.9}$$

The distinction is useful for interpretation: Cartesian coordinates emphasize translation along camera axes, while spherical coordinates emphasize viewing directions and angular sampling. For wide-angle cameras, spherical representations are often more relevant since it makes the depth prediction independent of the viewing direction [55]. This can easily be seen with 180° fisheye cameras, where points projecting to the edge of the image would have a Cartesian depth $Z = 0$ but a more intuive spherical depth ρ. In this thesis, since the datasets do not use wide angle cameras, we keep the Cartesian camera-frame formulation for consistency with the pinhole model and standard monocular depth pipelines.

2.2.2 Extrinsic Parameters

While the intrinsic matrix K describes the internal geometry of the camera, the extrinsic parameters describe the camera's position and orientation in the world. These parameters are essential when working with multiple cameras or when tracking camera motion over time, as is the case in structure from motion or visual odometry.

Rotation and Translation

The extrinsic parameters consist of a rotation matrix $R \in SO(3)$ and a translation vector $t \in \mathbb{R}^3$, which together describe the rigid transformation from world coordinates to camera coordinates. Given a 3D point $P_w = (X_w, Y_w, Z_w)$ in world coordinates, its position in the camera frame $P_c = (X_c, Y_c, Z_c)$ is obtained by:

$$P_c = RP_w + t \tag{2.10}$$

The rotation matrix R is a 3×3 orthonormal matrix ($R^T R = I$, $\det(R) = 1$) representing the camera's orientation. The translation vector t represents the position of the world origin as seen from the camera frame. Alternatively, we can express this transformation using homogeneous coordinates:

$$\begin{bmatrix} P_c \\ 1 \end{bmatrix} = \begin{bmatrix} R & t \\ 0^T & 1 \end{bmatrix} \begin{bmatrix} P_w \\ 1 \end{bmatrix} = T \begin{bmatrix} P_w \\ 1 \end{bmatrix} \tag{2.11}$$

where T is the 4×4 extrinsic matrix (also called the camera pose matrix) that maps world coordinates to camera coordinates.

Complete projection pipeline

To project a 3D point from world coordinates to image coordinates, we combine both intrinsic and extrinsic parameters. The complete projection is given by:

$$Z \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K[R|t] \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix} \tag{2.12}$$

The matrix product $K[R|t]$ is called the projection matrix or camera matrix, which directly maps world coordinates to homogeneous image coordinates.

Coordinate frame conventions

The complete projection pipeline involves distinct coordinate frames, and it is important to clearly distinguish between them. Figure 2.12 summarizes the relationship between the three main frames involved in this thesis.

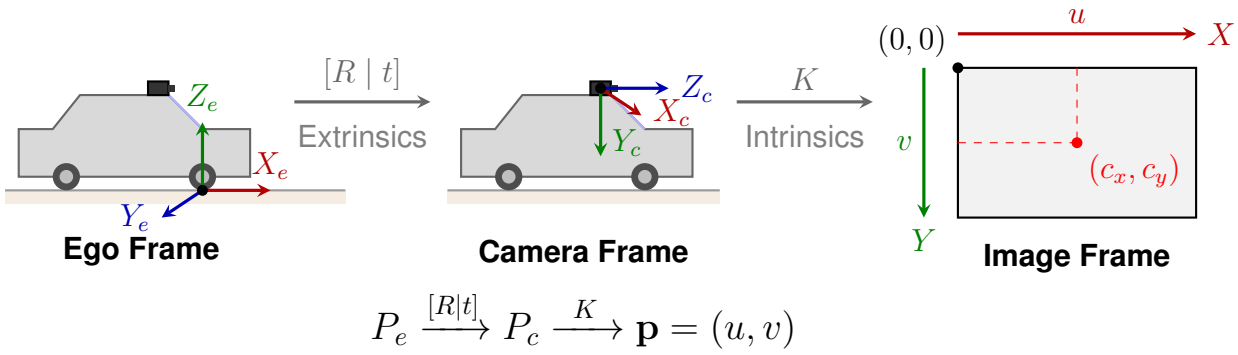


Figure 2.12: The three coordinate frames involved in the projection pipeline. A point P_e in the ego frame is transformed to the camera frame via the extrinsic parameters $[R \mid t]$, then projected onto the image frame via the intrinsic matrix K. The camera is mounted on a vehicle, illustrating a typical autonomous driving setup.

The *world frame* is an arbitrary 3D coordinate system in which scene points P_w are defined. Its origin and orientation depend on the application: for instance, it may be anchored to a fixed landmark. In autonomous driving, this frame is mostly used to track the position of the vehicle across time and the origin is the first position of the vehicle.

The *ego frame* (or vehicle frame) is a coordinate system attached to the mobile platform (e.g., the center of the rear axle of a car). As we are not doing a global mapping in this work, this is the main reference frame we will use. The extrinsic parameters (R, t) typically transform points from the ego frame P_e to the camera frame.

The *camera frame* is a 3D coordinate system centered at the camera's optical center. The standard convention in computer vision uses a right-handed system with the Z-axis pointing forward, the X-axis pointing right, and the Y-axis pointing down. A 3D point in this frame is denoted $P_c = (X_c, Y_c, Z_c)$.

The *image frame* is the 2D pixel coordinate system resulting from the projection through the intrinsic matrix K. The origin is located at the top-left corner of the image, with the X-axis pointing right and the Y-axis pointing down. Pixel coordinates (u, v) are obtained by applying the projection function $\pi(KP_c)$ to a point in the camera frame. The principal point (c_x, c_y), where the optical axis intersects the image plane, typically lies near the center of the image.

Point Cloud from Depth Map

Given an image I and its corresponding depth map $\mathcal{Z}$, we can use the camera intrinsics to reconstruct a 3D point cloud with one point per pixel. Each pixel (u, v) in the image corresponds to a point $P_{i,j} = (X, Y, Z)$ in 3D space, which can be calculated as follows:

$$P_{i,j} = \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \mathcal{Z}_{u,v} K^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \tag{2.13}$$

Once the 3D points are computed, we can also map the RGB color from the image to each point in the point cloud, resulting in a colored point cloud like Figure 2.2.

2.3 Neural Networks

Despite the known depth cues, monocular depth estimation remains a very challenging task: most of these cues are not easily extractable from images and often require a good understanding of the scene, which is difficult to obtain with traditional computer vision techniques. In practice, this creates a circular dependency: estimating depth would benefit from higher-level cues such as object segmentation, occlusions, shadows, or scene layout, yet inferring these cues reliably also often depends on some notion of depth. Neural networks have been shown to be very effective at learning complex patterns and representations from data, making them well suited for monocular depth estimation [25,38,67,100].

2.3.1 Architectures

Neural networks for monocular depth estimation typically follow an encoder–decoder architecture [25, 38, 67, 99]. The encoder extracts features from the input image, while the decoder generates the depth map from those features. This decomposition reflects the two main requirements of the task: first, building features rich enough to capture object identity, layout, and long-range context, and second, recovering a dense pixel-aligned output with sufficiently sharp local detail. Since the encoder usually reduces spatial resolution to enlarge the receptive field, most practical architectures also rely on skip connections or explicit multi-scale fusion to recover fine structures and object boundaries in the final prediction [38, 52, 78, 101].

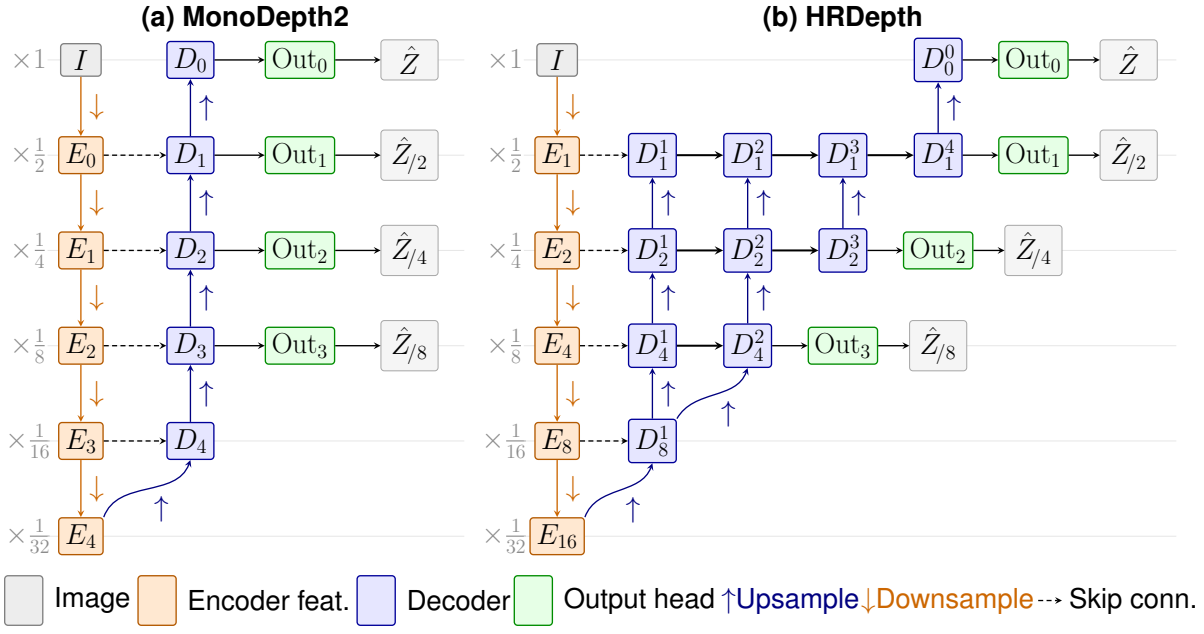


Figure 2.13: Example architectures for monocular depth estimation: Monodepth2 (left) and HR-Depth (right)

Encoders

Encoders provide the multi-scale image features that depth decoders consume. Common choices are ResNets [46] or its more modern variants ConvNeXts [76]. Convolutional encoders remain particularly common because their inductive biases match image structure well: locality, weight sharing, and translation equivariance make them efficient at extracting edges, corners, textures, and progressively larger scene patterns [46, 112]. In monocular depth estimation this is important because the network must jointly reason about low-level cues, such as gradients and contours, and higher-level cues, such as object extent and scene layout.

Vision Transformers [23] or hybrid variants (e.g., DPT [99], Swin [75], MonoViT [146], DepthFormer [73]) are also first class choices since they are better at capturing long-range context and global layout, but are usually more computationally intensive. This makes them attractive when accuracy on diverse scenes is the main objective, but less convenient in settings where training cost, inference latency, or repeated experimentation are important constraints. For this reason, much of the self-supervised literature, and the models discussed later in this manuscript, still rely primarily on convolutional backbones [37, 38, 78, 149].

Modern convolutional backbones narrow part of the gap with transformer models. For instance, ConvNeXt [76], retains a purely convolutional design while adopting several effective modern design choices, such as patch-like downsampling, larger kernels, depthwise convolutions, and layer normalization. The result is a backbone that preserves the scaling and deployment properties of convolutions while substantially improving representational power. The work of Spencer *et al.* [115] compared the contribution of self-supervised methods using a common ConvNeXt-Base backbone, and found that the choice of architecture itself can account for a large part of the performance gap between methods, which highlights the importance of carefully controlling for architectural differences.

Decoder and fusion

Decoders and feature-fusion together convert encoder features into a dense depth map. Their role is not only to upsample coarse latent features, but also to reinject spatial precision that was lost during encoding. In practice, this is usually done by combining deep semantic features with earlier higher-resolution activations through skip connections, with explicit feature pyramids [52, 78, 101]. Such fusion is especially important for preserving object contours, thin structures, and small foreground elements, which would otherwise be oversmoothed.

The lightweight Monodepth-style decoder [38] inspired from U-Nets [101] upsamples deep features with small conv blocks and uses skip connections or multi-scale supervision for stability. This design is simple and efficient, which explains its early adoption in self-supervised monocular depth estimation [34, 37, 149]. High-resolution designs (HR-Depth [78]) explicitly preserve a high-resolution stream and repeatedly fuse it with deeper semantics, yielding more detailed predictions and better thin-structure recovery; Figure 2.13 contrasts these two design philosophies directly. In general, these decoders are often similar to those used in semantic segmentation, which also requires dense pixel-aligned outputs and benefits from multi-scale fusion [101]. Transformer-based decoders (DPT [99]) reassemble encoder tokens into image-like feature maps, progressively refine them through transformer blocks with convolutional readout projections, and fuse multi-scale features to predict depth.

2.3.2 Outputs

All depth estimation networks produce per-pixel predictions, but they differ in the prediction formulation and the depth space used. Design choices at the output layer are not merely implementation details: they directly affect model performance across depth ranges, and the type of geometric prior that is easiest for the network to learn [7, 32, 95].

Prediction Formulation

The most direct approach treats depth prediction as a regression problem: the network outputs a continuous depth value for each pixel [25, 38, 69]. This formulation is conceptually simple and matches the continuous nature of depth but can't represent the uncertainty in the prediction [56, 97]. Alternatively, histogram-based methods [7, 32] discretize the depth range into bins and predict a probability distribution over these bins it often followed by an expectation to obtain the final depth value. This turns prediction into a classification-like problem, which can make optimization easier and gives the model a way to represent ambiguity over depth hypotheses. Figure 2.14 shows such a per-pixel distribution over discretized bins.

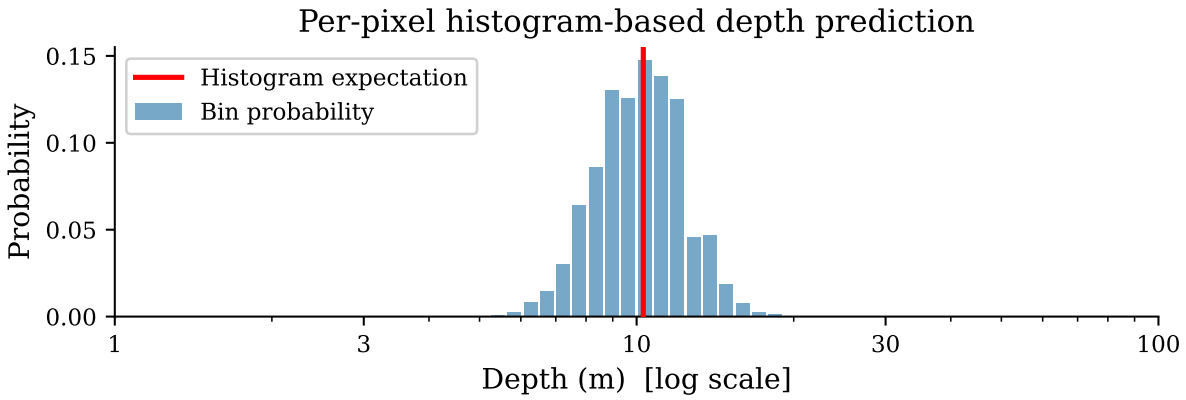
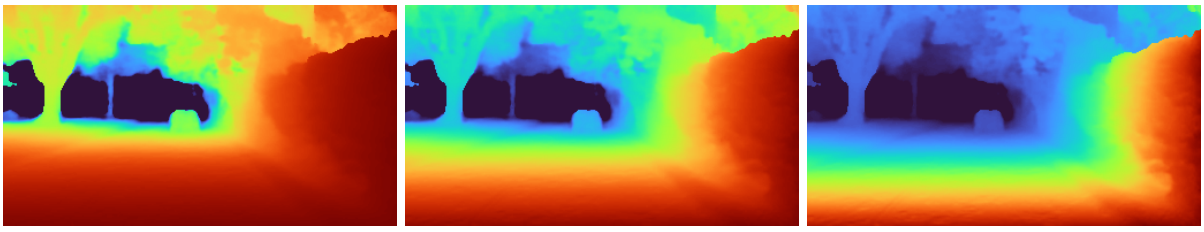


Figure 2.14: Example discretized depth bins prediction for a single pixel.

Depth Space

Networks can make predictictions in different depth spaces. Direct depth prediction [25, 67] is intuitive but gives uniform weight to all depth ranges and typically uses ReLU outputs or sometimes softplus ($\log(1 + e^x)$) to ensure positive depth values. Disparity (inverse depth) [17, 37, 38] is often preferred because it has a linear relationship with stereo pixel shifts, constant spatial derivatives on planar surfaces, and provides better sensitivity to nearby objects, often of more importance for safety-critical applications; networks predicting disparity commonly use a sigmoid activation to bound the output to $(0, 1)$, which is then scaled to the target disparity range using constants such as z_min and z_max. Log-depth [95] is also an interesting option, it offers scale-invariance properties, naturally de-emphasizes distant objects and can use linear outputs since log-depth may be negative; it is especially suitable for scenes with large depth ranges. Figure 2.15 compares how the colormap of the same scene looks with these three parameterizations.

Beyond the mean depth value itself, some methods predict additional quantities at each pixel. A common extension is predictive uncertainty to weight unreliable regions [56, 97].



(a) Depth (b) Log-depth (c) Disparity

Figure 2.15: Visual depth spaces with a maximum depth of 50m.

Other approaches jointly predict related scene properties, such as surface normals or semantic labels, in order to encourage more structured internal representations [24]. Even when these auxiliary outputs are not used at inference time, they can shape the learned features and improve depth quality during training.

2.3.3 Evaluation Metrics

Evaluating monocular depth estimation requires careful consideration of multiple complementary metrics. Depth map based metrics introduced by Eigen *et al.* [24, 25] established the standard evaluation protocol and the community has adopted a diverse set of metrics presented in Table 2.1 that each capture different aspects of prediction quality. Absolute error metrics (MAE, RMSE) measure accuracy in metric units, making them interpretable but weight all points similarly. Relative metrics like Absolute Relative error (AbsRel) and Squared Relative error (SqRel) emphasize near-field accuracy and can be interpreted as a percent error. Log-space metrics $\text{RMSE}_{\log}$ and Scale Invariant Logarithmic error (SILog) reduce the influence of distant regions and provide better invariance to the scale of the scene, with SILog specifically designed to ignore uniform scale offsets entirely. Threshold-based accuracy metrics ($\delta < 1.25^k$) offer an intuitive measure of the fraction of accurate predictions, and are fully robust to outliers.

The scaling protocol is also interesting to note. Since monocular depth estimation can be ambiguous up to a global scale factor, many benchmarks report results after applying a median scaling to align predictions with ground truth [38, 149]. For metric methods, however, the task is harder since the scale is estimated by the model and the predictions are evaluted directly without scaling [13, 95, 141].

Many applications such as robotics, autonomous driving, or AR/VR use depth as input for 3D reconstruction. In these cases, evalutating point clouds directly in 3D space can be more informative. Point cloud metrics like Chamfer Distance and Earth Mover's Distance measure geometric similarity between predicted and ground-truth 3D structures [1, 28], while precision-recall-based metrics (F-score, IoU at distance τ) quantify the accuracy at a given tolerance [61]. These 3D metrics summarized in Table 2.2 complement image-space measurements by directly assessing the quality of the final reconstructed geometry rather than per-pixel agreement.

Throughout this thesis, we report results using the full suite of standard image-based metrics to ensure comprehensive comparisons with prior work. For applications involving 3D reconstruction (Chapter 5), we additionally evaluate point cloud quality using Chamfer Distance, F-score and IoU metrics to directly assess geometric fidelity.

Metric	Formula	Key Properties
AbsRel	$\frac{1}{N}\sum_i \frac{\lvert\hat{z}_i - z_i\rvert}{z_i}$	Interpretable (%); emphasizes near objects; sensitive to small z_i
SqRel	$\frac{1}{N}\sum_i \frac{(\hat{z}_i - z_i)^2}{z_i^2}$	Penalizes large errors; strongly influenced by outliers
MAE	$\frac{1}{N}\sum_i \lvert\hat{z}_i - z_i\rvert$	Metric units (m); robust; scale-dependent
RMSE	$\sqrt{\frac{1}{N}\sum_i (\hat{z}_i - z_i)^2}$	L2 error in meters; sensitive to large errors
$\text{RMSE}_{\log}$	$\sqrt{\frac{1}{N}\sum_i (\log \hat{z}_i - \log z_i)^2}$	Emphasizes relative errors; reduces influence of distant pixels
SILog	$\sqrt{\frac{1}{N}\sum_i e_i^2 - \left(\frac{1}{N}\sum_i e_i\right)^2}$ where $e_i = \log \hat{z}_i - \log z_i$	Scale-invariant; penalizes structural errors
$\delta < 1.25^k$	$\frac{1}{N}\#\left\{i \mid \max\left(\frac{\hat{z}_i}{z_i}, \frac{z_i}{\hat{z}_i}\right) < 1.25^k\right\}$	Binary accuracy with $k \in \{1, 2, 3\}$

Table 2.1: Standard Depth map-based evaluation metrics. All metrics are computed only over N valid pixels in the ground truth. $\hat{z}_i$ denotes predicted depth and z_i ground-truth depth at pixel i.

Metric	Formula	Key Properties
Chamfer (CD)	$\frac{1}{\lvert P\rvert}\sum_{p\in P} \min_{q\in Q} \lVert p - q\rVert_2^2 + \frac{1}{\lvert Q\rvert}\sum_{q\in Q} \min_{p\in P} \lVert q - p\rVert_2^2$	Bidirectional nearest-neighbor; sensitive to density and outliers
EMD	$\min_\phi \sum_{p\in P} \lVert p - \phi(p)\rVert$ where $\phi : P \to Q$ bijection	Optimal transport; perceptually meaningful; $\mathcal{O}(n^3)$ complexity
F-score_τ	$\frac{2 \cdot \text{prec}_\tau \cdot \text{rec}_\tau}{\text{prec}_\tau + \text{rec}_\tau}$	Precision-recall balance at threshold τ; interpretable
IoU_τ	$\frac{\text{prec}_\tau \cdot \text{rec}_\tau}{\text{prec}_\tau + \text{rec}_\tau - \text{prec}_\tau \cdot \text{rec}_\tau}$	Spatial overlap ratio; complements F-score

Table 2.2: Point cloud-based evaluation metrics. P is the predicted point cloud and Q is the ground truth point cloud. Precision_τ and recall_τ denote the fraction of predicted or ground truth points within distance τ of the other set.

2.4 Datasets and Sensors

Evaluating monocular depth estimation methods requires carefully curated datasets with accurate ground truth depth measurements. The choice of dataset significantly impacts both training and evaluation, as different benchmarks exhibit distinct characteristics in terms of scene diversity, depth acquisition methods, and annotation density. This section reviews the primary sensors used to estimate depth and the datasets used throughout this thesis, highlighting their strengths, limitations, and specific use cases in our experiments.

We focus on four complementary benchmarks that span different environments and sensor modalities. KITTI provides the standard outdoor driving benchmark with sparse LiDAR ground truth, serving as the primary testbed for self-supervised methods. VKITTI2 offers synthetic outdoor scenes with dense, pixel-perfect depth annotations, enabling fine-grained analysis of depth discontinuities. DDAD extends outdoor evaluation with greater environmental diversity and 6 cameras to get 360 degrees coverage. Finally, NYUv2 represents indoor scenarios captured with structured-light depth sensors, featuring dense annotations and different scene characteristics. Together, these datasets allow comprehensive assessment of method performance across varied conditions and domains.

2.4.1 Depth Sensors

Although this work focuses on monocular depth estimation, it is important to understand the sensor technologies commonly used to acquire depth measurements [62], as they often serve as ground truth for training and evaluating monocular models.

LiDAR (Light Detection And Ranging)

LiDAR sensors emit laser pulses and measure the time it takes for each pulse to bounce off surfaces and return to the sensor. By computing the round-trip time, the distance to each surface point can be determined with high metric accuracy, typically at centimeter-level precision [72, 102, 108]. Modern automotive LiDAR sensors can produce hundreds of thousands of measurements per second. However, the resulting depth maps are inherently *sparse* as shown in Figure 2.16: a typical LiDAR provides depth for only small portions of the pixels in a corresponding camera image. LiDAR sensors are also expensive, mechanically complex, and sensitive to adverse weather conditions such as rain and fog [8].

Figure 2.16: Example of a sparse depth map from LiDAR sensors with typical line patterns. Only a fraction of pixels have valid depth measurements, which poses challenges for training and evaluating monocular depth estimation models.

Stereo cameras

Stereo camera systems use two horizontally displaced cameras to capture the scene from slightly different viewpoints, mimicking human binocular vision. Depth is estimated by computing the *disparity* between corresponding points in the left and right images: the horizontal shift in pixel position, which is inversely proportional to depth [47, 106] . Stereo systems can produce dense depth maps at relatively low cost, but their accuracy depends heavily on the quality of the stereo matching algorithm and degrades at longer ranges where disparity differences become sub-pixel [98]. They also struggle in textureless regions where correspondences are ambiguous. Recently, deep stereo matching paradigms and foundation models [29] have emerged to resolve these ambiguities, bridging the gap toward zero-shot metric prediction across multiple domains.

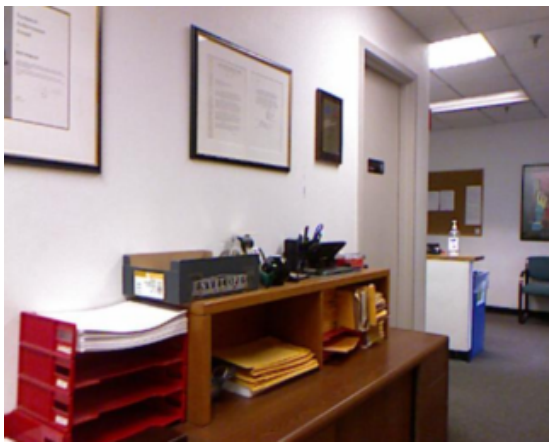
Time-of-Flight (ToF) cameras

ToF cameras measure depth by emitting active light signals, typically infrared, and measuring either the direct photon return time or the phase shift of the returned continuous wave signal [31, 42]. Unlike LiDAR, which scans the scene point by point, ToF cameras capture the entire scene simultaneously, producing dense depth maps at high frame rates with exceptionally low latency. This makes them highly suitable for dynamic tracking and robotics. However, they typically have limited range (up to about 10 meters for consumer devices), lower spatial resolution compared to RGB cameras [53], and can suffer from severe multi-path interference in complex indoor environments or when surface albedo varies highly [65, 86].

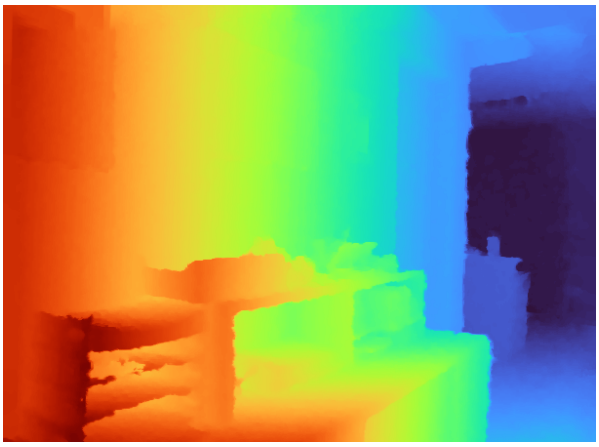
Structured light sensors

Structured light sensors, such as the often used Microsoft Kinect [89, 110, 145], project a known, mathematically coded pattern (e.g., an infrared dot grid) onto the scene. A displaced camera infers depth from the geometric deformation of this pattern by applying strict triangulation principles. They produce precise and dense depth maps for indoor environments as in Figure 2.17b. However, the technology suffers from severe vulnerabilities in uncontrolled environments. Because the system relies on observing the projected pattern, strong ambient lighting easily washes out the infrared projector, rendering it unsuitable for outdoor use [41, 93]. Furthermore, when multiple active structured light sensors operate in the same space, their projected patterns overlap and interfere, causing severe degradation with structural holes and phantom geometries [81].

While these sensors can provide accurate depth measurements, they each come with limitations in terms of cost, range, or environmental robustness. This motivates the pursuit of monocular depth estimation from a single RGB image, which offers a cost-effective, lightweight, and universally deployable alternative. It is worth noting that depth sensors, especially LiDAR, are routinely used to generate ground truth for training and evaluating monocular depth estimation models. A major consideration in this context is the sparsity of sensor measurements: the LiDAR example in Figure 2.16 covers only a fraction of the image pixels, whereas the structured-light example in Figure 2.17 is much denser but also less precise. This difference has direct implications for both training supervision and evaluation metrics.



(a) RGB image.



(b) Depth map.

Figure 2.17: Example of a depth map from a structured light sensor. While it provides dense depth measurements, it is limited to indoor environments and is generally not used for outdoor applications such as autonomous driving because of its limited range and sensitivity to ambient light.

Sensor fusion

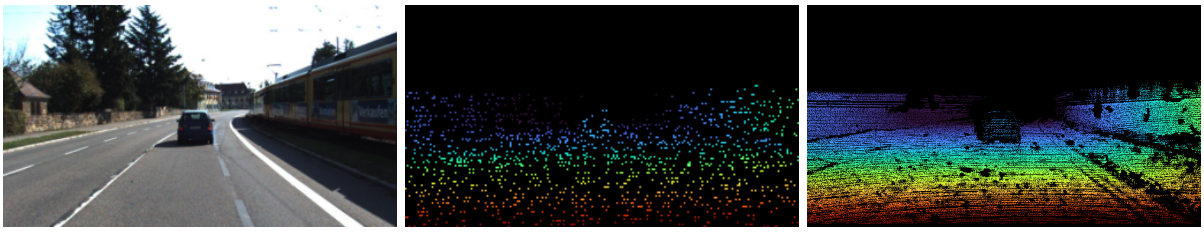
To overcome the limitations of individual sensors, modern autonomous systems often employ *sensor fusion*. Because the different sensors provide complementary information, combining disparate data streams into unified representations has become standard practice. A prominent example is depth completion, which fusions sparse but metricially accurate LiDAR measurements with dense, high-resolution RGB camera images [15, 66, 79]. In this paradigm, the system outputs a completely dense depth map that retains the quantitative precision of the LiDAR while drawing structural details, semantic context, and sharp boundaries from the visual cues in the RGB image. Sensor fusion strategies are very common in applications like autonomous driving, where both robustness against environmental variables [8] and high-fidelity point cloud generation are required simultaneously.

2.4.2 Datasets

Building on the sensor overview, we next describe the datasets commonly used in depth estimation research. The following subsections describe the key benchmarks used in this work, each offering distinct characteristics that enable comprehensive evaluation across diverse scenarios and sensor modalities.

KITTI

The KITTI dataset [35] is the de facto standard benchmarks for outdoor self-supervised monocular depth estimation. KITTI offers synchronized and calibrated stereo RGB images (baseline ≈ 54 cm), 3D LiDAR scans (Velodyne HDL-64) and an inertial navigation system (GPS/IMU). The images have varying resolutions near 1242×375 after rectification. Sparse ground-truth depth for evaluation is obtained by projecting LiDAR scans to images. Consequently, default depth supervision is very sparse (roughly 4–5% valid pixels) and biased towards lower parts of the scene where the LiDAR points at. These



(a) RGB image (b) LiDAR ground truth (c) Improved ground truth [123]

Figure 2.18: Example images from the KITTI dataset. The original LiDAR ground truth is very sparse and has artefacts at object borders due to occlusions. The improved ground truth is denser and more accurate, but removes some inconsistent parts of the original ground truth.

lower regions are also where most useful scene content lies (road, vehicles, pedestrians) as opposed to the top part which are typically sky or high building and trees.

Improved Labels. The original KITTI depth ground truth is very sparse and the projection to images has some limitations. The projection step is flawed because the LiDAR sensors is not exactly placed at the same position as RGB cameras, leading to occlusion artifacts at objects borders. To address this, [123] proposed a method to densify the LiDAR point clouds by accumulating multiple scans through time and combined with stereo depth to remove artefacts. The difference is visible in Figure 2.18. These labels, however, are still not always used in the literature, so caution is advised when comparing methods.

Common splits and evaluation. The most common split for self-supervised monocular depth estimation is the Zhou split [149], it is composed of 39810 training images and 4424 validation images. They are selected to remove parts with no car motion and are from the same 32 sequences. Then there is the eigen test set composed of 697 images from 28 sequences not in the training or validation sets. There is also the "eigen benchmark" test set with a subset of 652 images with improved ground truth from [123].

Preprocessing. We follow prevalent practice: (i) resize frames to a network input resolution of 192×640 preserving aspect ratio as closely as possible, (ii) apply random horizontal flips and color jitter.

Postprocessing. Following common practice, we resize the predicted depth maps back to the original image resolution for evaluation using bilinear interpolation. This allows for direct comparison with the sparse ground truth depth points projected from LiDAR which are defined in the original image space. The maximum depth is set to 80m for evaluation, as the LiDAR returns become increasingly sparse beyond this range.

VKITTI2

VKITTI2 [33] is a synthetic dataset created to mimic the KITTI dataset. It contains 5 different scenes, each one with 4 different weather/illumination conditions (clone, morning, sunset, overcast, rain). Each scene is composed of 1242 frames a resolution of 375×1242.

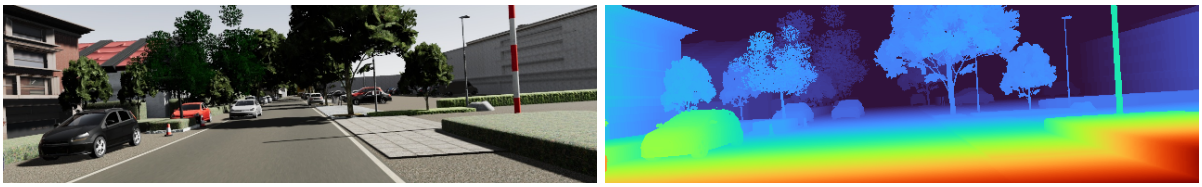


Figure 2.19: Example images from the VKITTI2 dataset with synthetic depth

The depth maps are dense and accurate, with a maximum depth of 655.3m. The images, however, are not photorealistic and so have a domain gap with real images.

Usage. We use VKITTI2 to evaluate the quality of the predicted depth maps at object boundaries. As can be seen in Figure 2.19, the synthetic depth maps are dense and perfectly accurate allowing for fine grained evaluation. In particular we evaluation the distance of the predicted depth discontinuities to the ground truth ones.

DDAD

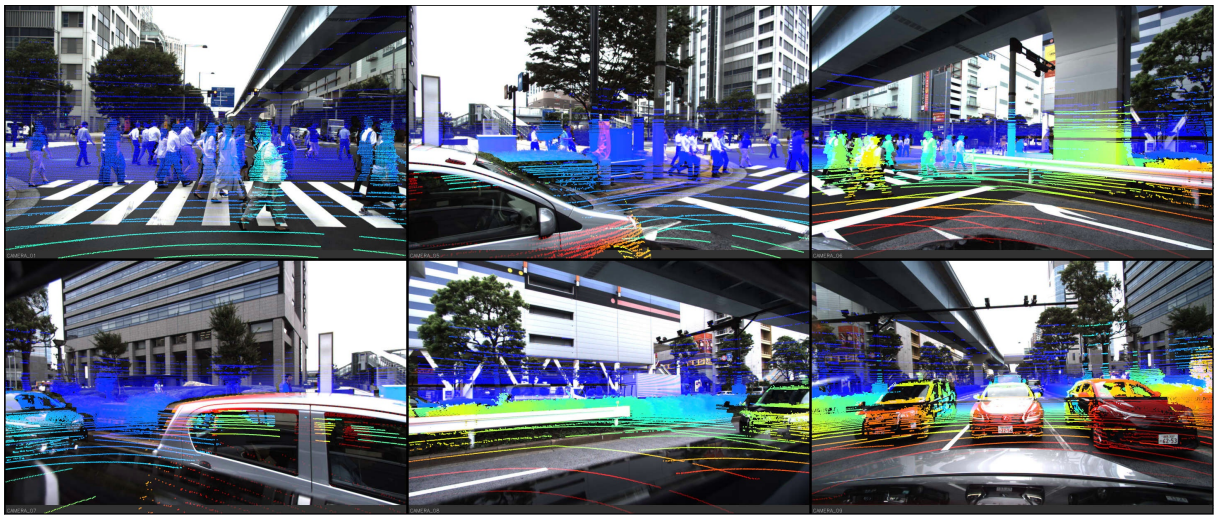


Figure 2.20: Example images from all cameras of DDAD dataset with LiDAR sensor.

The DDAD (Diverse Depth) dataset was introduced alongside advances in self-supervised metric depth estimation (e.g., PackNet / 3D Packing [40]) to provide broader environmental diversity than KITTI: varied cities, suburban and rural scenes, different weather (clear, overcast, rain) and illumination (day, dusk, night) conditions. It offers high-resolution multi-camera imagery (front, lateral, and rear views) with long-range LiDAR supervision aggregated from multiple sweeps, yielding denser coverage at far distances (beyond 80 m). While our primary monocular experiments emphasize KITTI for comparability, DDAD serves two purposes: (i) assessing generalization of scale-aware models to distributional shifts, and (ii) evaluating metric constraints (ground plane, scale cues) in scenes with richer 3D layout diversity, as illustrated by the different viewpoints and overlaid LiDAR annotations in Figure 2.20.

Splits and metrics. Public releases defines training and validation sequences with disjoint geographic regions to test generalization; we adhere to the official split when

available (details enumerated in the experimental section) and evaluate with the same depth metrics as KITTI.

Preprocessing. The images are resized to a target aspect-preserving resolution of 384×640 given the higher native resolution for training. And for generalization experiments, we keep the same aspect ratio as Kitti with a resolution of 192×640 by center cropping the height. During training, we apply random horizontal flips and color jitter, similarly to KITTI.

NYUv2

Figure 2.21: Example images from the NYUv2 dataset with the Kinect depth

NYUv2 [88] is an indoor RGB-D dataset captured with a Microsoft Kinect structured-light sensor, providing dense (though noisy) depth at VGA resolution (640×480) across 464 indoor scenes (kitchens, offices, living rooms, etc.). Figure 2.21 gives a representative RGB-depth pair from this benchmark. Compared to outdoor benchmarks, NYUv2 features shorter depth ranges, abundant occlusions, reflective/transparent surfaces, and strong lighting/color temperature shifts between artificial and natural illumination.

Splits and evaluation. We follow the standard protocol using the official train/test lists (249 scenes for training and 215 for testing), evaluating on the canonical 654 test images.

Preprocessing. We perform standard image borders removal of 16 pixels before resizing to 256×320 and keep random horizontal flips and colorjitter.

2.5 Conclusion

This chapter established the geometric and computational foundations of monocular depth estimation. First, the pinhole camera model and the underlying perspective projection demonstrated the fundamental ambiguity of predicting depth from a single viewpoint, an issue rooted in the loss of 3D information. Because building a 3D scene from 2D coordinates requires resolving this ambiguity, depth inference fundamentally relies on exploiting the visual cues and learned priors that are inherently present in natural images.

We then presented the different types of depth sensors used to generate ground-truth annotations use for both supervsied training and evaluation, including LiDAR, structured-light cameras, and stereo rigs. Each of these sensors has its own advantages and limitations in terms of accuracy, density, range, and cost.

In the second part, we address how modern deep neural networks method are used for monocular depth estimation, leveraging the different priors in images. The chapter reviewed the encoder-decoder architectures commonly used for this task, together with the output parameterizations that shape how depth is predicted. It also introduced the standard evaluation measures used throughout this thesis, from image-space error metrics to point-cloud-based criteria, which together provide a more complete view of prediction quality.

The chapter concluded with the datasets used to train and benchmark these models, ranging from sparse outdoor driving benchmarks such as KITTI and DDAD to dense synthetic or indoor datasets. These resources have made supervised monocular depth estimation highly effective, but they also expose its main practical limitation: obtaining reliable ground-truth depth remains expensive and difficult to scale. The next chapter therefore turns to self-supervised learning, where multi-view geometry is used to recover dense depth supervision directly from video.

Chapter 3

Self-Supervised Depth and Pose Estimation

The previous chapter introduced the problem of monocular depth estimation and the neural network architectures that predict depth from a single image. A central question remains: how to obtain a training signal without expensive ground-truth depth annotations. Collecting dense, pixel-aligned depth maps at scale requires specialized sensors such as LiDAR or structured-light cameras, careful calibration, and significant post-processing effort. Self-supervised methods circumvent this requirement by exploiting multi-view geometry as a free source of supervision. The core idea is that a pair or sequence of images captured by a moving camera already contains the geometric information necessary to learn depth: if we can correctly predict both the depth of a scene and the camera motion between frames, we can synthesize how one frame should appear from the viewpoint of another and compare this prediction against reality. This principle, rooted in classical structure from motion, is reformulated within a differentiable learning framework so that photometric reconstruction error drives the training of depth and pose networks jointly.

This chapter is organized in three parts. Section 3.1 reviews the multi-view geometry foundations of stereo vision and structure from motion that motivate the self-supervised approach and provide the geometric vocabulary used throughout. Section 3.2 presents the reprojection mechanism at the heart of self-supervised depth learning: pixels in a reference frame are unprojected into 3D using predicted depth, transformed by predicted camera pose, and projected back into a support frame where their color is sampled via differentiable bilinear interpolation. Section 3.3 describes the loss functions that together form the complete training objective: a photometric reconstruction loss combining SSIM and L1, edge-aware smoothness regularization, auto-masking strategies for handling dynamic objects and stationary regions, and multi-scale training for robust convergence. This framework, introduced by Godard *et al.* [37], Zhou et al. [149] and refined later [38], has become the standard paradigm for self-supervised monocular depth estimation and serves as the foundation for the contributions presented in Chapters 4 and 5.

3.1 Multi-view Geometry

Multi-view geometry provides the theoretical foundation for self-supervised depth estimation and presented in mutiple works [44, 117, 118]. When the same scene is observed from two or more viewpoints, the geometric relationships between views encode information about the three-dimensional structure of the scene. This section reviews two classical for-

mulations of this principle (stereo vision and structure from motion) that directly motivate the learning-based approach developed in the following sections.

3.1.1 Stereo Vision

Stereo vision starts from a simple idea: if two cameras observe the same scene from different viewpoints, the same 3D point appears at different image positions in the two views. An example is shown in Figure 3.1 where a tree is observed from two different viewpoints. This positional shift (disparity) carries depth information, such that larger shifts correspond to closer points and smaller shifts to farther points.

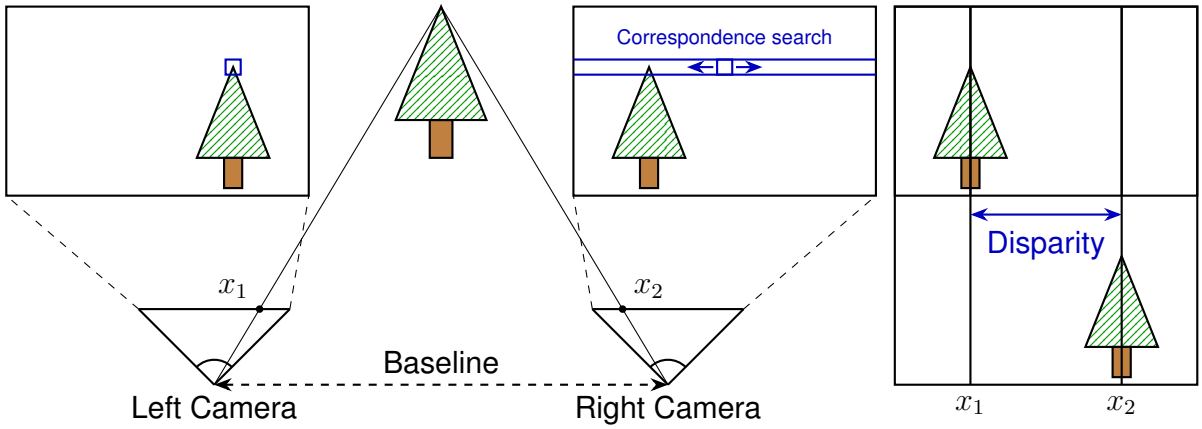


Figure 3.1: Disparity between two views. Disparity is the horizontal pixel difference between corresponding points in rectified stereo images, inversely proportional to depth.

In this context, the core computer-vision task is correspondence estimation: for a point in one image, find the matching point in the other image. Once correspondences are known, depth can be recovered by triangulation.

Consider a 3D point P observed by two cameras centered in C_1 and C_2, its projection in the images will be $\mathbf{p_1}$ and $\mathbf{p_2}$. The points C_1, C_2, and P define what is called an epipolar plane, which intersects the two image planes along epipolar lines l_1 and l_2 (Figure 3.2). The baseline is the translation between camera centers, represented by the segment $\overline{C_1C_2}$. This baseline intersects each image plane at the epipoles $\mathbf{e_1}$ and $\mathbf{e_2}$, and every epipolar line passes through its corresponding epipole. As a result, once $\mathbf{p_1}$ is known, its match in the second image must lie on l_2, reducing correspondence search from 2D to 1D.

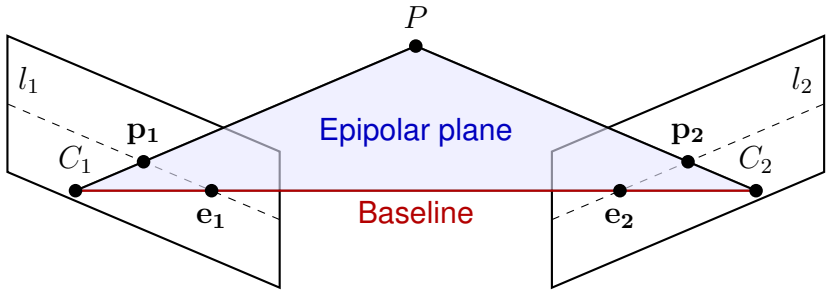


Figure 3.2: Epipolar geometry: the epipolar plane defined by two camera centers C_1, C_2 and a 3D point P intersects each image planes in epipolar lines l_1 and l_2. Given a point $\mathbf{p_1}$, its correspondence must lie on l_2, reducing the search to one dimension.

Algebraically, this relation is encoded by the fundamental matrix $F \in \mathbb{R}^{3\times3}$ for uncalibrated views. It maps an homogenous point $\tilde{p}$ in one image to its epipolar line in the other, i.e., $l_2 = F\tilde{p}_1$ and $l_1 = F^T\tilde{p}_2$. Corresponding points satisfy

$$\tilde{p}_2^T F \tilde{p}_1 = 0,$$

which states that $\tilde{p}_2$ lies on the epipolar line induced by $\tilde{p}_1$.

When camera intrinsics are known, image points are normalized as $\tilde{x}_i = K_i^{-1}\tilde{p}_i$, and the same constraint is expressed by the rank 2 essential matrix $E \in \mathbb{R}^{3\times3}$:

$$\tilde{x}_2^T E \tilde{x}_1 = 0, \qquad E = K_2^T F K_1. \tag{3.1}$$

Unlike F, E lives in normalized camera coordinates and directly encodes relative pose through the cross product $E = \mathbf{t} \times R$ where $\mathbf{t}$ and R are the translation and rotation between the two camera positions. The essential matrix has 5 degrees of freedom: while both rotation and translation contribute 3 degrees of freedom each, the constraint in Equation 3.1 is invariant to scaling of the translation vector. Consequently, translation is recoverable only up to an unknown global scale.

While the epipolar constraint reduces correspondence search to one dimension, the epipolar lines in stereo configurations may be arbitrarily oriented. To exploit this constraint efficiently, stereo images are often rectified so that epipolar lines become horizontal and coincide with image rows, equivalent to a configuration of perfectly aligned cameras as in Figure 3.1. Correspondence search then reduces to a simple horizontal scan along each row, and disparity becomes a direct horizontal pixel difference.

In a rectified stereo pair where u_1 and u_2 are the horizontal coordinates of corresponding points, the disparity is the horizontal coordinate difference $d = |u_1 - u_2|$, and metric depth follows

$$Z = \frac{fB}{d},$$

where f is the focal length in pixels and $B = \|C_2 - C_1\|$ is the baseline length between the two camera centers C_1 and C_2. This inverse relation explains why small disparity errors at large distances can produce significant depth errors, motivating accurate calibration and sub-pixel disparity estimation.

3.1.2 Structure from Motion

Structure from motion (SfM) extends the same epipolar geometry to temporally separated views from a moving camera. Instead of a fixed stereo baseline, the baseline is induced by camera motion, and both scene structure and camera trajectory are estimated jointly from image correspondences.

For two frames I_t and I_{t+1}, matched points satisfy the same normalized epipolar constraint introduced above,

$$\tilde{x}_{t+1}^T E \tilde{x}_t = 0,$$

with $E = [\mathbf{t}]_\times R$ describing relative pose. The key difference from calibrated stereo is that this pose is estimated from data, and translation is recovered only up to an unknown global scale.

SfM pipelines generally follow a sequence of well-defined steps to recover both camera poses and 3D structure. The first step is to find correspondences. For this, sparse features are detected in both images and matched together. Classical methods rely on hand-crafted

descriptors like SIFT [77], SURF [6], ORB [103], or Harris corners [43]. Since there are often many correspondences and the pose only has 5 degrees of freedom, it is often fitted robustly using RANSAC [30] variants, filtering out bad correspondences. Modern approaches increasingly use learned matchers such as SuperPoint [22] and LoFTR [116], which offer improved robustness under extreme viewpoint and illumination changes. Figure 3.3 provides a typical example of tracked correspondences forming the basis of these estimates.

Figure 3.3: Feature tracking across two frames: keypoints are matched to establish correspondences. Correspondences are obtained using LoFTR [116].

This two-view initialization can then be extended to longer sequences through incremental SfM. As new frames are acquired, they are registered to the existing 3D structure using Perspective-n-Point (PnP) [70]. PnP estimates the camera pose for a new image by aligning its 2D detected features with the already registered 3D points. Once the new camera pose is localized, novel features observed in this view are added to continue the process.

As frames are incrementally added, small errors in pose, matching, and triangulation inevitably accumulate, leading to drift. To mitigate this, SfM employs Bundle Adjustment (BA) [120]. While frequently used as a continuous refinement stage in incremental pipelines, BA is fundamentally a standalone optimization framework. It jointly optimizes all camera poses and 3D point coordinates simultaneously by minimizing the total reprojection error, which is the squared distance between the observed 2D image points and the projections of the estimated 3D points into the estimated camera views:

$$\min_{\{R_i,\mathbf{t}_i\},\{P_j\}} \sum_{i,j} \rho\left(\|\pi(R_i,\mathbf{t}_i,P_j) - p_{ij}\|^2\right),$$

where π is the projection function, p_{ij} are observed image coordinates, and ρ is a robust loss (e.g., Huber) for outlier resistance.

SfM still degrades in low-parallax or degenerate settings: pure rotation yields no triangulation baseline, planar or weakly textured scenes produce ambiguous matches, and dynamic objects violate the static-scene assumption. Practical systems therefore combine robust estimators such as RANSAC [30] or MAGSAC [5], careful keyframe selection, and outlier filtering. The reconstruction in Figure 3.4 is representative of the sparse trajectory-and-point-cloud output obtained at the end of that process.

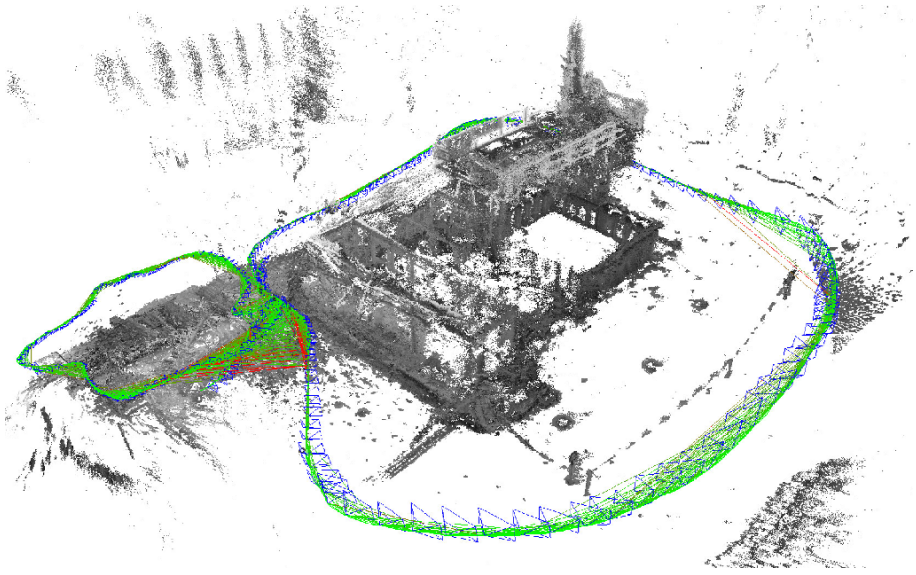


Figure 3.4: Reconstructed camera trajectory and sparse 3D point cloud from a monocular video sequence. Figure taken from LSD-SLAM [26].

Classical SfM pipelines such as COLMAP [107] provide accurate sparse reconstructions but require careful parameter tuning and can be computationally expensive. Modern self-supervised depth and pose methods, which we explore in the following sections, implicitly encode SfM principles within differentiable neural networks. Instead of explicit feature matching and bundle adjustment, these methods learn to predict dense depth and relative pose by optimizing photometric consistency across frames, effectively learning a continuous, dense variant of structure from motion from video data alone.

3.2 Learning Depth and Pose

This section details the core mechanism of self-supervised depth and pose learning: the reprojection of pixels between frames and the use of photometric consistency as a training signal. The foundational work by Godard *et al.* [37] with stereo data and Zhou *et al.* [149] with monocular videos introduced this paradigm, which has since become the standard approach for learning depth from monocular video without ground truth labels. The key insight is that if we correctly predict both the depth map of a reference frame and the relative camera pose to a nearby support frame, we can synthesize how the support frame should appear from the reference viewpoint. By comparing this synthesized view with the actual reference frame, we obtain a photometric error that serves as a self-supervised loss. This creates an autoencoder-like structure: the depth and pose networks encode geometric information, and the warping operation decodes it back into image space, where reconstruction quality can be directly measured. Figure 3.5 summarizes this pipeline before explaining each step in details.

This process consists in three main steps: (1) the reprojection equation is responsible for mapping each pixel in the reference frame to the support frame (2) the inverse warping operation that synthesizes the reference image by transforming the support image, and (3) the loss function that compares the synthesized image to the actual reference frame.

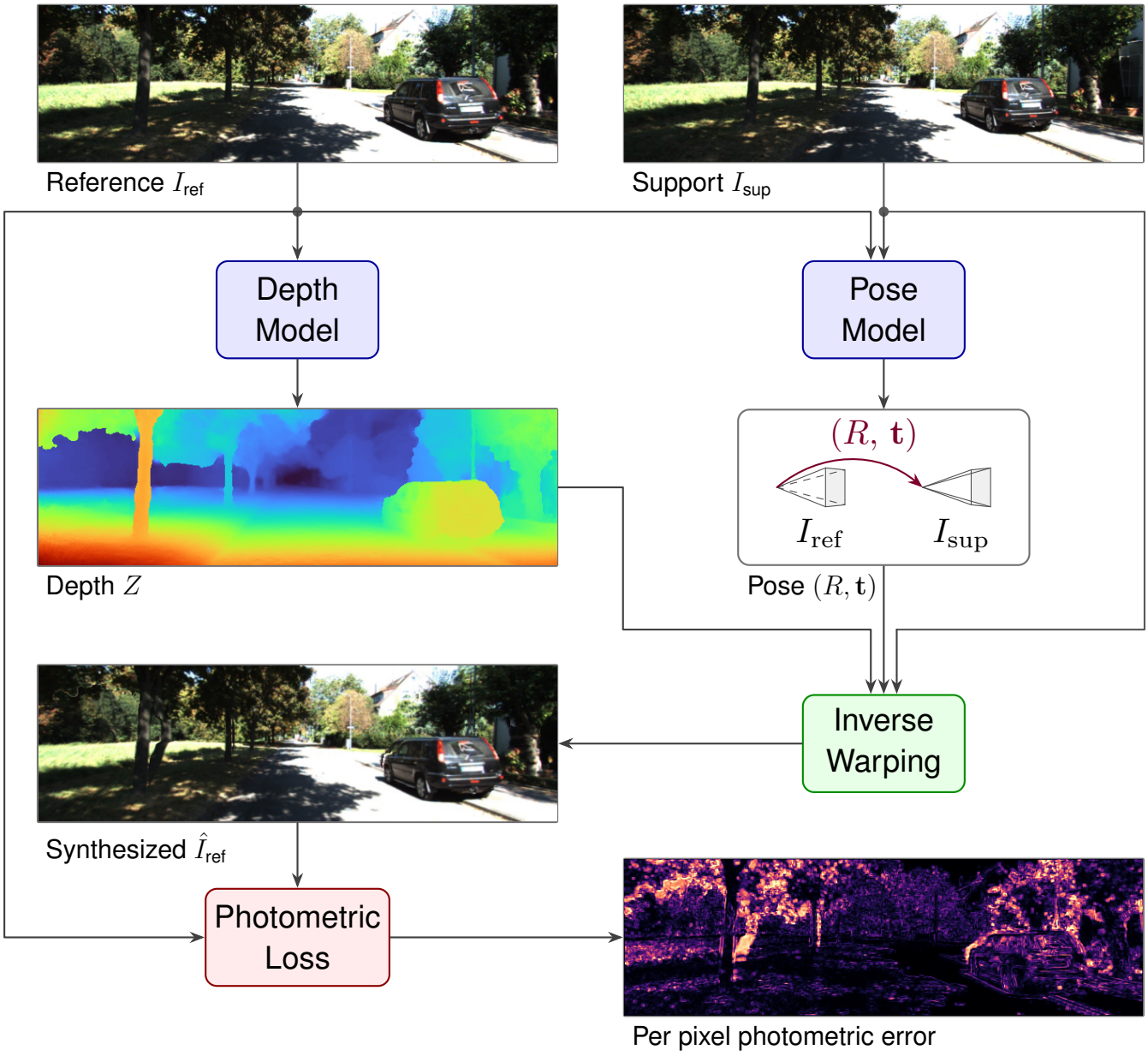


Figure 3.5: Self-supervised learning pipeline: depth and pose networks predict the geometric parameters which are used to warp the support image into the reference frame. The photometric loss compares the warped image to the reference frame to optimize these models by backpropagation.

3.2.1 Reprojection Equation

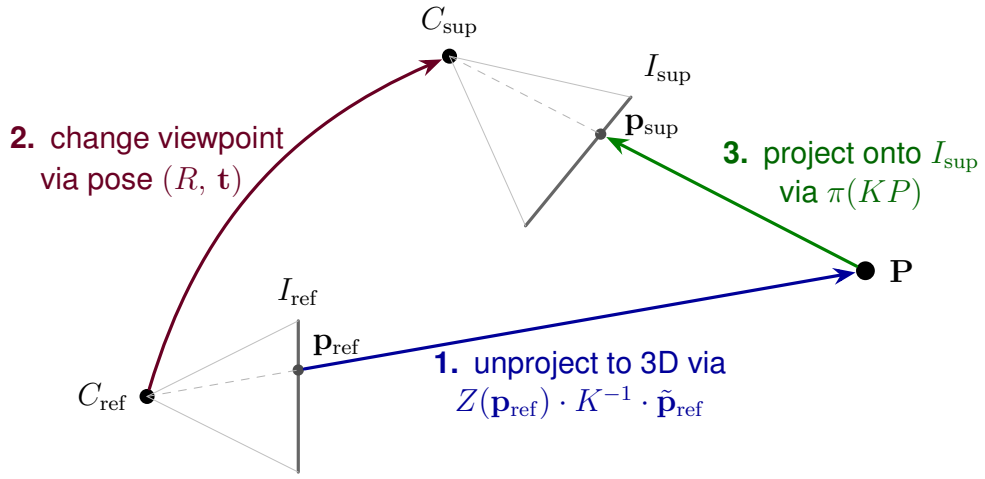


Figure 3.6: Reprojection: a pixel in the reference frame is unprojected using depth, transformed by relative pose, and projected into the support frame.

The reprojection operation is the step that determines where a pixel visible in the reference frame I_{ref} would be placed in the support frame I_{sup} using predicted depth and relative pose. This process is illustrated in Figure 3.6.

The first step is to lift a pixel into 3D. Consider a pixel $\tilde{\mathbf{p}}_{\text{ref}} = (u, v, 1)$ in the reference frame with predicted depth $Z(\tilde{\mathbf{p}}_{\text{ref}})$. Using the camera intrinsics matrix K, we unproject this pixel to a 3D point $\mathbf{P}_{\text{ref}}$ in the reference camera coordinate frame:

$$\mathbf{P}_{\text{ref}} = Z(\mathbf{p}_{\text{ref}}) \cdot K^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} X \\ Y \\ Z \end{bmatrix},$$

This operation lifts the 2D pixel into 3D space, with the depth value determining how far the point lies along the viewing ray. Next, we transform this 3D point from the reference camera frame to the support camera frame using the predicted relative pose $(R, \mathbf{t})$, where R is the rotation matrix and $\mathbf{t}$ the translation vector:

$$\mathbf{P}_{\text{sup}} = R\mathbf{P}_{\text{ref}} + \mathbf{t}.$$

This rigid transformation accounts for camera motion between the two frames, under the assumption that the scene is static. Finally, we project the transformed 3D point back onto the support image plane using the camera intrinsics:

$$\mathbf{p}_{\text{sup}} = \pi(K\mathbf{P}_{\text{sup}}),$$

where π is the perspective projection that divides by depth:

$$\mathbf{p}_{\text{sup}} = \begin{bmatrix} u_{\text{sup}} \\ v_{\text{sup}} \end{bmatrix} = \begin{bmatrix} f_x X_{\text{sup}}/Z_{\text{sup}} + c_x \\ f_y Y_{\text{sup}}/Z_{\text{sup}} + c_y \end{bmatrix}.$$

The resulting coordinates $(u_{\text{sup}}, v_{\text{sup}})$ indicate where the reference pixel should appear in the support frame.

Combining all steps, the full reprojection mapping from reference to support coordinates is:

$$\mathbf{p}_{\text{sup}} = \pi \left(K \left[R \left(Z(\mathbf{p}_{\text{ref}}) K^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \right) + \mathbf{t} \right] \right).$$

3.2.2 Reprojection as Rigid Optical Flow

The reprojection equation can be interpreted as computing a special case of optical flow where the flow field is constrained by rigid-body geometry. General optical flow methods estimate a dense, per-pixel 2D displacement field between consecutive frames, capturing the apparent motion of brightness patterns in the image. These unconstrained flow fields allow arbitrary per-pixel motion to accommodate independently moving objects and deformable surfaces. In contrast, the reprojection operation computes *rigid optical flow*: a geometrically-structured flow field arising purely from camera motion over a static scene, where all pixel displacements are determined by depth and pose rather than estimated independently.

Given a reference pixel $\mathbf{p}_{\text{ref}}$ with depth $Z(\mathbf{p}_{\text{ref}})$ and relative pose $(R, \mathbf{t})$, the reprojection maps it to support coordinates $\mathbf{p}_{\text{supp}}$. We can express this as a flow vector $\mathbf{f}(\mathbf{p}_{\text{ref}})$:

$$\mathbf{f}(\mathbf{p}_{\text{ref}}) = \mathbf{p}_{\text{sup}} - \mathbf{p}_{\text{ref}}.$$

Combining the reprojection equation with this flow definition, the flow can be written explicitly as

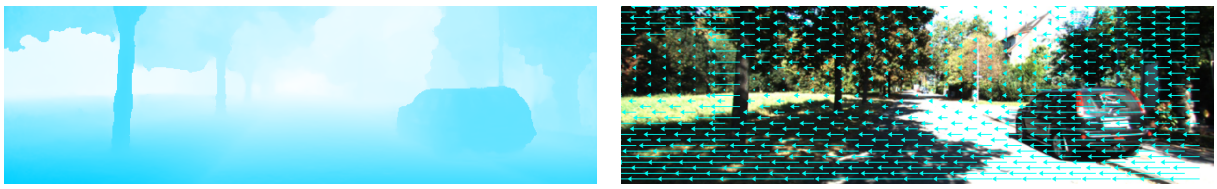
$$\mathbf{f}(\mathbf{p}_{\text{ref}}) = \pi\left(K\left[R\left(Z(\mathbf{p}_{\text{ref}})K^{-1}\tilde{\mathbf{p}}_{\text{ref}}\right) + \mathbf{t}\right]\right) - \mathbf{p}_{\text{ref}},$$

showing that flow is a nonlinear function of both depth $Z(\mathbf{p}_{\text{ref}})$ and pose $(R, \mathbf{t})$, with a fundamental coupling between the two.

The rigid flow field exhibits characteristic structure. Consider the simplified case of pure translation $\mathbf{t}$ with no rotation $(R = I)$. The 3D point transforms as $\mathbf{P}_{\text{sup}} = \mathbf{P}_{\text{ref}} + \mathbf{t}$, and the resulting flow magnitude is approximately

$$\|\mathbf{f}(\mathbf{p}_{\text{ref}})\| \propto \frac{\|\mathbf{t}\|}{Z(\mathbf{p}_{\text{ref}})}.$$

This is the motion parallax relationship: flow magnitude is inversely proportional to depth. Nearby objects (small Z) move more in the image than distant objects (large Z) under the same camera translation, and this geometric cue is what allows self-supervised networks to disentangle depth from motion. In Figure 3.7, the color view in Figure 3.7a emphasizes displacement magnitude, while Figure 3.7b shows the same pattern as explicit vectors.



(a) Flow colored by magnitude (b) Flow vectors (arrows)

Figure 3.7: Rigid optical flow field induced by a pure lateral translation (stereo cameras). Nearby objects exhibit larger displacements due to motion parallax.

For general motion including rotation, the flow decomposes into a translational component $\mathbf{f}_t$ proportional to $\mathbf{t}/Z$ that depends strongly on depth, and a rotational component $\mathbf{f}_r$ that is depth-independent. This means that the depth is primarily constrained by the translational component, while accurate rotation estimates remain essential for correct reprojection. The pose estimation networks automatically learn to separate these components during training. Since vehicles mostly move in the forward direction, we also provide

an example of this case in Figure 3.8: where the hue corresponds to flow direction and saturation to its magnitude, especially visible in Figure 3.8a. Notice how this magnitude changes between the trees.

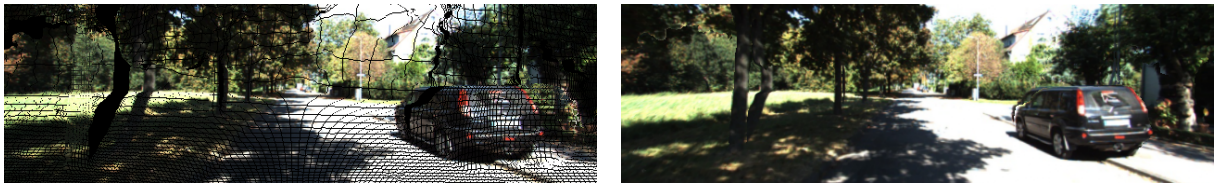
(a) Flow colored by magnitude and direction (b) Flow vectors (arrows)

Figure 3.8: Rigid optical flow field induced by forward camera motion. Hue indicates flow direction while saturation indicates magnitude.

3.2.3 Inverse Warping

The goal of warping is to synthesize how the support frame would appear from the reference camera's viewpoint. Critically, we want to warp the support frame onto the reference frame grid, ensuring that every reference pixel receives a color value. This is achieved through *inverse warping* (also called backward warping).

There are two conceptually different approaches. Forward warping (where each reference pixel is projected to the support frame and its color deposited there) suffers from fundamental problems: multiple reference pixels may project to the same location, creating collisions that require conflict resolution and some pixels may receive no contributions, leaving holes. Those failure modes are easy to spot in Figure 3.9a. Inverse warping solves these issues by working in the opposite direction: for each pixel on the reference grid, it computes where that pixel should be in the support frame and samples the color from there. This guarantees that every reference pixel receives exactly one color value, with no holes or undefined regions, and produces a clean, differentiable sampling operation, which is why the result in Figure 3.9b is much better behaved.



(a) Forward warping: holes and collisions arise from many-to-one and one-to-zero mappings (b) Inverse warping: every target pixel is filled by sampling from the source image

Figure 3.9: Comparison of forward vs. inverse warping: forward warping (a) can create holes and collisions, while inverse warping (b) ensures every reference pixel is filled by sampling from the support image, but can duplicate occluding objects.

The inverse warping operation produces a synthesized image $\hat{I}_{\text{ref}}$ that represents the support frame warped to the reference viewpoint. For each pixel location $\mathbf{p}_{\text{ref}}$ in the reference frame grid, the predicted depth $Z(\mathbf{p}_{\text{ref}})$ and relative pose $(R, \mathbf{t})$ determine where this 3D point appears in the support frame via the reprojection equation. The color at that computed location $\mathbf{p}_{\text{supp}}$ is sampled from the support image and assigned to position

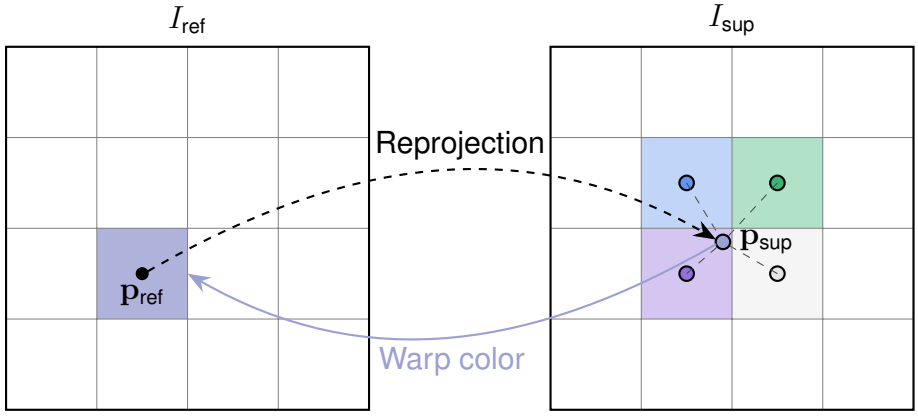


Figure 3.10: Inverse warping mapping: a target pixel position $\mathbf{p}_{\text{ref}}$ in the reference frame is projected to the support frame, landing at sub-pixel coordinates $\mathbf{p}_{\text{sup}}$. Nearby pixels are interpolated to get the color and warped back to fill the target pixel color.

$\mathbf{p}_{\text{ref}}$ in the synthesized image. This process is visualized in Figure 3.10. Mathematically:

$$\hat{I}_{\text{ref}}(\mathbf{p}_{\text{ref}}) = I_{\text{sup}}\Big(\text{reproject}(\mathbf{p}_{\text{ref}}, Z(\mathbf{p}_{\text{ref}}), R, \mathbf{t})\Big),$$

where reproject$(\cdot)$ encapsulates the full geometric transformation from reference to support coordinates. This warped image $\hat{I}_{\text{ref}}$ can be directly compared to the actual reference frame I_{ref} to compute the photometric loss, as both are now defined on the same pixel grid.

Depth plays a critical role in determining the warp field. Pixels corresponding to nearby objects (small depth) undergo larger displacements than distant objects (large depth) under the same camera motion. This is motion parallax. Incorrect depth predictions produce misaligned warps: if depth is overestimated, the warp displacement is too small; if underestimated, too large. The photometric error gradient with respect to depth directly encodes this information, allowing the depth network to learn correct geometric structure.

3.2.4 Differentiable Sampling

The reprojection equation computes where to sample from the support image, but these coordinates $\mathbf{p}_{\text{sup}}$ are typically non-integer. Since images are discrete grids, we need an interpolation scheme to obtain color values at fractional coordinates, and the interpolation must be differentiable to allow backpropagation of gradients through the warping operation to the depth and pose networks.

Consider a reference pixel $(u, v) = (10, 20)$ that, after reprojection, maps to support coordinates $(u', v') = (15.7, 23.3)$. The support image only has integer pixel locations, so we cannot directly read $I_{\text{sup}}[15.7, 23.3]$ and must interpolate from the surrounding pixels. Given a sampling location (u', v'), we define the floor and ceiling functions as $\lfloor x \rfloor$ and $\lceil x \rceil$ respectively. Let $\alpha = u' - \lfloor u' \rfloor$ and $\beta = v' - \lfloor v' \rfloor$ be the fractional offsets in $[0, 1)$. Bilinear interpolation computes a weighted average of the four neighboring integer pixels as shown in Figure 3.11, and expressed in matrix form as:

$$I(u', v') = \begin{bmatrix} 1-\alpha & \alpha \end{bmatrix} \begin{bmatrix} I(\lfloor u' \rfloor, \lfloor v' \rfloor) & I(\lfloor u' \rfloor, \lceil v' \rceil) \\ I(\lceil u' \rceil, \lfloor v' \rfloor) & I(\lceil u' \rceil, \lceil v' \rceil) \end{bmatrix} \begin{bmatrix} 1-\beta \\ \beta \end{bmatrix}.$$

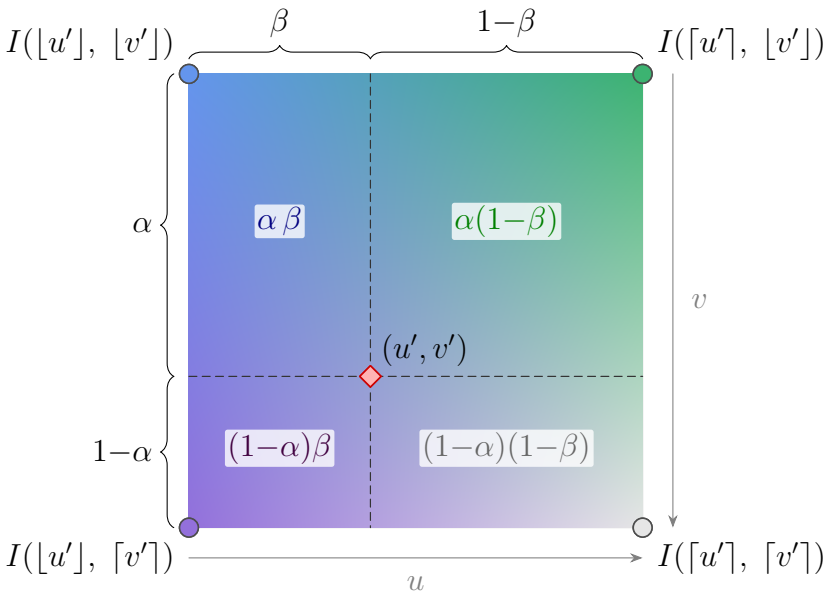


Figure 3.11: Bilinear interpolation: a fractional sampling location (u', v') is resolved by blending the four nearest integer-grid pixels. The weight assigned to each neighbour equals the area of the opposing rectangle, determined by the fractional offsets α (row) and β (column).

Bilinear interpolation is differentiable with respect to the sampling coordinates (u', v'). The gradient of the interpolated color with respect to the sampling location is:

$$\frac{\partial I(u', v')}{\partial u'} = (1-\beta)\left[I(\lceil u' \rceil, \lfloor v' \rfloor) - I(\lfloor u' \rfloor, \lfloor v' \rfloor)\right] + \beta\left[I(\lceil u' \rceil, \lceil v' \rceil) - I(\lfloor u' \rfloor, \lceil v' \rceil)\right],$$

with a similar expression for $\partial I / \partial v'$. During backpropagation, when minimizing the photometric loss, gradients flow through the chain:

$$\text{Loss} \rightarrow \text{Color difference} \rightarrow \text{Sampling location } (u', v') \rightarrow \text{Depth } Z \text{ and Pose } (R, \mathbf{t}).$$

A small change in depth $Z(u, v)$ shifts the reprojected location (u', v'), which changes the interpolated color via the gradients above, which in turn affects the photometric error. This gradient path tells the depth network how to adjust each pixel's depth to improve alignment. The tight coupling between geometry and appearance is what makes self-supervised depth learning possible: the photometric loss carries geometric information encoded in the parallax-induced displacements, and the network learns depth not by direct supervision but by discovering the geometric configuration that best explains the observed photometric consistency across frames.

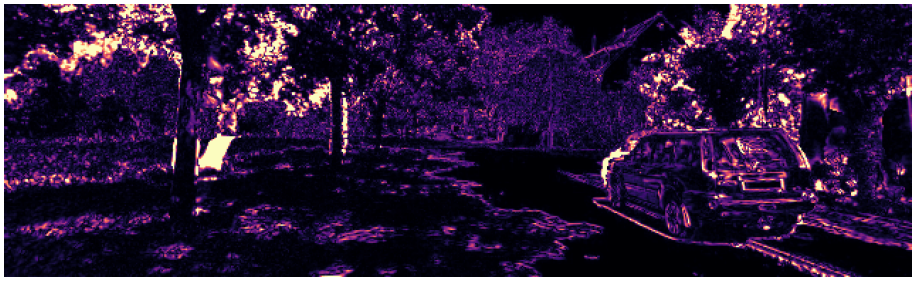
In practice, the entire warping and sampling pipeline is implemented as a differentiable operation using grid sampling in frameworks such as PyTorch or TensorFlow, with efficient GPU-accelerated implementations. Pixels that reproject outside the support image bounds are masked out from the loss computation as they have no valid correspondence. Occlusions (where a point visible in the reference frame is hidden in the support frame) also require special handling, which we address in the next section on loss functions and masking strategies.

3.3 Loss Functions

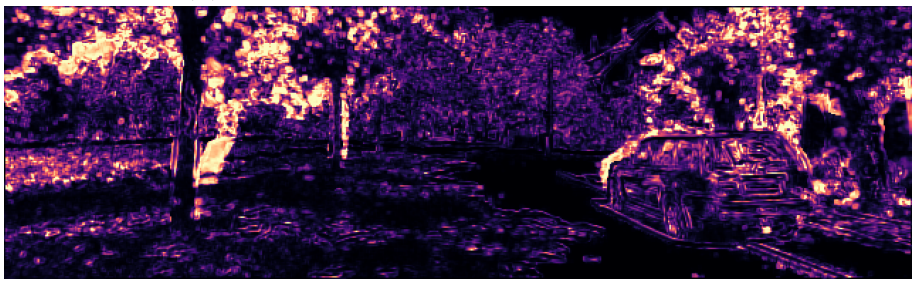
The overall training objective for self-supervised depth and pose estimation combines multiple terms that encourage photometric consistency, geometric regularity, and robust handling of violations to the static scene assumption. This section details the photometric reconstruction loss, smoothness regularization, the auto-masking strategies introduced in Monodepth2 [38], and multi-scale training.

3.3.1 Photometric Loss: SSIM and L1

(a) L_2 loss: sensitive to large errors



(b) L_1 loss: linear penalty on misalignment



(c) SSIM loss: structural similarity

Figure 3.12: Comparison of L_2, L_1, and SSIM losses: SSIM captures structural similarity and is more robust to illumination changes, while L_1 handles outliers better than L_2.

The core self-supervised signal comes from comparing the warped support frame $\hat{I}_{\text{ref}}$ with the actual reference frame I_{ref}. The most natural choice would be the L_2 loss $\|I_{\text{ref}} - \hat{I}_{\text{ref}}\|^2$, but it is sensitive to outliers: large errors from occlusions, moving objects, or lighting changes contribute quadratically, dominating the gradient signal and potentially destabilizing training. A single badly misaligned pixel can produce disproportionately large loss values, causing the network to overfit to violations of the static scene assumption rather than learning correct geometry.

The L_1 loss provides better outlier robustness by growing linearly rather than quadrat-

ically with error magnitude:

$$\mathcal{L}_{L_1} = |I_{\text{ref}} - \hat{I}_{\text{ref}}|$$

This reduces the influence of large misalignments from dynamic objects, occlusions, and other violations of brightness constancy, making training more stable. However, L_1 measures only pixel-wise intensity differences. The Structural Similarity (SSIM) [133] adds more information by capturing perceptual similarity by comparing local image structure: luminance, contrast, and structural patterns. For a local window around pixel (u, v), SSIM is defined as:

$$\text{SSIM}(I_{\text{ref}}, \hat{I}_{\text{ref}}) = \frac{(2\mu_I\mu_{\hat{I}} + C_1)(2\sigma_{I\hat{I}} + C_2)}{(\mu_I^2 + \mu_{\hat{I}}^2 + C_1)(\sigma_I^2 + \sigma_{\hat{I}}^2 + C_2)},$$

where μ denotes mean, σ^2 variance, $\sigma_{I\hat{I}}$ covariance, and C_1, C_2 are small stabilization constants. SSIM ranges from -1 to 1, with 1 indicating perfect similarity. The SSIM loss is then:

$$\mathcal{L}_{\text{SSIM}} = \frac{1 - \text{SSIM}(I_{\text{ref}}, \hat{I}_{\text{ref}})}{2}.$$

SSIM is particularly effective in textured regions, where it measures whether local patterns such as edges and textures are preserved rather than just absolute intensity values. This makes it more robust to illumination changes and provides stronger gradients in high-texture areas where geometric correspondences are most reliable.

In practice, SSIM and L_1 are combined with a weighting parameter α (typically $\alpha = 0.85$):

$$\mathcal{L}_{\text{photo}} = \alpha\mathcal{L}_{\text{SSIM}} + (1 - \alpha)\mathcal{L}_{L_1}.$$

This hybrid approach leverages SSIM's texture sensitivity for well-textured regions while L_1 provides a stable baseline in low-texture areas. Empirically, this combination significantly improves depth accuracy compared to using either loss alone. Figure 3.12 makes the contrast concrete: Figure 3.12a highlights the sensitivity of L_2 to large errors, Figure 3.12b shows the more robust linear penalty, and Figure 3.12c focuses on structural differences.

3.3.2 Smoothness Regularization

Without additional constraints, the photometric loss alone can lead to degenerate solutions or noisy depth maps. Smoothness regularization encourages locally smooth depth predictions, leveraging the prior that natural scenes typically have piecewise-smooth surfaces. A naive smoothness penalty $\|\nabla D\|$ would be very bad at object boundaries, so instead an edge-aware L_1 smoothness term is used that allows sharp depth changes where there are corresponding image edges:

$$\mathcal{L}_{\text{smooth}} = |\partial_x D|e^{-|\partial_x I|} + |\partial_y D|e^{-|\partial_y I|},$$

where ∂_x and ∂_y denote spatial derivatives. The exponential weighting $e^{-|\partial_x I|}$ reduces the smoothness penalty where image gradients are large, allowing depth discontinuities to align with visual boundaries.

An important implementation detail is that depth must be normalized before applying the smoothness loss. In practice, depth is divided by its mean:

$$\mathcal{L}_{\text{smooth}} = |\partial_x \bar{D}|e^{-|\partial_x I|} + |\partial_y \bar{D}|e^{-|\partial_y I|}, \quad \text{where } \bar{D} = \frac{D}{\text{mean}(D)}.$$

Without this normalization, the network could minimize the smoothness loss simply by predicting depth values close to zero, since smaller depth values naturally have smaller gradients. Normalizing by the mean makes the loss scale-invariant, ensuring that the penalty depends only on relative depth variation rather than absolute magnitude. This is also consistent with the scale ambiguity inherent in self-supervised monocular depth estimation: since absolute scale is arbitrary, the smoothness regularization should not bias toward any particular scale.

The choice of norm affects the resulting depth maps. The default L_1 penalty $|\nabla D|$ assumes a piecewise-constant depth prior that preserves sharp discontinuities at object boundaries, but can produce staircase artifacts where depth maps exhibit flat plateaus separated by sharp jumps. The alternative L_2 penalty $\|\nabla D\|^2$ assumes a piecewise-linear prior that encourages smoothly varying depth with constant slope, producing more natural-looking depth on planar or gently curved surfaces but with less sharp boundaries.

The smoothness loss implicitly assumes that depth varies slowly in regions of uniform color or texture. This is generally valid for rigid objects with continuous surfaces but can fail for thin structures such as poles and wires that have depth discontinuities without strong image edges, for textureless surfaces where depth varies without corresponding image gradient cues, and for painted textures on flat surfaces where image edges do not correspond to depth discontinuities. As noted by Spencer et al. [115], the smoothness term introduces systematic biases: while it improves image-based metrics, it can degrade geometric accuracy when evaluated using 3D point clouds, trading off geometric correctness for visual plausibility. This bias is particularly problematic in applications requiring accurate 3D reconstruction rather than perceptually pleasing depth visualizations.

The smoothness term is weighted by a hyperparameter λ_{smooth} (typically 10^{-3} to 10^{-4}) and added to the total loss:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{photo}} + \lambda_{\text{smooth}}\mathcal{L}_{\text{smooth}}.$$

Careful tuning of λ_{smooth} balances regularization against over-smoothing: too large and fine details are lost; too small and depth maps become noisy and unstable. Figure 3.13 summarizes the overall effect. Even if edges have high smoothness loss, the gradient in these regions is small due to the exponential weighting. Since the L1 norm has constant derivative (-1 or 1), the gradient is proportional to the exponential weighting, which is small at edges and large in low-gradient regions.

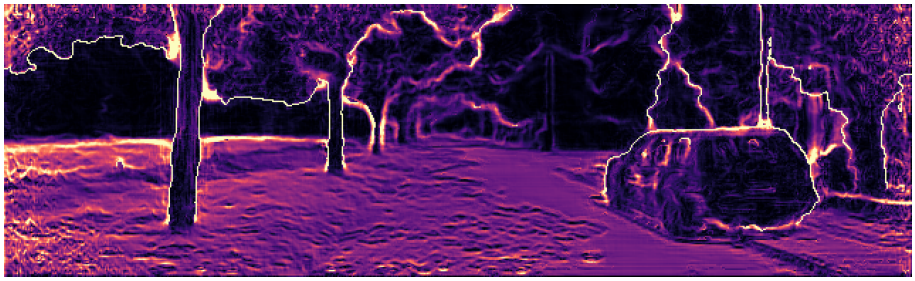


Figure 3.13: Smoothness loss visualization: the edge-aware weighting allows depth discontinuities to align with image edges, while encouraging smooth depth in low-gradient regions. Normalization by mean depth ensures scale-invariance and prevents trivial solutions.

3.3.3 Auto-Masking and Minimum Reprojection

A major challenge in self-supervised depth learning is handling violations of the static scene assumption: dynamic objects, occlusions, and low-texture regions produce unreliable photometric correspondences. Monodepth2 [38] introduced two key improvements (minimum reprojection loss and per-pixel auto-masking) that significantly enhance robustness by systematically removing these outliers.

Instead of using a single support frame, the method uses two support frames: one before and one after the reference frame (frames $t-1$ and $t+1$ for reference frame t). This temporal redundancy allows the method to robustly handle outliers by selecting the most reliable photometric correspondence at each pixel. For each support frame s, a warped image $\hat{I}^s_{\text{ref}}$ and photometric loss $\mathcal{L}^s_{\text{photo}}$ are computed. The minimum reprojection loss selects the lowest loss at each pixel:

$$\mathcal{L}^{\min}_{\text{photo}}(u,v) = \min_{s\in\{t-1,t+1\}} \mathcal{L}^s_{\text{photo}}(u,v).$$

This per-pixel minimum operation automatically handles occlusions through temporal redundancy. A point visible in the reference frame may be occluded in one support frame, but since the camera poses between reference and past/future frames are inverse to each other, occluding surfaces that block the view from one direction do not block it from the opposite direction. A point occluded in the past frame is unlikely to be occluded in the future frame, so the minimum operation naturally selects the valid correspondence, as shown by the two source-frame losses in Figure 3.14a and Figure 3.14b and their per-pixel minimum in Figure 3.14c.

In driving scenarios, large portions of the image (sky, distant background) may not move across frames. For these pixels, the photometric loss between the reference and support frames is already small even with incorrect depth predictions, creating a degenerate solution where the network can minimize loss by predicting infinite depth (no parallax). To prevent this, Monodepth2 introduces per-pixel auto-masking based on the identity warp. For each pixel, the photometric loss of the warped image is compared with the loss of using the support frame directly (the identity):

$$\mathcal{L}^s_{\text{identity}} = \mathcal{L}_{\text{photo}}(I_{\text{ref}}, I_s).$$

A binary mask is then computed:

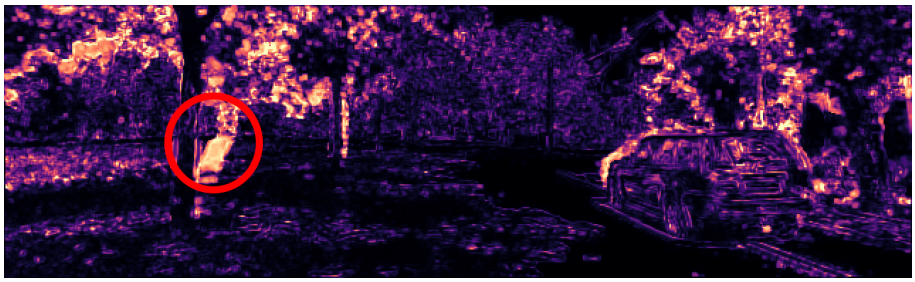
$$\mathcal{M} = [\mathcal{L}^{\min}_{\text{photo}} < \mathcal{L}^{\min}_{\text{identity}}],$$

Only pixels where the geometric warp outperforms the identity are included in the loss, effectively creating a minimum threshold that the predictions must beat: the network must produce better alignment through geometric reasoning than simply assuming no motion.

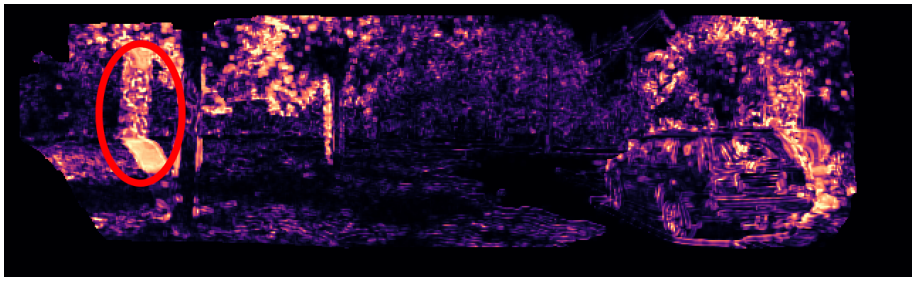
The complete photometric loss with auto-masking is:

$$\mathcal{L}^{\text{masked}}_{\text{photo}} = \frac{1}{|\mathcal{M}|}\sum_{u,v} \mathcal{M}(u,v)\cdot\mathcal{L}^{\min}_{\text{photo}}(u,v),$$

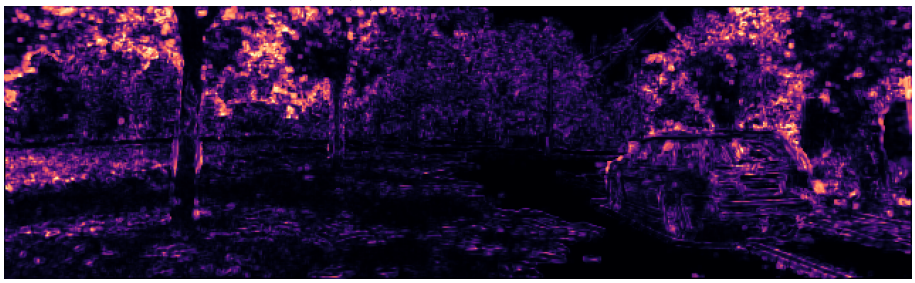
where $|\mathcal{M}|$ is the number of valid (unmasked) pixels. This ensures that occluded regions are handled via minimum reprojection, stationary or low-motion regions are masked out, and the network focuses on pixels with reliable geometric correspondences. These improvements from Monodepth2 are now standard practice and provide substantial gains



(a) Loss from I_{t-1}



(b) Loss from I_{t+1}



(c) Per-pixel minimum

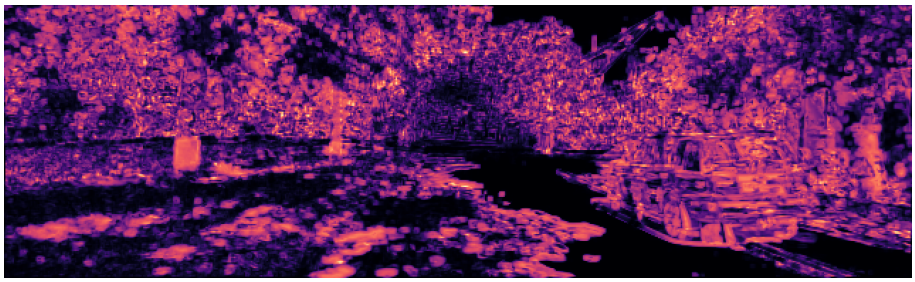
Figure 3.14: Minimum reprojection with two support frames. The circled areas show points occluded in the reference frame but still visible in the supports. They have high error because they take the color from the occluder. The per-pixel minimum (c) selects the lowest error at each location, effectively handling occlusions without explicit masking.

in robustness and accuracy, particularly in challenging real-world driving scenarios. Figure 3.15 shows the full comparison, with the identity loss in Figure 3.15a, the warped photometric loss in Figure 3.15b, and the final keep-mask in Figure 3.15c.

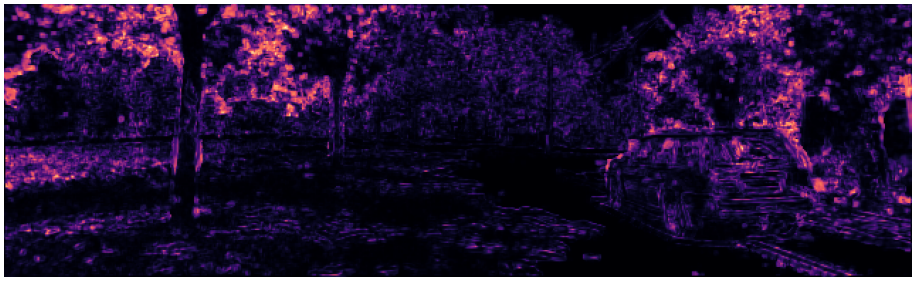
3.3.4 Multi-Scale Training

Neural network-based depth estimation typically uses encoder-decoder architectures that produce predictions at multiple resolutions. Multi-scale training leverages these intermediate outputs to improve convergence and generalization. Training only at the finest resolution can lead to convergence in local minima of the highly non-convex photometric landscape, large pixel displacements that move features outside their receptive fields under significant camera motion, and predictions that lack global scene context. Multi-scale training provides a coarse-to-fine optimization strategy: coarse scales capture global structure and large displacements, while fine scales refine details.

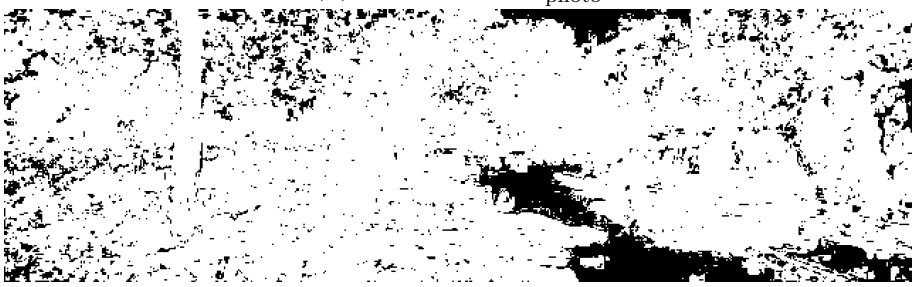
Given scales s where 0 is the finest scale, the photometric and smoothness losses are



(a) Identity loss $\mathcal{L}_{\text{identity}}$



(b) Warped loss $\mathcal{L}^{\text{min}}_{\text{photo}}$



(c) Auto-mask

Figure 3.15: Auto-masking visualization: the mask (c) retains pixels where geometric warping (b) produces lower error than the identity (a), discarding stationary regions and focusing training on geometrically informative correspondences.

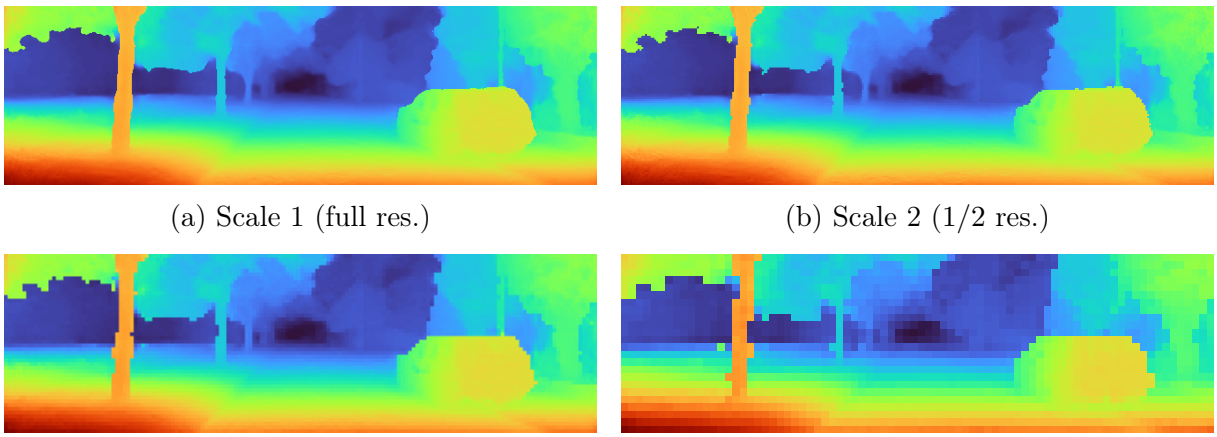
computed at each scale and combined:

$$\mathcal{L}_{\text{total}} = \sum_{s=1}^{S} \left(\mathcal{L}^{s}_{\text{photo}} + \lambda^{s}_{\text{smooth}} \mathcal{L}^{s}_{\text{smooth}} \right),$$

where the photometric loss is weighted equally across scales and the smoothness weight $\lambda^{s}_{\text{smooth}}$ decreases with resolution (e.g., $\lambda^{s}_{\text{smooth}} = \lambda_{\text{smooth}}/2^{s}$) since coarser scales are already more blurry. Each scale contributes to the gradient signal, with coarser scales providing global guidance and finer scales capturing details. This accelerates convergence by providing stronger gradients early in training when fine-scale predictions are still poor, and improves generalization by learning depth representations that are consistent across resolutions.

In practice, depth networks typically produce four scales (original resolution down to 1/8 resolution). To compute photometric losses at each scale, two strategies exist: downscaling the input images to each coarse resolution before warping, which is cheaper but loses fine-scale details; or upscaling the depth predictions from coarser scales while keeping images at their original resolution, which preserves image details and typically provides a better learning signal at higher computational cost. Modern implementations

usually prefer upscaling depth predictions despite the additional cost. Figure 3.16 shows a typical four-scale pyramid, starting from the full-resolution prediction in Figure 3.16a and then moving through the coarser outputs in Figure 3.16b, Figure 3.16c and Figure 3.16d.



(a) Scale 1 (full res.)

(b) Scale 2 (1/2 res.)

(c) Scale 3 (1/4 res.)

(d) Scale 4 (1/8 res.)

Figure 3.16: Multi-scale depth predictions: the network produces depth at four resolutions, from full resolution (a) to 1/8 resolution (d). Coarser scales provide global structural guidance while finer scales capture fine boundary details.

3.3.5 Complete Loss Function

Combining all components, the complete self-supervised depth and pose training objective is:

$$\mathcal{L} = \sum_{s=0}^{S} \left(\mathcal{L}_{\text{photo}}^{\text{masked},s} + \lambda_{\text{smooth}}^{s} \mathcal{L}_{\text{smooth}}^{s} \right),$$

where $\mathcal{L}_{\text{photo}}^{\text{masked},s}$ includes the SSIM+L_1 combination with minimum reprojection over multiple frames and identity-based auto-masking, $\mathcal{L}_{\text{smooth}}^{s}$ is the edge-aware smoothness regularization with scale-dependent weight $\lambda_{\text{smooth}}^{s}$, and the sum runs over S scales (typically four, from original to 1/8 resolution). This loss function combines the different elements of self-supervised depth learning, balancing photometric fidelity, geometric regularity, and robustness to real-world challenges. Despite its complexity, each component addresses specific failure modes and contributes measurably to final performance.

3.4 Conclusion

This chapter has detailed the foundational principles and practical machinery of self-supervised monocular depth estimation. By reformulating classical structure from motion into a differentiable learning framework, it is possible to train deep neural networks to predict dense depth and camera ego-motion using only raw video sequences as supervision. The core mechanism driving this process is photometric consistency: lifting pixels into 3D using the predicted depth, transforming them via the predicted relative pose, and projecting them into adjacent frames to synthesize novel views.

To make this approach robust in practice, several carefully designed components are necessary. We presented the inverse warping operation and differentiable sampling that

enable end-to-end gradient flow. We also detailed the loss function required to guide the networks toward geometrically meaningful solutions. This includes the combination of SSIM and L_1 penalties for robust photometric comparison, edge-aware smoothness regularization to enforce surface continuity, and critical auto-masking strategies to handle occlusions and dynamic objects that violate the static scene assumption.

While this self-supervised paradigm successfully recovers the 3D structure of a scene without expensive ground-truth annotations, it suffers from a fundamental geometric limitation: scale ambiguity. Because depth and translation are inherently coupled in the reprojection equations, the entire scene and the camera motion can be arbitrarily scaled without affecting the photometric error. Consequently, the networks learn to predict depth only up to an unknown, arbitrary scale factor. The next chapter explores this scale ambiguity problem in detail and discusses methods to recover true absolute scale.

Chapter 4

Monocular Metric Depth Estimation

Depth estimation from monocular vision has made remarkable progress in recent years, with neural networks achieving impressive results on relative depth prediction. However, many practical applications require not just knowing that one object is farther than another, but precisely how far each object is from the camera—this is the problem of metric depth estimation.

The challenge of obtaining metric depth from a single camera is fundamental. Unlike stereo vision or LiDAR, which have inherent distance measurements through triangulation or time-of-flight, monocular vision suffers from scale ambiguity: a small object nearby and a large object far away can produce identical images. This ambiguity persists even when multiple views are available through structure-from-motion, as the entire reconstruction can be uniformly scaled without violating geometric constraints.

For autonomous systems such as self-driving vehicles and mobile robots, metric depth is crucial for safe decision-making. Knowing that a pedestrian is "closer than the background" is insufficient, the system needs to know whether the pedestrian is 3 meters or 30 meters away to determine appropriate braking distances and trajectory planning. This requirement motivates the development of methods that can recover absolute scale from monocular observations.

This chapter explores the problem of metric depth estimation through three interconnected perspectives. First, we examine the fundamental scale ambiguity problem and how it manifests in both classical geometry-based methods and modern learning approaches. Second, we survey various sources of information, *scale clues*, that can be leveraged to resolve this ambiguity, with particular emphasis on understanding how neural networks implicitly learn to use these cues. Finally, we present our ground constraint method, a self-supervised approach that integrates ground plane priors to achieve metric depth estimation without requiring expensive depth sensors.

The chapter is organized into three main parts. Section 4.1 formalizes the scale ambiguity problem, explaining why monocular depth is inherently ambiguous and how this affects both structure-from-motion and neural network approaches. It also discusses scale-invariant and shift-invariant loss functions that enable learning despite this ambiguity. Section 4.2 examines various scale clues that can resolve the ambiguity, including ground plane geometry, object size priors, and camera parameters. Special attention is given to Van Dijk and De Croon's influential study on how neural networks actually perceive depth, which revealed the dominance of ground contact cues and the resulting sensitivity to camera pose changes. It also reviews recent supervised methods that successfully integrate ground priors. Finally, Section 4.3 presents the ground constraint formulation,

which extends ground-prior integration to the self-supervised regime. Through novel loss functions and training strategies, this method demonstrates that metric depth can be learned from unlabeled monocular video when camera parameters are known, achieving performance competitive with supervised methods.

4.1 The Scale Ambiguity Problem

Monocular depth estimation, whether performed through classical structure-from-motion (SfM) or modern neural network approaches, faces a fundamental challenge: the inherent ambiguity in determining the absolute scale of a scene. This section explores the mathematical foundations of this problem and how it manifests in both geometric and learning-based methods.

4.1.1 Scale Ambiguity in Monocular Vision

The scale ambiguity problem is a well-documented limitation in monocular computer vision that has profound implications for depth estimation. At its core, this issue arises because a single image may correspond to various 3D scenes that differ only by a global scale factor.

The projection ambiguity

Consider a point $P = (X, Y, Z)$ in 3D space being projected onto an image plane. The fundamental problem is that for any scaled version of the scene $\lambda P = (\lambda X, \lambda Y, \lambda Z)$ with $\lambda > 0$, the projection onto the image remains identical. The projection equation looks like this:

$$\mathbf{p} = \pi(\lambda P, K) = \begin{bmatrix} f_x \frac{\lambda X}{\lambda Z} + c_x \\ f_y \frac{\lambda Y}{\lambda Z} + c_y \end{bmatrix} = \begin{bmatrix} f_x \frac{X}{Z} + c_x \\ f_y \frac{Y}{Z} + c_y \end{bmatrix} = \pi(P, K) \tag{4.1}$$

where K is the camera intrinsic matrix, (u, v) are pixel coordinates, and Z is the depth.

Concrete example: The toy car illusion To make this concrete, imagine photographing a toy car positioned 30 cm from the camera. Now consider a real car positioned 3 meters away (exactly 10 times farther). If the real car is also exactly 10 times larger in all dimensions (length, width, height), both objects will produce identical projections on the image plane. A 10 cm long toy car at a 30 cm distance projects to $\frac{10}{30} = \frac{1}{3}$ camera units, while a 100 cm long real car at a 300 cm distance projects to the exact same ratio of $\frac{100}{300} = \frac{1}{3}$ camera units.

Without additional information, these two scenarios are indistinguishable from the image alone. The camera observes only angular size, not absolute size or distance.

The scaling relationship More generally, consider two scenes that differ by a scale factor λ. In the first scene, an object of size s is located at distance d. In the second scene, an object of size λs is located at distance λZ. Both produce identical image projections because the ratio $\frac{s}{Z} = \frac{\lambda s}{\lambda Z}$ remains constant. This means any depth prediction $\hat{Z}$ can be scaled by an arbitrary factor $\lambda \hat{Z}$ and still be geometrically consistent with the observed image.

Implications for depth estimation This ambiguity has several critical implications for depth estimation. First, it implies a lack of absolute distance information, as we cannot determine whether a pedestrian is 3 meters or 30 meters away without additional cues. Second, the predicted depth values inherently use arbitrary units, making them meaningful only relative to each other rather than in metric measurements. Finally, resolving this ambiguity essentially requires external information, such as known object sizes, a known physical embodiment, or supplementary sensors like an IMU, LiDAR, or stereo vision setup.

Note that even when multiple views in time are available, as in classical structure-from-motion (SfM) pipelines, the scale ambiguity persists. While SfM can recover the 3D structure of a scene and camera motion through multi-view geometry, the reconstruction is only determined up to an unknown scale factor. Formaly, given a set of camera poses $\{T_i\}$ and 3D points $\{\mathbf{P}_j\}$ that satisfy the epipolar geometry constraints, the scaled versions $\{T'_i = [R_i|\lambda\mathbf{t}_i]\}$ and $\{\mathbf{P}'_j = \lambda\mathbf{P}_j\}$ satisfy the same constraints for any $\lambda > 0$.

4.1.2 Affine-Invariant Losses

Supervised depth estimation approaches circumvent the scale problem by training with ground truth depth annotations from sensors like LiDAR or stereo vision. However, since this task is inherently ambiguous, even supervised methods have recognized that learning scale-invariant representations improves generalization. This has led to the development of affine-invariant loss functions.

Scale-Invariant Loss

Eigen et al. [25] introduced a scale-invariant loss that evaluates depth predictions based on their relative structure rather than absolute values:

$$\mathcal{L}_{\text{si}} = \frac{1}{n}\sum_i d_i^2 - \frac{\lambda}{n^2}\left(\sum_i d_i\right)^2 \tag{4.2}$$

where $d_i = \log \hat{Z}_i - \log Z_i$ is the log-depth difference, $\hat{Z}_i$ is the predicted depth, Z_i is the ground truth depth, and λ is a weighting parameter. This loss allows the model to learn depth relationships without being penalized for uniform scaling errors.

Affine-Invariant Loss

More recent work has extended this concept to be invariant to both scale and shift transformations. Ranftl et al. [99, 100] popularized the affine-invariant loss, which measures the similarity between depth maps after optimal alignment:

$$\mathcal{L}_{\text{ssi}}(\hat{Z}, Z) = \min_{s,t} \frac{1}{n}\sum_i \|s \cdot \hat{Z}_i + t - Z_i\|^2 \tag{4.3}$$

where s is a scale factor (multiplicative) and t is a shift (additive offset). By optimizing over these parameters, the loss focuses on the relative depth structure rather than absolute values. This formulation is particularly useful when using very heterogeneous data where the predicted depth may be arbitrarily scaled and shifted, effectively decoupling the learning of depth relationships from the recovery of absolute scale.

Recent work has further refined this approach by applying affine-invariance locally rather than globally. Wang et al. [127] introduced a multi-scale local geometry loss that enforces affine-invariance within local patches, enabling the model to learn fine-grained local depth relationships while remaining robust to global scale ambiguity. This local formulation better preserves geometric details.

While these invariant losses enable models to learn robust depth representations, they do not solve the fundamental problem: for metric depth estimation, the absolute scale must eventually be recovered through additional constraints or prior knowledge. The following sections explore various strategies for achieving this.

4.2 Scale Clues

While monocular vision alone cannot always determine absolute scale, various contextual cues and prior knowledge can resolve this ambiguity. This section explores different sources of scale information that have been exploited in the literature, with particular emphasis on understanding how neural networks implicitly use these cues.

4.2.1 Understanding Depth Perception in Neural Networks

Before diving into specific scale recovery methods, it is crucial to understand how neural networks actually perceive depth from monocular images. Van Dijk and De Croon [124] conducted a study investigating this question through systematic experimentation, revealing fundamental insights about the implicit priors learned by depth estimation networks.

Experimental Methodology

Van Dijk and De Croon's investigation centered on two competing hypotheses for depth perception in driving scenarios. The *contact point hypothesis* posits that networks use the vertical position where an object contacts the ground to infer its distance, assuming a flat ground plane and consistent camera height. The *size prior hypothesis* suggests that networks learn typical object sizes and use apparent image size to estimate distance. Through carefully designed experiments manipulating synthetic and real-world images, the authors systematically tested these hypotheses.

To test the contact point hypothesis, they vertically shifted objects in images while keeping their size constant. Under the assumption that networks rely primarily on ground contact points, shifting an object upward should cause increased predicted depth, since a higher contact point in the image plane normally corresponds to greater distance on a flat ground. Their experiments confirmed this behavior: vertical displacement of objects produced proportional increases in predicted depth as can be seen in Figure 4.1. Conversely, when testing the size prior hypothesis by manipulating object dimensions while maintaining vertical position, they observed a significantly weaker effect. Doubling an object's size changed predicted distance by merely 20%, whereas identical vertical displacement could increase distance predictions by 100% or more.

Findings and Analysis

The study established that the vertical position of ground contact points serves as the dominant cue for monocular depth networks in ground-based scenarios, accounting for

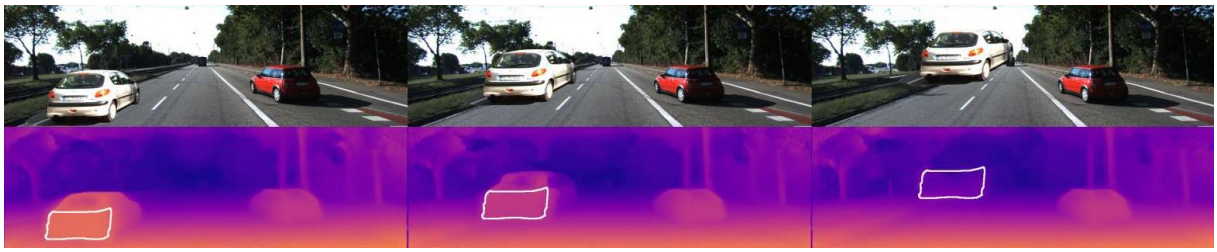


Figure 4.1: Van Dijk and De Croon's experiment [124] demonstrating the dominance of ground contact cues. Left: Original image with object at natural position. Middle and Right: Object shifted vertically upward causes predicted depth to increase significantly. This shows networks rely primarily on vertical position (ground contact) rather than object size for depth estimation.

the majority of their depth estimation capability. This heavy reliance on ground plane geometry has significant consequences for model behavior and generalization.

A critical finding concerns the networks' sensitivity to camera parameters. The KITTI dataset, on which these networks were trained, exhibits relatively consistent camera configuration: the camera is rigidly mounted on the vehicle with nearly constant height and orientation, with only minor variations due to vehicle suspension and terrain slopes. Networks trained on this data learn an implicit mapping between vertical image position and metric distance that is specific to KITTI's camera geometry. When camera height or pitch varies beyond the narrow range seen during training, this learned mapping becomes invalid, producing substantial errors in predicted distances.

Importantly, Van Dijk and De Croon demonstrated that networks only partially recognize changes in camera pose. When camera pitch was artificially varied by cropping images at different heights (emulating pitch changes of $\pm$2-3 degrees), all tested networks detected the change but underestimated its magnitude, with the estimated horizon shift being only 60% of the true shift. Similarly, when roll angles were introduced, networks detected but underestimated the change. This partial adaptation is insufficient: since the networks rely on vertical position for distance estimation, even small uncompensated changes in camera pitch directly translate to proportional errors in predicted distances to obstacles. An object at a given vertical position will be assigned the same distance regardless of whether the camera has pitched upward or downward, leading to systematic over- or underestimation of distances.

The observed behavior reveals a fundamental overfitting problem: models trained on a specific camera configuration internalize that setup's particular geometric relationship between image position and ground distance. This relationship is encoded implicitly in the network weights rather than being derived from explicit geometric reasoning. Consequently, these networks severely lack the ability to generalize to novel camera poses, making them unsuitable for deployment scenarios where camera parameters differ from those in the training data.

Confirmation Through Explicit Modeling

Recent work by Koledić et al. [63] provides strong confirmation of Van Dijk and De Croon's findings through a novel architectural approach. Rather than relying on implicit learning of depth cues, GVDepth explicitly models both hypotheses through separate computational pathways: a vertical image position branch using known ground plane

geometry, and a semantic branch based on learned priors like object size. The model predicts depth from both cues independently, then fuses them using learned uncertainty weights.

Their probabilistic fusion approach reveals that the learned uncertainty automatically adapts to favor the more reliable cue for different scene regions: vertical position dominates for close-range objects with visible ground contact, while semantic clues becomes more relevant when this contact point is occluded or distance is large.

Implications for Metric Depth Estimation

These findings have profound implications for designing robust metric depth estimation systems, particularly distinguishing between relative and metric depth requirements. The primacy of ground contact information suggests that providing explicit ground plane priors should significantly improve both accuracy and generalization. However, Van Dijk and De Croon's results reveal a critical limitation: without explicit modeling of camera parameters, networks learn dataset-specific geometric relationships that fail to generalize.

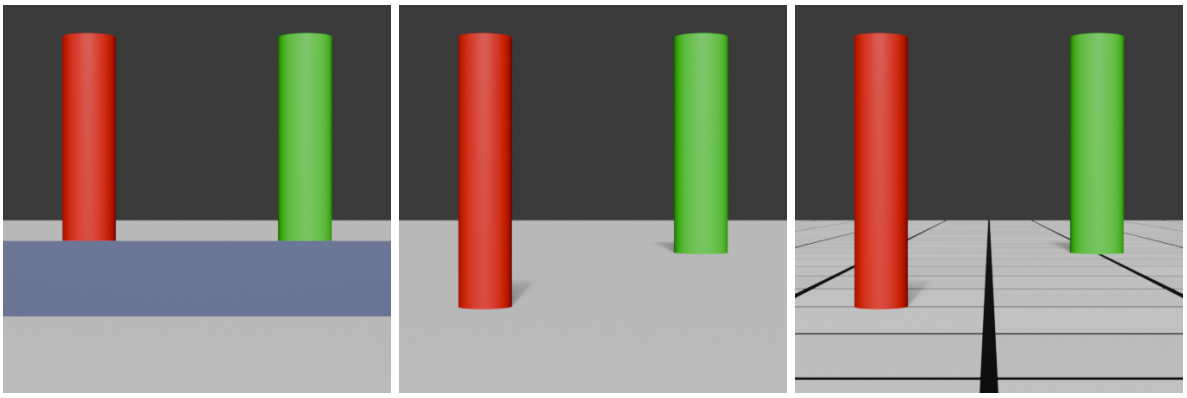
For relative depth estimation, where only the ordering and proportional relationships between depths matter, the learned vertical position cue can function adequately even when camera parameters change. The network may still correctly identify which objects are closer or farther, even if the absolute distances are incorrect. However, for metric depth estimation, where absolute distances in real-world units are required, the camera parameter dependency becomes a fundamental obstacle. The relationship between vertical image position and metric ground distance is determined by the camera's intrinsic parameters (focal length, principal point) and extrinsic parameters (height above ground, pitch, roll). When these parameters change, the mapping from image coordinates to metric distances changes proportionally. A network that has implicitly learned this mapping for one camera configuration will systematically mis-estimate metric distances when deployed with different camera parameters.

This explains why metric depth estimation fundamentally requires explicit knowledge of camera parameters. Unlike relative depth, which can be learned from image statistics alone, metric depth inherently depends on the projective geometry linking the image plane to the 3D world.

The observed overfitting to camera configurations underscores the necessity of: (1) training with diverse camera poses to learn robust geometric relationships, and (2) explicitly providing camera parameters to the network so it can correctly compute the image-to-world mapping. Ground-based systems such as vehicles and mobile robots possess a natural advantage in this regard: the ground plane provides a consistent geometric reference that, when combined with known camera parameters, enables principled metric scale recovery, forming the foundation for methods explored in subsequent sections.

4.2.2 Ground Plane and Camera Height

For ground-based systems such as vehicles and mobile robots, the ground plane is perhaps the most powerful source of scale information as can be seen in Figure 4.2. When the camera height h above the ground is known, the ground provides a direct metric reference that can be used to recover the scale of the entire scene.



(a) Impossible Estimation (b) Relative Depth (c) Absolute Metric Depth

Figure 4.2: Steps to get to metric depth. When the contact point with the ground is hidden, depth estimation is impossible in general. When the contact point is visible, relative depth can be estimated but the scale is still ambiguous. Finally, when the camera height is known, metric depth can be recovered by using the ground plane as reference.

Ground Plane Geometry

We place the ego frame on the ground plane with orientation: X = Left, Y = Down, Z = Forward. This makes the ground simply $Y = 0$, and the camera height h equals the world Y-coordinate of the camera center, i.e., $h = \mathbf{r}_2^\top \mathbf{t}$, where $\mathbf{r}_2$ is the second row of R.

For a camera with known intrinsic matrix K and extrinsic parameters $(R, \mathbf{t})$, a 3D world point $P = (X, Y, Z)$ projects onto pixel $\tilde{\mathbf{p}} = (u, v, 1)$ at depth $Z_{u,v}$ according to the pinhole model:

$$Z_{u,v}\,\tilde{\mathbf{p}} = K\big(R\,\mathbf{P} + \mathbf{t}\big) \tag{4.4}$$

Inverting gives

$$\mathbf{P} = R^\top\big(Z_{u,v}\,K^{-1}\tilde{\mathbf{p}} - \mathbf{t}\big) \tag{4.5}$$

For a ground point, $Y = 0$, so the second row of this equation must equal zero:

$$0 = \mathbf{r}_2^\top\big(Z_{u,v}\,K^{-1}\tilde{\mathbf{p}} - \mathbf{t}\big) \iff Z_{u,v}\,\mathbf{r}_2^\top K^{-1}\tilde{\mathbf{p}} = \underbrace{\mathbf{r}_2^\top \mathbf{t}}_{=h} \tag{4.6}$$

Solving for $Z_{u,v}$ yields:

$$Z_{u,v} = \frac{h}{\mathbf{r}_2^\top K^{-1}\tilde{\mathbf{p}}} \tag{4.7}$$

This equation provides a direct mapping from image coordinates to metric ground depth, effectively encoding the scale information in a form that can be used by the model. The geometry behind this construction is summarized in Figure 4.3.

In the special case where the camera is parallel to the ground (no rotations) we have:

$$Z_{u,v} = \frac{h\,f_y}{v - c_y} \tag{4.8}$$

This makes the inverse relationship between image row v and ground depth explicit: pixels closer to the horizon ($v \to c_y$) correspond to very large depths, while pixels near the bottom of the image (large $v - c_y$) correspond to nearby ground.

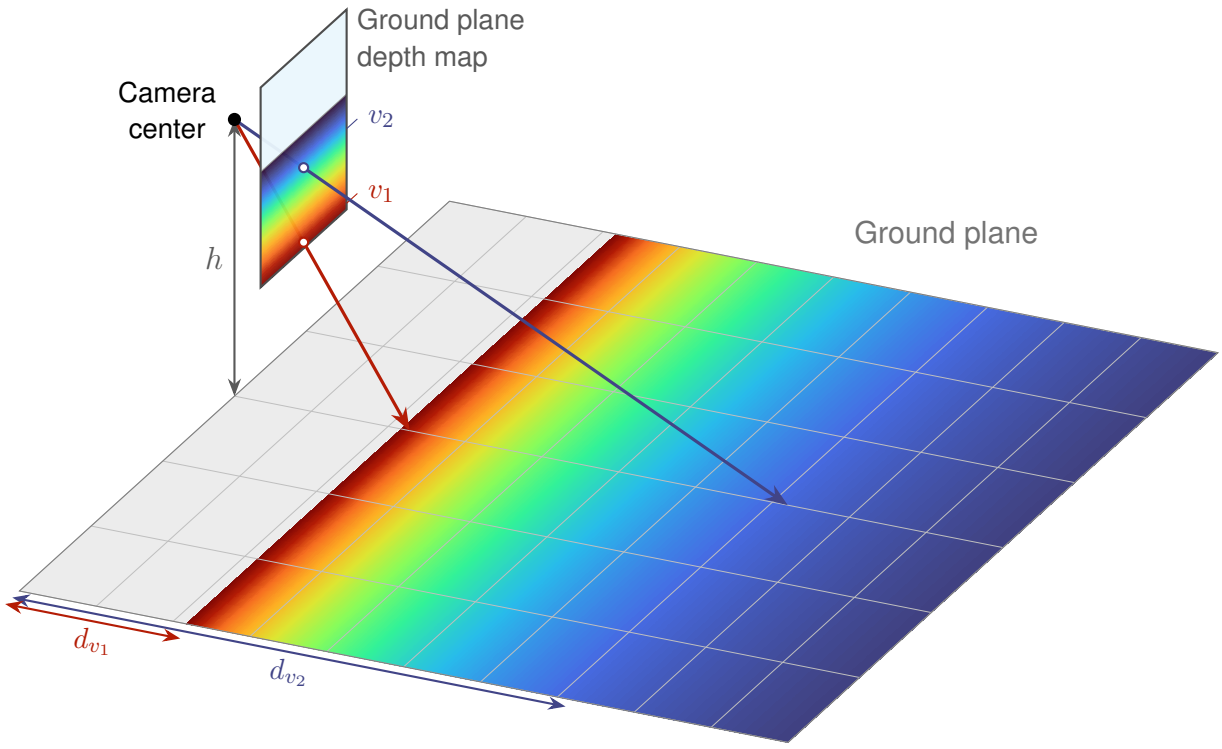


Figure 4.3: Ground plane geometry for scale recovery. Given known camera height h and orientation (extrinsics R, t), we can compute the theoretical depth to the ground plane at each pixel location. This provides a direct metric reference that resolves scale ambiguity for ground-based systems.

Ground-Based Scale Recovery Methods

Since the ground plane provides a direct metric reference, mutiple self-supervised methods have been proposed to leverage this cue for scale recovery. These methods typically involve some form of ground plane detection or segmentation, followed by fitting a plane to the detected ground pixels to estimate the camera height and recover metric scale:

Scale Recovery (Wagstaff and Kelly) [125]: This self-supervised approach integrates metric scale directly into the training process through novel loss functions. The method trains three networks: a depth network, an egomotion network, and a ground plane segmentation network. The ground plane segmentation network is trained with a geometry-based loss that identifies ground pixels as points under a certain height threshold.

The key innovation is a scale recovery loss that enforces metric scale during training. For each training image, they compute the estimated camera height $\hat{h}_t$ by fitting a plane to the segmented ground pixels using weighted least squares. Rather than directly comparing $\hat{h}_t$ to the known camera height h_{gt}, they compute an image-specific scale factor $s_t = h_{gt}/\hat{h}_t$ and generate per-pixel depth scaling targets $Z_t^{DS} = s_t \hat{Z}_t$. The depth scaling loss then minimizes $|\hat{Z}_t - Z_t^{DS}|/Z_t^{DS}$ for all pixels, forcing the network to produce metrically-scaled predictions by driving the scale factor toward unity. A similar translation scaling loss is applied to the egomotion predictions.

Critically, this approach only requires ground plane visibility during training. At test

time, the networks produce metrically-scaled outputs without any ground plane detection or post-processing. This contrasts with post-hoc scaling methods that must detect and fit the ground plane for every test image. The integration of scale constraints during training also improves depth consistency compared to methods that apply scale correction only at inference time.

The drawback of this method, is the interative way it is trained. First a relative depth model is trained, then the ground is segmented from its predictions and finally the depth model is retrained to get metric scale. This multi-stage training seems unecessary. Addinionally, the ground segmentation is done with a heuristic that that is not accurate for parts touching the ground. For example, the bootom of car wheels are segmented as ground.

VADepth (Xiang et al.) [135]: This method exploits the KITTI dataset's specific structure where a rectangular region in the bottom-middle of images reliably contains ground pixels. Combined with camera height prior, this heuristic allows scale determination. However, this approach is dataset-specific and may not generalize to other camera configurations or environments.

Camera Height-Based Methods (Kinoshita and Nishino) [59]: By using available ground and vehicle segmentation models, this approach leverages the information that the camera height is constant over time. The key insight is that projecting vehicle points onto the plane orthogonal to the ground should give consistent heights regardless of vehicle distance, enabling both camera height and scene scale recovery.

All these methods share a common limitation: they need an heuristic to segment the ground plane. Whereas, intuitively, the depth estimation model is already able to detect it since it predicts its geometry and relies on it to estimate the depth of other objects. Moreover, they all rely on dataset specific assumptions either explicitly (e.g., VADepth's fixed ground region) or implicitly by training on a single dataset with consistent camera parameters. This limits their generalization to new environments and camera configurations, which is a critical requirement for real-world deployment.

4.2.3 Vehicle Speed and Odometry as Scale Priors

Another relevant alternative is to use vehicle speed or odometry as the metric cue. If the displacement of the vehicle between two frames is known from external measurements, the scale ambiguity of monocular self-supervision can be reduced because the translational component of the camera motion is no longer only known up to scale. This idea is used in PackNet-SfM [40], where accurate platform motion is available and can be used as an additional supervisory signal.

In principle, if the ego-motion over time is measured reliably enough, one could even remove the need for a learned pose network and directly use the measured trajectory in the reprojection pipeline. In practice, however, learned pose models often remain more accurate at the frame-to-frame level. A more practical use of odometry is then to keep the pose network but calibrate its translation scale. For example, the average magnitude of the predicted translations can be constrained to match the average displacement measured by the vehicle sensors, providing a global scale prior while preserving the local flexibility of the learned pose estimation.

Despite its appeal, this strategy has several drawbacks. First, it requires external motion measurements from GPS, wheel-speed sensors, inertial systems, or a fused odometry stack. Such information is not always available, especially in datasets or deployment settings focused on monocular vision only. Second, the quality of the scale prior depends strongly on synchronization between the camera frames and the external motion estimate. Small timing offsets can already produce significant inconsistencies, particularly at low frame rates or during acceleration. Third, the external measurements themselves can be noisy or biased, for example because of wheel slip, GPS degradation, or imperfect state estimation.

For the present thesis, the more important limitation is conceptual: odometry-based scale priors mainly help during training and do not directly improve the robustness of the depth network at inference time. In the standard monocular deployment setting, only the depth network is used at inference, so the external motion signal that provided the scale cue during training is no longer available. The model may therefore recover metric scale on average for the training domain while still remaining sensitive to camera pose changes, as discussed earlier in this chapter. In theory, one could again use vehicle speed at inference to rescale the predicted depth, but this would require very accurate and well synchronized odometry as well as point correspondences between frames. More importantly, it breaks down in situations where the vehicle is stationary. This is a significant limitation for applications such as parking or maneuvering, where reliable metric depth is still required even though no translational motion is available to anchor the scale.

4.2.4 Known Object Sizes

Another source of scale information is prior knowledge about typical object sizes in the scene. If we know the real-world dimensions of certain object categories (e.g., average car dimensions), we can use their apparent size in the image to infer depth.

However, as Van Dijk and De Croon's work [124] demonstrated, size priors are less reliable than ground contact information for several reasons. First, there is high intra-class variance among objects of the same category, such as the significant difference in dimensions between various types of vehicles. Second, partial occlusions introduce ambiguity that makes estimating the true apparent size difficult. Finally, the inherent challenges in precise object instance segmentation mean that the boundaries used to calculate size are often unreliable.

Nevertheless, size priors can complement other scale cues, particularly for aerial objects or scenes without visible ground.

4.2.5 Focal Length and Camera Intrinsics

The camera focal length plays a crucial role in metric depth estimation. For a pinhole camera model, the relationship between depth Z, focal length f, and the size of an object's projection on the image sensor is given by similar triangles. However, focal length alone is insufficient to recover metric scale without additional information about object sizes or ground plane position.

That said, accurate knowledge of camera intrinsics is essential for all geometric-based scale recovery methods. Errors in estimated focal length or principal point can propagate to significant scale estimation errors. Modern self-supervised methods often estimate camera intrinsics jointly with depth [39], but this adds another degree of freedom that

can complicate scale recovery.

4.2.6 Integration of Ground Prior in Supervised Methods

Recent supervised methods have moved beyond post-hoc scale recovery to integrate ground plane information directly into the network architecture. This section examines two prominent approaches that demonstrate the benefits of this integration.

GeDepth (Yang et al.) [140]

GeDepth introduces a ground attention mechanism that allows the network to dynamically blend its learned depth predictions with ground plane prior information. The approach has two variants:

Vanilla Version: The ground depth image G (computed from camera parameters using Equation 4.7) is normalized and concatenated with the input RGB image. The network then predicts both a depth map $\hat{Z}$ and an attention map α. The final depth is computed as:

$$Z_i = (1 - \alpha_i) \cdot \hat{Z}_i + \alpha_i \cdot G_i \tag{4.9}$$

This formulation allows the network to choose between its predicted depth and the ground prior on a per-pixel basis. In practice, the attention map learns to identify flat ground regions where the prior is reliable. The main limitation of this approach is that it assumes a flat ground plane, which may not hold in all scenarios.

Adaptive Version: The adaptive variant extends the vanilla version by allowing the network to predict a per-pixel slope adjustment for the ground prior. This provides flexibility to handle non-flat terrain while still benefiting from the ground reference. However, this added flexibility has limitations. For instance, the slope estimation cannot adjust the horizon line, which limits its applicability in the presence of positive slopes. Furthermore, the attention mechanism sometimes incorrectly identifies the bottom portions of obstacles as ground, mistakenly applying the slope adjustment to these regions to achieve a seemingly improved depth estimate.

Results and Generalization: GeDepth demonstrates generalization to unseen datasets. The ground attention provides an interpretable mechanism showing which image regions the model considers as the ground.

GenDepth (Koledić et al.) [64]

GenDepth tackles the generalization problem across different camera configurations through a ground plane embedding strategy. Rather than concatenating ground prior directly with input as in GeDepth, this method encodes the theoretical ground depth through Fourier transform and concatenates it in the latent space.

The training strategy combines synthetic diversity with domain adaptation. The model learns from diverse CARLA simulator data with varying camera parameters alongside a single real-world dataset (KITTI) with fixed camera setup. Through adversarial feature alignment, the network learns camera-invariant features from synthetic data while adapting to real-world appearance. Crucially, ground plane embeddings are injected after

domain alignment, preventing the network from learning to ignore camera parameters when processing real images.

This approach enables better zero-shot generalization to multiple datasets (DDAD, Argoverse, Waymo, nuScenes).

Limitations of Supervised Ground Prior Integration

Despite their success, both GeDepth and GenDepth share a critical limitation: they require ground truth depth annotations. This dependency prevents their application in purely self-supervised settings, where the goal is to learn from monocular video alone.

The challenge is clear: how can we integrate ground plane priors in a self-supervised framework where we must simultaneously recover both the scene structure and its absolute scale? This question motivates the ground constraint formulation presented in the following section.

4.3 Ground Constraint for Self-Supervised Metric Depth

The previous section established the points that directly motivate our first contribution denominated GroCo[1] [13]. In this work, the objective was to leverage the ground prior in order to solve multiple limitations of self-supervised monocular depth estimation simultaneously: Provide a clue to solve the scale ambiguity that is inherent to monocular models, improve robustness to camera pose changes, provide a metric scale supervision to train the model and remove the need for any ground segmentation model or heuristic. Our ground constraint formulation addresses both limitations by integrating a geometric ground prior directly into the depth estimation model and by learning, at each pixel, when this prior should be trusted.

4.3.1 Motivation and Problem Formulation

Standard self-supervised monocular depth estimation methods rely on photometric consistency across adjacent views. They can learn scene structure effectively, but only up to an unknown scale factor. If $\hat{Z}$ denotes the predicted depth and $\hat{t}$ the predicted camera translation, then $(\hat{Z}, \hat{t})$ and $(\lambda\hat{Z}, \lambda\hat{t})$ produce the same photometric reprojection for any positive scalar λ. This scale ambiguity is the main obstacle preventing direct use of self-supervised monocular models in applications that require metric distances.

At first sight, the geometric ground depth derived from the camera parameters, introduced in the previous section through Equation 4.7, appears to solve the problem. If a model could be forced to agree with that prior on the road, the correct scale would follow. In practice, however, two difficulties arise.

The first is *scale coupling*. Because the reprojection objective is scale-invariant, the network has no natural incentive to adopt the metric scale of the ground prior. It can instead keep an arbitrary internal scale and ignore the prior to preserve photometric consistency.

The second is *ground identification*. The theoretical ground prior is only valid on pixels that actually correspond to visible, sufficiently flat ground. It should not be enforced on obstacles, curbs, steep slopes, or image regions above the horizon where no valid ground

[1] GroCo: Ground Constraint for Self-Supervised Metric Depth, ECCV24

depth exists. In supervised methods this selection can be learned indirectly from depth labels. In the self-supervised setting it must emerge from the training objective itself.

Our method resolves these two issues jointly. The model predicts a depth map and an attention map that acts as a soft ground selector. Training then encourages the depth to match the metric ground prior where the attention is active, while keeping the full prediction consistent with multi-view geometry. In this way, the method does not need a separate ground segmentation stage and does not rely on a global post-processing scale correction during inference.

4.3.2 Architecture Overview

The method builds on the ground-attention formulation introduced by GeDepth [140], but adapts it to the self-supervised regime. The ground depth image G, computed analytically from the camera intrinsics, extrinsics, and camera height, is normalized and concatenated with the RGB input. This fourth channel gives the network explicit access to the camera pose information that standard monocular models otherwise have to infer only indirectly from appearance.

We deliberately adopt the vanilla attention formulation rather than the adaptive slope variant of GeDepth. In the supervised case, learning a per-pixel slope correction can improve fit to non-flat terrain. In the self-supervised case, however, adding this extra degree of freedom introduces a major ambiguity. Since the photometric loss is scale-invariant, the network can satisfy the ground constraint at any arbitrary scale λ by simply adjusting the predicted slope to match the depth map's internal scale. In our experiments, this prevented the emergence of a meaningful metric grounding and causes the attention mechanism to fail, as it can no longer distinguish between actual ground and scaling artifacts. The method therefore keeps the geometric prior simple and uses the attention mechanism to select where it is reliable, rather than asking the network to modify the prior itself.

The architecture follows a standard encoder-decoder structure, shown in Figure 4.4. From the concatenated image and ground prior, the network predicts an intermediate depth map $\hat{Z}$ and a per-pixel attention map $\alpha \in [0, 1]$. The attention can be interpreted as a confidence score indicating whether the current pixel belongs to a region where the ground prior should be used, effectively resulting in a soft ground segmentation.

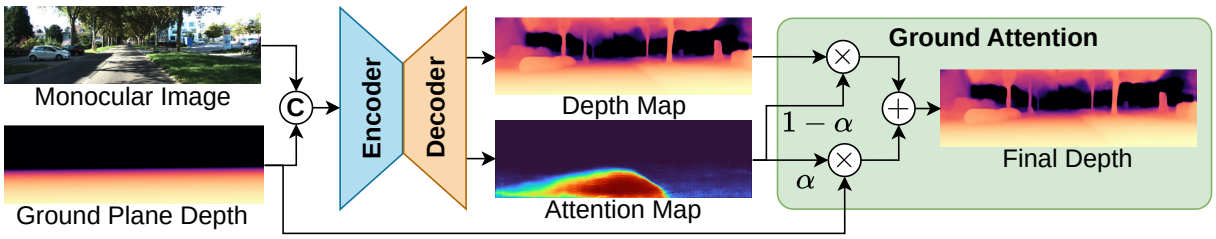


Figure 4.4: Illustration of the model architecture. The input image and ground depth are concatenated to provide ground-aware features. The ground attention mechanism combines the depth map with the ground depth, guided by the attention map, to produce a refined final depth estimation.

The final depth output combines the predicted depth and ground prior via the attention mechanism:

$$Z_i = (1 - \alpha_i) \cdot \hat{Z}_i + \alpha_i \cdot G_i \tag{4.10}$$

where i indexes pixels. When $\alpha_i = 1$, the output follows the analytic ground prior exactly. When $\alpha_i = 0$, the output reduces to the network prediction. Intermediate values interpolate smoothly between the two.

This design provides a clean separation of roles: $\hat{Z}$ accounts for the general scene geometry, while α decides where the ground prior should anchor the scale. On its own, however, the architecture is insufficient. A model trained only with self-supervised photometric losses would quickly learn that setting $\alpha \approx 0$ everywhere avoids conflicts with the scale-invariant reprojection objective. The key contribution of this method is therefore not architectural. It lies in the loss design that prevents this collapse and turns the attention into a meaningful metric grounding mechanism.

4.3.3 Ground Constraint Loss

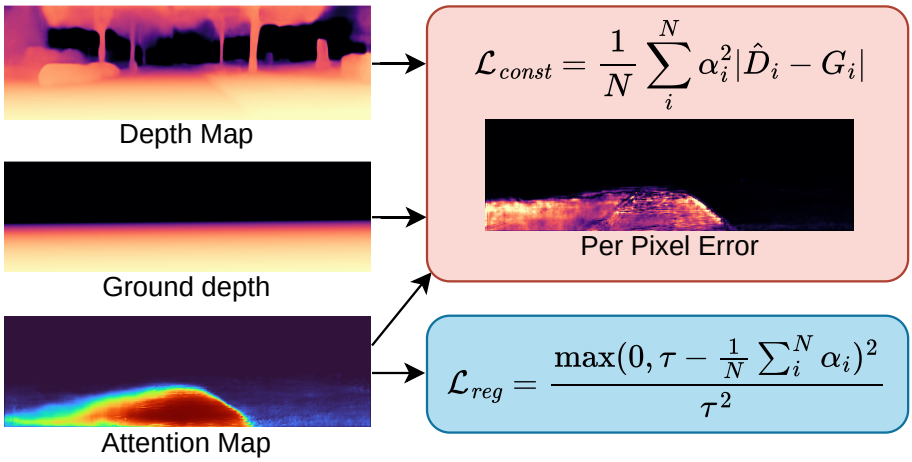


Figure 4.5: Overview of the proposed ground constraint loss $\mathcal{L}_{const}$ and attention regularization $\mathcal{L}_{reg}$. The error image in $\mathcal{L}_{const}$ illustrates how this loss penalizes disagreement between the depth map and ground depth, indirectly ensuring that the scale of depth converges to the one of the ground.

The central mechanism of the method is the ground constraint loss $\mathcal{L}_{const}$, which couples scale recovery and ground selection in a single expression; Figure 4.5 gives an overview of how this loss and the attention regularizer interact:

$$\mathcal{L}_{const} = \frac{1}{N} \sum_i^N \alpha_i^2 |\hat{Z}_i - G_i| \tag{4.11}$$

where N is the total number of pixels, α_i is the attention at pixel i, $\hat{Z}_i$ is the predicted residual depth, and G_i is the ground prior depth.

Its effect is twofold. On pixels where the attention is high, discrepancies between $\hat{Z}_i$ and G_i are penalized strongly, which forces the network to align its prediction with the metric scale of the ground prior. On pixels where the prior is inappropriate, for instance on vehicles or vertical structures, activating the attention would increase the loss because $|\hat{Z}_i - G_i|$ is large. The attention is therefore pushed away from those regions. In other

words, the same term that encourages scale alignment also teaches the model where the ground prior should not be applied.

Several choices in this formulation are important. First, the discrepancy is measured with an absolute distance rather than a relative one. This gives more weight to nearby ground, which is precisely where the flat-ground assumption is usually most reliable and where metric scale matters most for navigation. Second, the attention is squared. Using α_i^2 reduces the penalty on uncertain regions and yields softer transitions than a linear weighting, which in practice leads to cleaner blending between the learned depth and the ground prior. Third, the constraint is applied to $\hat{Z}$ rather than directly to the blended depth Z. This is what ties the learned representation itself to metric scale instead of merely correcting the final output after the fact.

Compared with previous self-supervised approaches that first estimate a ground mask and then compute a global scale, our method keeps the whole mechanism differentiable and end-to-end. The model is free to decide how much of the image can be metrically anchored through the ground prior, but it can only do so by making the underlying depth prediction compatible with that prior.

4.3.4 Attention Regularization

The constraint loss alone admits a degenerate solution: setting $\alpha_i = 0$ everywhere makes $\mathcal{L}_{const}$ vanish, but recovers no scale. To avoid this collapse, we introduces an asymmetric attention regularization term:

$$\mathcal{L}_{reg} = \frac{\max(0, \tau - \frac{1}{N}\sum_i^N \alpha_i)^2}{\tau^2} \tag{4.12}$$

where $\tau \in (0, 1)$ is a threshold parameter representing the expected proportion of ground pixels. This term only penalizes the model when the average attention falls below τ. Once the threshold is reached, the penalty becomes zero. The regularizer therefore behaves like a lower-bound constraint rather than a standard shrinkage prior. This is important for two reasons. It prevents the trivial all-zero attention solution, and it does so without forcing the model to use the ground prior everywhere. The attention only needs to activate on a sufficiently large fraction of pixels; beyond that point, the ground constraint loss determines where activation is actually beneficial.

The threshold τ is best understood as the proportion of the image for which one expects a reliable flat-ground prior during training. In most case we can have an idea of what this proportion should be, for exemple, in driving scenarios, we can expect two lanes of flat road to be visible in front of the camera. Such surfaces project to a triangle in the image like in Figure 4.6, so we proposes a simple geometric rule to estimate τ based on the navigable width:

$$\tau = \frac{P_w H}{4hW} \tag{4.13}$$

where P_w is the expected pathway width, H and W are the image dimensions, and h is the camera height. This rule keeps τ interpretable and dataset-dependent. In practice, a conservative value is preferable: it is better to anchor the scale on a smaller but reliable set of road pixels than to enforce the prior on ambiguous regions. Figure 4.7 shows that the method remains fairly stable over a broad range of τ values, with degradation appearing mainly when the threshold becomes too small to activate enough ground or too large to avoid ambiguous regions.

Figure 4.6: Estimation of τ based on road width and camera.

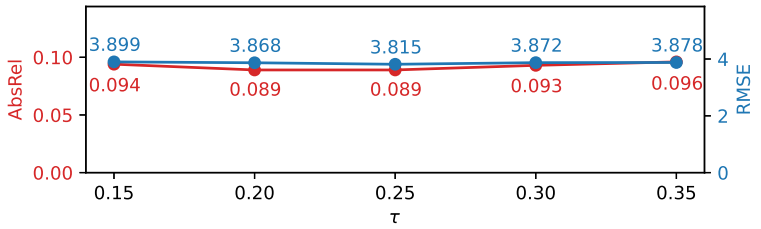


Figure 4.7: Sensitivity analysis: performance remains stable across a broad range of τ values on KITTI.

4.3.5 Complete Training Objective

The full training loss combines the ground-specific terms with standard self-supervised objectives:

$$\mathcal{L} = \mathcal{L}_{reproj} + \lambda_{smooth}\mathcal{L}_{smooth} + \lambda_{const}\mathcal{L}_{const} + \lambda_{reg}\mathcal{L}_{reg} \tag{4.14}$$

where $\mathcal{L}_{reproj}$ is the photometric reprojection loss from Godard et al. [38], $\mathcal{L}_{smooth}$ is the usual edge-aware smoothness term, and the weighting coefficients are set to $\lambda_{smooth} = 10^{-2}$, $\lambda_{const} = 0.1$, and $\lambda_{reg} = 0.1$.

The four terms have complementary roles. The reprojection loss learns the relative scene geometry. The smoothness term regularizes local fluctuations. The constraint loss anchors the learned depth to the metric ground prior where this is geometrically plausible. The attention regularization ensures that the anchoring mechanism is actually used during training. The resulting objective turns the ground plane into a metric reference while preserving the flexibility needed to handle non-ground regions and imperfect flat-ground assumptions.

Unlike the adaptive variant of GeDepth [140], the model does not predict a slope correction for the ground prior. In the self-supervised setting, this extra degree of freedom could absorb part of the scale error instead of resolving it. Moreover, it is not strictly necessary: if the local ground geometry deviates from the planar prior, the model can simply lower the attention on that area and rely on the learned depth instead.

4.3.6 Practical Implementation Details

The implementation is built on top of the Monodepth2 training pipeline [38]. We use a ResNet-50 encoder [46] pretrained on ImageNet [21] together with the standard Monodepth2 decoder [37]. The only architectural change at the input level is the addition of the normalized ground depth as a fourth channel. To preserve the pretrained behavior of the encoder, the weights corresponding to that extra channel are initialized to zero, while the RGB weights are kept unchanged.

At the output level, we replace the usual sigmoid depth parameterization with a softplus activation so that the network predicts a strictly positive depth directly, which is more consistent with the geometric prior. An additional attention head is attached to the decoder using multi-scale features, following the same general design as GeDepth. The attention branch remains lightweight compared with the backbone and does not significantly change the model size.

Training uses Adam [58] with learning rate 10^{-4}, batch size 12, and 20 epochs on KITTI [35]. On a single RTX 3090 GPU this corresponds to roughly 10 hours of training. For KITTI we set $\tau = 0.25$, while for DDAD images with higher vertical field of view we

use $\tau = 0.5$. These values correspond to a conservative assumption of about two traffic lanes of navigable flat area.

4.3.7 Rotation Augmentation for Robustness

A key insight from Van Dijk and De Croon's work is that monocular depth networks overfit to camera pose. This is addressed through geometric data augmentation that simulates camera rotations during training, with pitch and roll sampled uniformly from $[-5°, 5°]$ and yaw from $[-15°, 15°]$. These ranges are chosen to avoid introducing black borders or requiring image upscaling.

For each augmented sample, the RGB image is warped according to the sampled rotation, and the ground prior G is recomputed by rotating the camera extrinsics before evaluating Equation 4.7. This avoids the interpolation artifacts that would appear if one tried to rotate the already computed ground depth map. The same geometric logic is also applied at evaluation time when testing robustness under synthetic rotations: the LiDAR ground truth is transformed consistently by rotating the point cloud and projecting it back into the image plane.

This augmentation is central to the method's generalization objective. By exposing the network to changing camera positions while keeping the analytic ground prior correct, the training procedure discourages memorization and forces the model to use the provided camera-dependent cue. Figure 4.8 illustrates the effect of this augmentation on the input image, the model predictions, an the error.

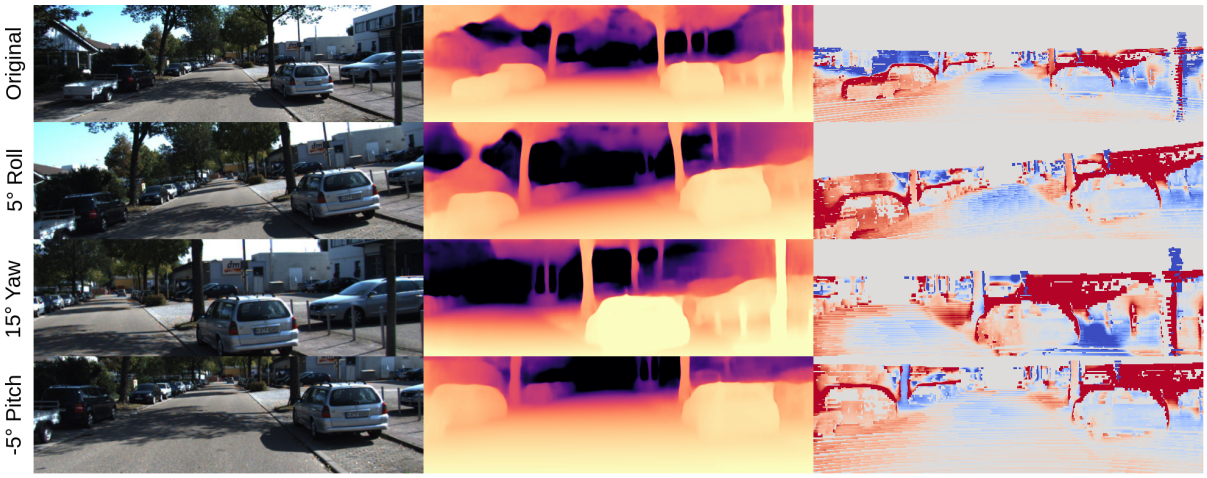


Figure 4.8: Visualisation of predictions on the same image with the various rotation augmentations. The last column is the relative per pixel error with the ground truth. The error is between -20% in red and 20% in blue, with 0% or absence of ground truth in white.

4.3.8 Interpretability through Ground Attention

An important secondary effect of the attention mechanism is interpretability. The map α provides a direct visualization of where the model considers the ground prior reliable. In a standard self-supervised depth model, it is difficult to tell whether an apparently plausible prediction is grounded in stable geometric cues or in brittle dataset-specific shortcuts. With the attention mechanism, there is a direct diagnostic signal.

In practice, the attention maps are more conservative than those of supervised methods such as GeDepth. This is expected since the self-supervised objective gives the model a strong incentive to avoid activating the prior on ambiguous areas, because an incorrect activation would directly penalize both the attention and the depth estimate through $\mathcal{L}_{const}$. The resulting behavior favors precision over recall, which is desirable here: a smaller but accurate set of grounded pixels is sufficient to recover scale and propagate it through the reprojection objective, as visible in Figure 4.9, where the attention activates on a smaller but cleaner support region than the GeDepth variants.

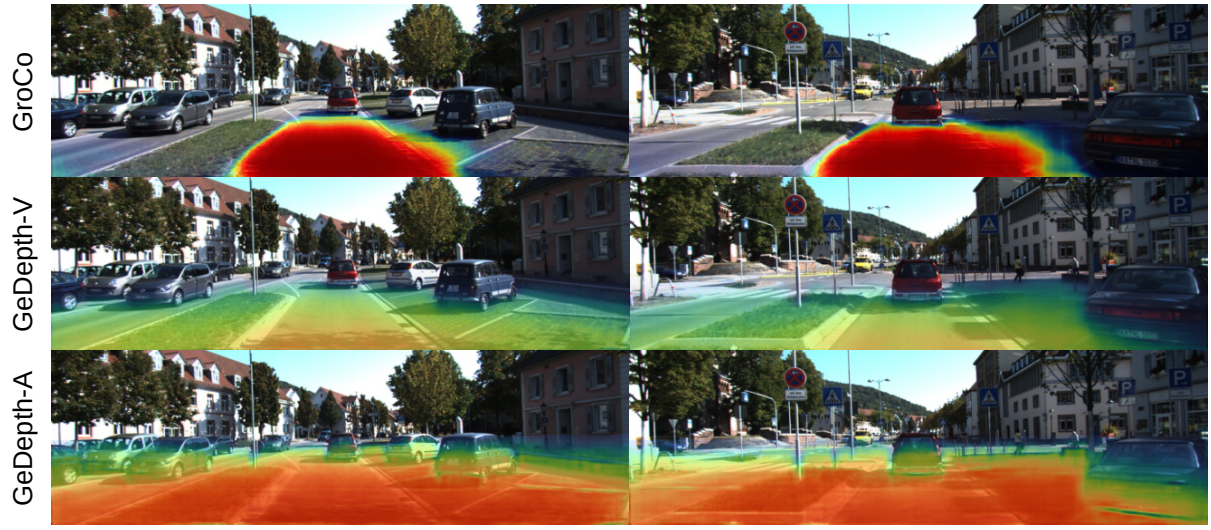


Figure 4.9: Comparison of attention maps: GroCo (top) vs GeDepth vanilla and adaptive versions. Our method produces more precise ground segmentation with higher certainty, while GeDepth shows higher recall but tends to consider bottom parts of obstacles as ground.

This precision also makes failure cases easier to detect. If the camera sees no visible road, or if the terrain departs too strongly from the assumed planar geometry, the attention can decrease accordingly instead of forcing the wrong scale into the prediction. Such behavior is useful both for human inspection and for downstream systems that may want to modulate their confidence based on whether the model successfully grounded its prediction. The example in Figure 4.10 is representative: even near obstacles, the attention remains concentrated on the traversable ground.

Figure 4.10: Example of predicted attention maps. Even near close obstacles, the activated regions remain concentrated on flat ground rather than spilling over object boundaries.

4.3.9 Experimental Results

The experiments are designed to evaluate three aspects separately: metric accuracy on the standard benchmark, robustness to camera pose changes, and generalization to new cameras and new datasets. Unless noted otherwise, evaluation follows the common KITTI protocol based on the Eigen split, and zero-shot experiments on DDAD are performed with image height 192 and depth capped at 80 m to match the KITTI setting.

KITTI Benchmark Performance

Table 4.1 compares our method with self-supervised methods that recover scale using camera height alone. We report results on both the original KITTI LiDAR labels and the denser improved labels from the depth benchmark. GroCo improves over previous camera-height methods on all error metrics for the original labels and remains competitive on the improved labels while still using only monocular self-supervision and camera parameters.

Table 4.1: KITTI self-supervised metric depth performance on the original and improved labels. Results from prior methods are taken from [59].

Labels	Method	Error (↓)				Accuracy (↑)		
		AbsRel	SqRel	RMSE	$\text{RMSE}_{\log}$	$\delta < 1.25$	$\delta < 1.25^2$	$\delta < 1.25^3$
Original	Scale Recovery [125]	0.123	0.996	5.253	0.213	0.840	0.947	0.978
	VADepth [135]	0.120	0.975	4.971	0.203	0.867	0.956	0.979
	GroCo	**0.113**	**0.851**	**4.756**	**0.197**	**0.870**	**0.958**	**0.980**
Improved	VADepth [135]	0.091	0.555	3.871	**0.134**	**0.913**	0.983	**0.995**
	GroCo	**0.089**	**0.517**	**3.815**	**0.134**	0.910	**0.984**	**0.995**

These results show that the method is not merely matching prior work through architectural capacity. The model remains relatively compact, yet the metric anchoring introduced by the ground constraint improves both scale-sensitive errors and robustness-oriented evaluations discussed next.

Robustness to Camera Pose Changes

To isolate the contribution of the learned attention mechanism, we compare GroCo with a baseline built from the standard Monodepth2 pipeline augmented with the scale-related losses of Wagstaff and Kelly [125]. Both models are trained with the same rotation augmentation. Table 4.2 shows that the two approaches are close on canonical KITTI images, but they diverge strongly once the camera pose changes.

This is the behavior one would expect if our model uses the ground prior in the intended way. On unrotated images, the baseline can exploit the strong regularity of KITTI and remain competitive. Under pitch, roll, and especially large yaw changes, that implicit dataset-specific mapping breaks down. GroCo degrades much less because the explicit ground conditioning and the learned attention provide a camera-dependent reference that remains valid after rotation.

The same trend appears when training on one DDAD camera and evaluating on the other cameras from the same platform. Table 4.3 shows that GroCo transfers better than the supervised camera-aware baselines reported by Yang et al. [140], despite being trained without depth annotations.

Table 4.2: Performance on KITTI under camera rotations at test time. Both models are trained with rotation augmentation.

Augment	Method	Error (↓)				Accuracy (↑)		
		AbsRel	SqRel	RMSE	$\text{RMSE}_{\log}$	$\delta < 1.25$	$\delta < 1.25^2$	$\delta < 1.25^3$
None	Baseline	0.092	**0.499**	**3.701**	**0.134**	**0.912**	**0.984**	**0.996**
	GroCo	**0.089**	0.517	3.815	**0.134**	0.910	**0.984**	0.995
5° Roll	Baseline	0.140	0.879	5.083	0.194	0.793	0.960	0.992
	GroCo	**0.101**	**0.635**	**4.301**	**0.147**	**0.891**	**0.979**	**0.994**
−5° Pitch	Baseline	0.145	0.891	5.341	0.197	0.782	0.952	0.989
	GroCo	**0.086**	**0.544**	**3.930**	**0.132**	**0.914**	**0.983**	**0.995**
5° Yaw	Baseline	0.107	**0.548**	**3.944**	0.149	0.891	0.980	**0.996**
	GroCo	**0.096**	0.552	**3.944**	**0.140**	**0.900**	**0.982**	0.995
15° Yaw	Baseline	0.209	1.246	5.919	0.268	0.483	0.932	0.986
	GroCo	**0.136**	**0.852**	**4.888**	**0.189**	**0.808**	**0.958**	**0.989**

Table 4.3: Generalization to unseen cameras within the same domain. The model is trained on the front camera and evaluated on the other DDAD cameras. Supervised comparison results are taken from [140].

Method	Mean	Back	Left	Right
DepthFormer [73]	0.93	0.83	0.98	0.97
DepthFormer + GeDepth Adaptive [140]	0.66	0.64	0.59	0.75
BTS [68]	0.72	0.82	0.98	0.97
BTS + GeDepth Adaptive [68, 140]	<u>0.62</u>	<u>0.62</u>	**0.56**	**0.67**
GroCo	**0.56**	**0.43**	<u>0.57</u>	<u>0.68</u>

Cross-Dataset Generalization

The most demanding evaluation trains the model on KITTI and applies it directly to DDAD without fine-tuning. This experiment changes not only the camera parameters, but also the sensor layout, viewpoint distribution, and scene statistics. Table 4.4 reports the results for each DDAD camera. GroCo improves over the self-supervised baseline on every camera and by a large margin on the front view, where the road prior is most informative.

The front-camera gain is particularly important because it shows that the method is not merely adjusting a fixed global scale learned on KITTI. The model transfers its metric behavior to a different platform. The side and rear cameras remain more difficult because the visible road is smaller, sometimes strongly oblique, and the ego-vehicle occupies part of the image. Even in those conditions, the attention often remains usable, as illustrated by the side-camera example in Figure 4.11.

Qualitatively, these results support the intended interpretation of the method. When the road is visible, the scale can be grounded on a subset of reliable pixels and use the reprojection loss to propagate that scale to the rest of the scene. When the road support becomes weak, performance decreases, but the attention mechanism makes that loss of support observable rather than hidden.

Table 4.4: Zero-shot transfer from KITTI to DDAD. The model is trained on KITTI and evaluated on each DDAD camera without fine-tuning.

Camera	Method	Error (↓)				Accuracy (↑)		
		AbsRel	SqRel	RMSE	$\text{RMSE}_{\log}$	$\delta < 1.25$	$\delta < 1.25^2$	$\delta < 1.25^3$
Front	Baseline	0.403	5.366	14.364	0.567	0.058	0.283	0.830
	GroCo	**0.154**	**1.853**	**8.588**	**0.239**	**0.760**	**0.929**	**0.975**
Back	Baseline	0.272	3.616	10.172	0.335	0.586	0.856	0.937
	GroCo	**0.233**	**3.031**	**9.753**	**0.318**	**0.655**	**0.874**	**0.946**
Left	Baseline	0.353	4.031	9.424	0.375	**0.798**	**0.916**	**0.975**
	GroCo	**0.256**	**2.875**	**8.656**	**0.321**	0.647	0.861	0.937
Right	Baseline	0.371	4.464	9.737	0.414	0.466	0.752	0.883
	GroCo	**0.334**	**3.832**	**9.190**	**0.389**	**0.512**	**0.790**	**0.899**

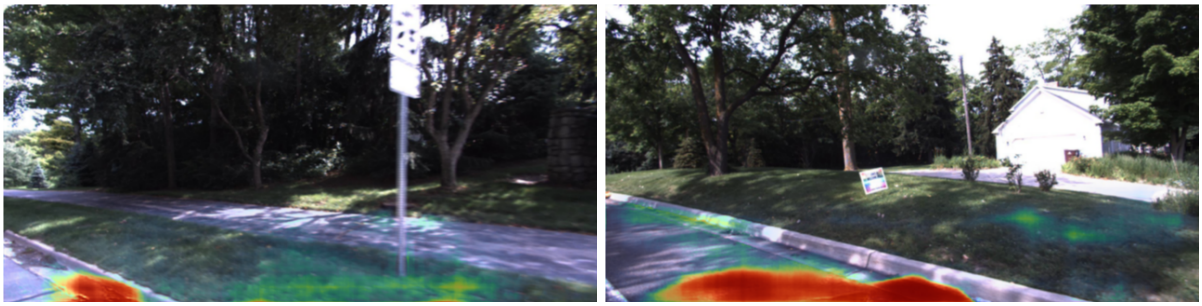


Figure 4.11: Attention maps on a DDAD side camera, a viewpoint never observed during training. Even when the road occupies only a small and oblique region of the image, the model still identifies a focused support area for metric grounding.

4.4 Conclusion

This contribution shows that metric supervision is not strictly necessary to integrate a ground prior effectively. What is needed is a formulation that couples three requirements that are usually treated separately: selecting valid ground regions, recovering a metric scale, and preserving self-supervised geometric consistency. The ground constraint and attention regularization provide that coupling in a compact form.

From a broader thesis perspective, this method is also a useful case study in structure-aware learning. The method does not attempt to replace self-supervision with stronger external supervision. Instead, it introduces a geometric structure that is already available in the problem setting, namely the relationship between the camera and the ground plane, and designs the optimization so that this structure becomes operational. The experimental results indicate that this is enough to improve both scale accuracy and generalization to new camera configurations.

Chapter 5

Boundary Sharpness in Depth Estimation

The previous chapters established the main ingredients of modern monocular depth estimation: Chapter 2 introduced the encoder–decoder architectures and output parameterizations used to predict dense depth maps, Chapter 3 showed how these models can be trained through reprojection-based self-supervision, and in Chapter 4 our contribution addressed the problem of metric scale. Yet even when scale is correct and global image metrics are strong, an important failure mode often remains: depth discontinuities at object boundaries are frequently too blurry.

This issue is not only visual. In driving scenes, foreground objects occlude the background and create abrupt depth jumps that should remain sharp in the prediction. When a model is forced to assign a single depth value to a pixel at the boundary between two objects, it often regresses to an intermediate estimate between foreground and background. The resulting depth map may still look good visually, but once reprojected into 3D it produces floating artifacts and spurious points between surfaces as visible in Figure 5.1. This observation also connects to the discussion in section 2.3.2: standard scalar outputs are well suited to smooth regions, yet they are poorly matched to the intrinsically ambiguous, multimodal nature of boundary pixels.

This chapter studies that boundary-sharpness problem in the self-supervised setting. Rather than treating each pixel as having a single deterministic depth, we represent boundary pixels with a two-component mixture distribution that can model multiple plausible depths. This allows the network to preserve discontinuities without requiring dense boundary annotations, while remaining compatible with the self-supervised learning pipeline introduced earlier. The main idea is to propagate these distributions through reprojection, interpolation, and loss computation so that uncertainty is handled explicitly during training instead of being collapsed into an averaged depth estimate.

The chapter is organized as follows. Section 5.1 discusses existing methods and expresses the problem more formally. It reviews mixture models and uncertainty estimation in relation to self-supervised depth estimation. Section 5.2 introduces our depth mixture formulation, together with its propagation and optimization framework and the associated edge sharpness metric. Section 5.3 then evaluates the method on KITTI and VKITTIv2, with both image-space and 3D pointcloud analysis.

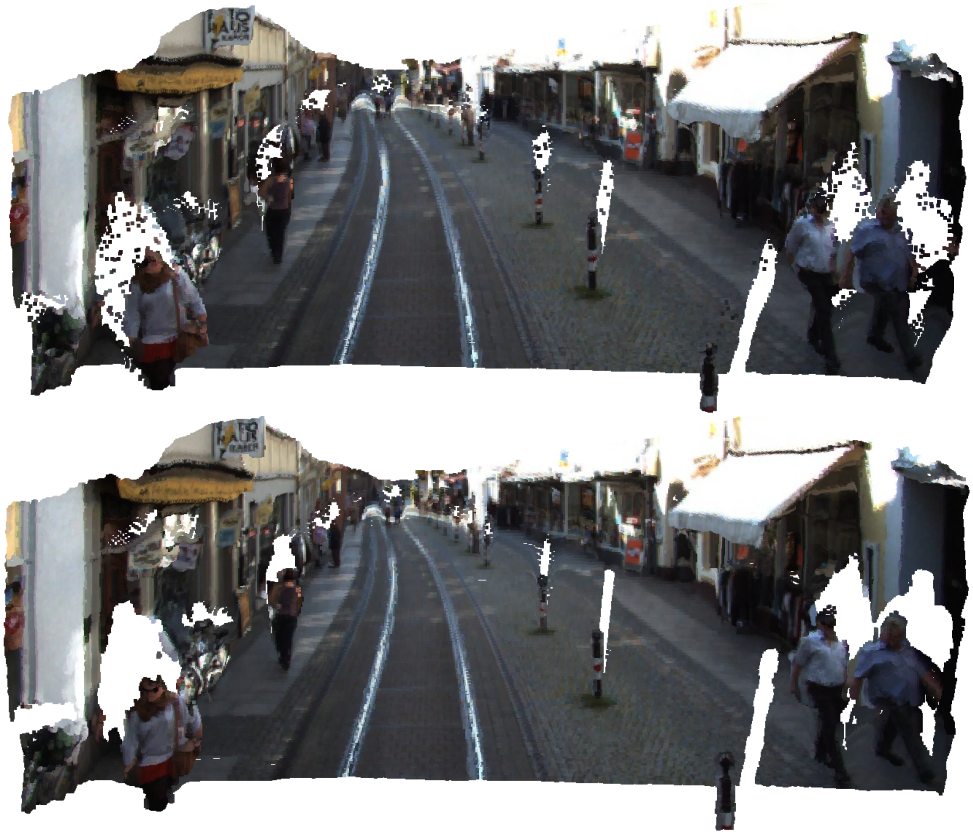


Figure 5.1: Other methods (top) typically produce floating artifacts at object boundaries, resulting in spurious points between foreground and background. In contrast, our method (bottom) generates clean point clouds with sharp object boundaries, effectively eliminating these artifacts.

5.1 Related Work

5.1.1 Sharp Borders in Self-Supervised Depth Estimation

As discussed in Chapter 3, self-supervised monocular depth estimation learns from view synthesis rather than direct depth annotations. Within this setting, most work improves boundary quality only indirectly, for example through architectural changes that better preserve spatial detail [16, 78, 146] or through auxiliary cues such as semantic segmentation [94,147]. These approaches generally help localize where a depth discontinuity should appear, but they do not explicitly address the fact that pixels near occlusion boundaries may correspond to more than one plausible depth.

In contrast, edge sharpness has been studied more directly in supervised settings [10, 119,136], where high-resolution images and fine-grained annotations make it easier to learn crisp discontinuities. This difference suggests that the limitation is not only architectural, but also representational: if the model predicts a single depth value per pixel, ambiguity at boundaries is naturally collapsed into an average. We also found that many conventional sharpness measures are strongly entangled with overall depth accuracy. This motivates the edge entropy measure introduced later in the chapter, which is designed to assess transition sharpness more directly.

One attractive approach is to predict a discrete depth distribution [7, 128, 132]. Although this approach has usually been used to improve overall depth accuracy, it also

has the potential to represent multimodal distributions. In Chapter 1, we introduced the intuition that this type of model predicts the underlying depth distribution, as illustrated in Figure 2.14. However, we found that this is not the case in practice. When evaluating state-of-the-art self-supervised depth estimation models with discrete depth representations, such as [132], we observed that their predictions are not representative of the underlying distribution. In particular, these models need some mechanism to smooth between bins. One might expect this to be achieved with a Gaussian-like distribution, but in practice we observed a different strategy: the model predicts a single bin with high confidence and then uses extreme depth values with low confidence to smooth the transition between bins. Since the values are averaged together, this strategy is perfectly valid for producing accurate depth estimates, but it does not represent the underlying distribution. This is problematic for our use case because it prevents sampling the mode of the distribution, which is necessary to produce sharp boundaries. Trying to do so leads to very large staircase artifacts when the principal bin changes.

In theory, one could try to learn the underlying distribution by computing the loss for each bin of the prediction, but in self-supervised learning the cost of this approach would be prohibitive, since it would require computing the reprojection loss for each bin of each pixel. A less costly alternative is to assume a specific kind of distribution, such as a mixture of Gaussian or Laplace distributions. We could try to recover such a distribution from the discrete prediction, but this would require a fitting procedure that we consider unnecessary. We also note that introducing a smoothing factor into the discrete prediction would not guarantee fixing the issue, since the model could still learn to use extreme values to affect the final prediction. Instead, this chapter takes the more direct approach of learning a parametric distribution directly, assuming a two-component Gaussian mixture. This assumes that at most two surfaces can intersect at a given pixel. In practice, this is reasonable, since edges occur between two surfaces, while transitions between three or more surfaces (T-junctions) occur only at single pixels. We also note the layered formulation of Tulsiani *et al.* [121], which decomposes the scene into multiple depth layers learned through view synthesis. This approach is conceptually similar to the mixture model, but it is designed to represent the entire scene as a combination of layers rather than to capture local multimodality at boundaries. In practice, it requires known egomotion and more complex forward rendering methods, but the main issue is the lack of information about when each layer should be used, which makes borders imprecise. This ambiguity is naturally resolved in the mixture model by the mixture weights.

5.1.2 Mixture Models and Density Estimation

Mixture models are a natural tool for representing multi-modal predictions. Classical formulations are often fitted with the Expectation-Maximization algorithm [20], while mixture density networks [9] learn conditional mixture distributions directly from data. In computer vision, such models have been used for robust estimation and ambiguity handling in tasks such as optical flow [131] and 3D tracking [4]. Most closely related to our setting, Tosi *et al.* [119] used a mixture of Laplacian distributions for supervised stereo depth estimation.

Introducing mixture distributions also raises the question of how uncertainty should be parameterized and optimized. Recent work on mean-variance estimation [109,113,114] highlights that likelihood-based training can become unstable when the predicted variance is poorly controlled, leading either to overconfident fits or to excessive smoothing. For

depth estimation specifically, Kendall and Gal [56] formalized the distinction between aleatoric and epistemic uncertainty, and Poggi *et al.* [97] analyzed uncertainty estimation in self-supervised monocular depth prediction.

The contribution developed in this chapter builds on these two lines of work: mixture models provide a suitable representation for ambiguous boundary pixels, and uncertainty estimation provides the tools needed to train such a representation within a self-supervised reprojection pipeline.

5.1.3 Mixture Models for Depth Estimation

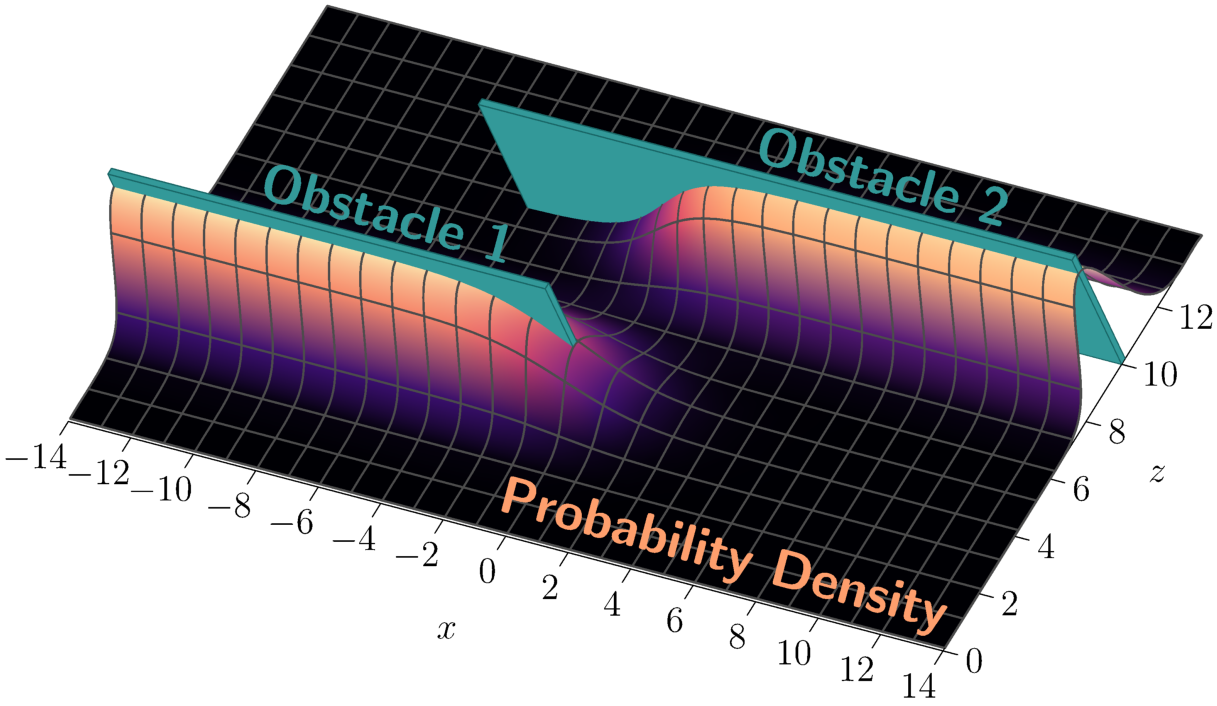


Figure 5.2: Visualization of a theoretical depth distribution on a row of pixels. It illustrates the expected behavior of the predicted depth distribution, with high probability density on visible surfaces. Obstacle 1 lies closer to the camera and occludes obstacle 2, which is farther away. When the two obstacles are well separated, the distribution is unimodal and can be represented by a single Gaussian. However, at the occlusion boundary $x = 0$, the distribution becomes bimodal and must be modeled with a Gaussian mixture to capture both surfaces.

In the present context, the key idea is simple: most pixels belong to a single surface, but pixels located on an occlusion boundary can mix evidence from two depth layers. A mixture model can represent this situation explicitly. Rather than forcing a single average depth at such locations, it allows the prediction to retain both foreground and background hypotheses. Figure 5.2 sketches the kind of one-dimensional depth profile that motivates this representation.

The depth distribution is one-dimensional, and for each pixel i we model it as a two-component mixture:

$$p(z_i) = \sum_{j=1}^{2} \alpha_j p_j(z_i) = \alpha_1 p_1(z_i) + \alpha_2 p_2(z_i) \tag{5.1}$$

where α_j is the mixing proportion for the j-th component, with $\alpha_1 + \alpha_2 = 1$, and $p_j(z_i)$ is the corresponding probability density function. If each component is parameterized by a location and a scale term, such as a Gaussian or Laplace distribution, the model must predict five parameters per pixel: two means, two variances or scales, and one independent mixture weight.

5.1.4 Mean Variance Estimation

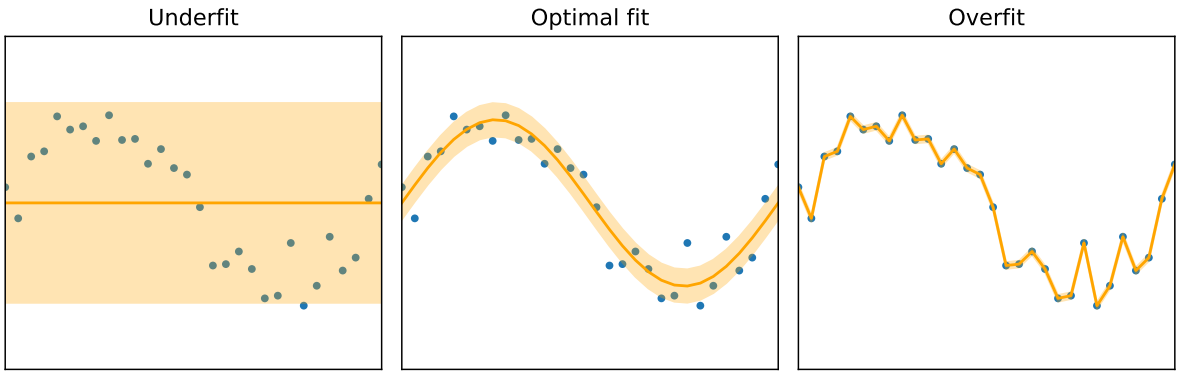


Figure 5.3: Illustration of the impact of variance on mean fitting. A variance that is too small leads to overfitting, whereas a variance that is too large results in underfitting.

Learning such a representation places the problem within the broader setting of mean-variance estimation, where the network predicts both a central value and an uncertainty term. This setting has been studied extensively [109, 113, 114], and several practical difficulties are already known.

In particular, when training is driven by likelihood terms, the predicted variance strongly influences the final solution. Figure 5.3 illustrates the main tradeoff: a variance that is too small encourages overfitting, whereas a variance that is too large leads to underfitting. Because the variance also affects the relative weighting of samples during optimization, poor uncertainty estimates can distort learning rather than simply describe confidence.

Uncertainty is usually separated into two categories. Aleatoric uncertainty comes from the data itself, for example image noise, limited resolution, or ambiguity at occlusion boundaries. Epistemic uncertainty comes from the model and reflects limits in what it has learned to represent.

In the example of Figure 5.3, the variance of a well-fitted model reflects mainly aleatoric uncertainty, because it corresponds to variability genuinely present in the data. By contrast, the broad variance produced by an underfitted model is largely epistemic, since it compensates for an inadequate representation rather than for inherent ambiguity in the observations.

5.2 Methodology

To address the boundary blurring problem, we propose the second contribution[2] of this thesis [14]. We represent per-pixel depth as a multimodal distribution, explicitly modeling both depths at boundaries, preserving sharp transitions and removing artifacts. Final model outputs are shown in Figure 5.4. We demonstrate how to integrate this mixture distribution representation into the standard self-supervised depth estimation pipeline by propagating the distributions through the various operations, including reprojection, color interpolation, and loss computation.

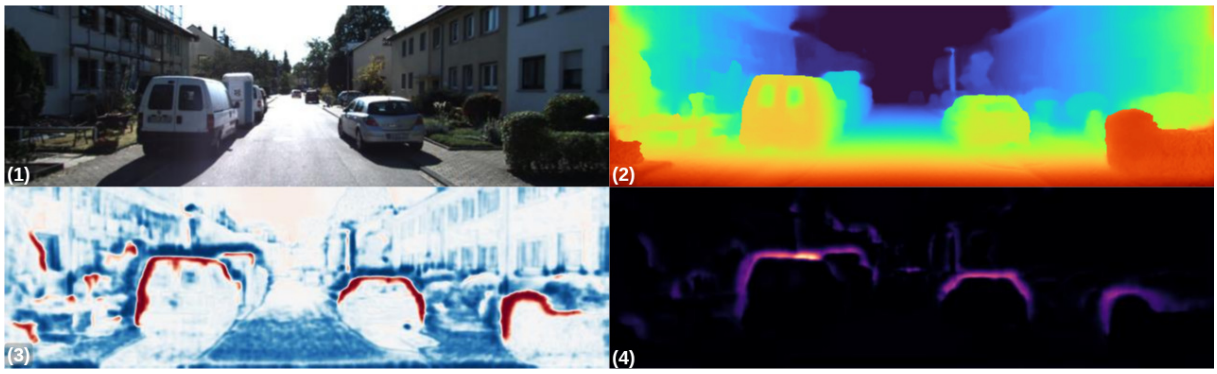


Figure 5.4: (1) Input Image (2) Predicted Depth (3) Mixture weight: Blue=0, White=0.5, Red=1 (4) Relative difference between components means. At object boundaries, our model correctly identifies both depth modes and creates sharp discontinuities.

5.2.1 Distribution Representation

For our representation, we work with disparity z^{-1} (inverse depth) since it makes distributions more Gaussian and reduces linearization errors in uncertainty propagation [17]. Substituting into the reprojection equation, the reprojection becomes linear in z^{-1}, which is beneficial for our uncertainty propagation framework.

For each pixel, we assume that the disparity distribution is a Gaussian mixture $\mathcal{M}(z^{-1})$ of two components, each parameterized by a mean μ_k and a variance σ_k^2, with mixing weights w_k that satisfy $w_1 + w_2 = 1$:

$$\mathcal{M}(z^{-1}) = w_1 \mathcal{N}(z^{-1}|\mu_1, \sigma_1^2) + w_2 \mathcal{N}(z^{-1}|\mu_2, \sigma_2^2) \tag{5.2}$$

The neural network predicts five parameters as separate output channels: $\mu_1, \mu_2, \sigma_1^2, \sigma_2^2$, and w_1, with $w_2 = 1 - w_1$ implicitly defined by the constraint. Following standard practice, we use a sigmoid activation for w_1 and μ_k and predict the log variance by using an exponential activation.

We use a Gaussian distribution since it is a common choice for modeling disparity distributions, allows uncertainty propagation and makes computing the best components analytically tractable.

[2]Towards Sharper Object Boundaries in Self-Supervised Depth Estimation, BMCV 2025 Oral

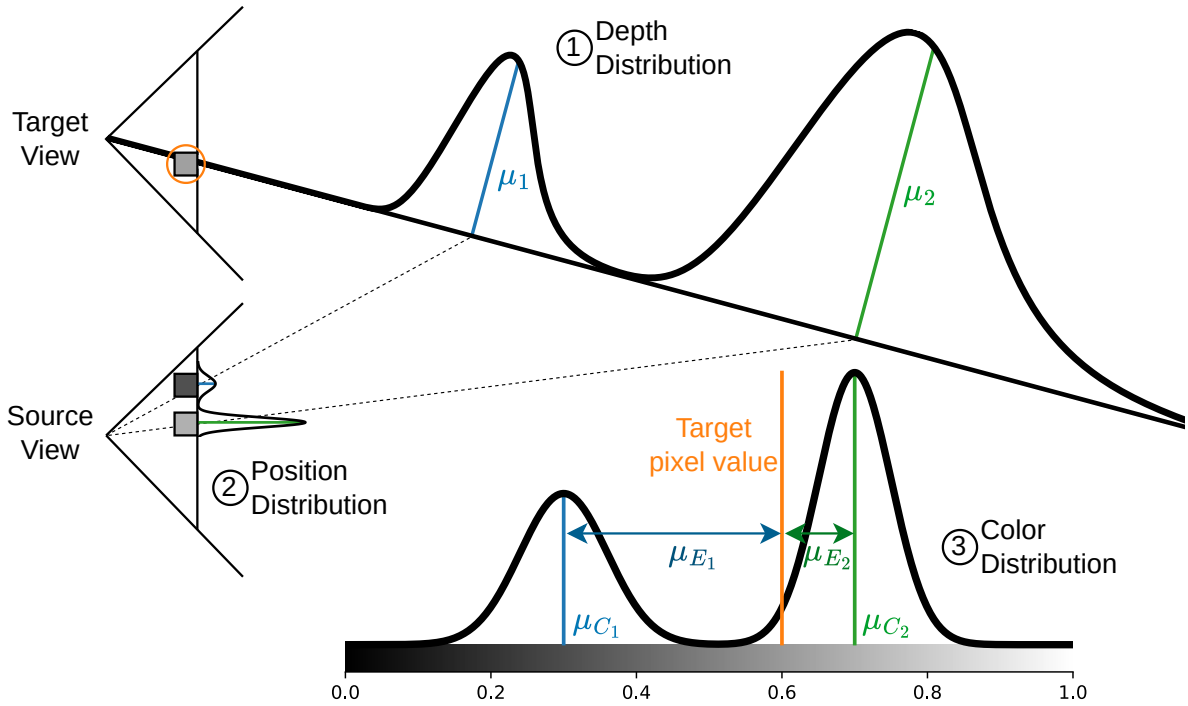


Figure 5.5: Reprojection pipeline with mixture distributions. ① Predicted depth distribution for each pixel. ② The depth distribution is projected into the source view, resulting in a distribution of positions. ③ These positions are sampled using bilinear interpolation to obtain a color distribution which is compared to the target color to compute the loss.

5.2.2 Distribution Propagation

To propagate distributions through our pipeline, we model each disparity component as a random variable $Z_k^{-1} \sim \mathcal{N}(\mu_k, \sigma_k^2)$. When applying a differentiable function f to Z_k^{-1}, we approximate the result as a normal distribution:

$$f(Z_k^{-1}) \sim \mathcal{N}(f(\mu_k),\ f'(\mu_k)^2\sigma_k^2) \tag{5.3}$$

This first-order approximation is accurate when f is approximately linear in the region of interest. In our case, this allows us to propagate uncertainty through the pipeline efficiently and with good accuracy. This method is commonly used in uncertainty propagation such as in the Extended Kalman Filter [54, 84].

The pipeline consists of three main steps between the distributions shown in Figure 5.5: reprojection, color interpolation, and loss computation. The reprojection can be written as a function $\mathbf{p}' = f_{\text{reproj}}(z^{-1}, \mathbf{p})$ that maps disparity z^{-1} at target pixel $\mathbf{p}$ to source coordinates $\mathbf{p}'$.

Once we have $\mathbf{p}'$, we use bilinear interpolation to sample the source image:

$$I_s(\mathbf{p}') = \begin{bmatrix} 1-\delta_u & \delta_u \end{bmatrix} \begin{bmatrix} I_{00} & I_{01} \\ I_{10} & I_{11} \end{bmatrix} \begin{bmatrix} 1-\delta_v \\ \delta_v \end{bmatrix} \tag{5.4}$$

where I_{ij} are the intensities of the four neighboring pixels and (δ_u, δ_v) are the fractional offsets within the pixel cell.

Rather than propagating the uncertainty through these two steps separately, we can define a composite function $s(Z_k^{-1}) = I_s(f_{\text{reproj}}(Z_k^{-1}))$ applied on all $\mathbf{p}$ that maps directly from disparity to sampled color. Using the chain rule we get:

$$s'(Z_k^{-1}) = \nabla I_s(f_{\text{reproj}}(Z_k^{-1})) \cdot f'_{\text{reproj}}(Z_k^{-1}) \tag{5.5}$$

where $\nabla I_s = [\frac{\partial I_s}{\partial u}, \frac{\partial I_s}{\partial v}]$ is the image gradient at the sampled position, and $f'_{\text{reproj}}(Z_k^{-1})$ is the derivative of the reprojection function with respect to disparity.

The warped color distribution is then:

$$\mu_{\hat{I}_k} \approx s(\mu_k) = I_s(f_{\text{reproj}}(\mu_k)) \tag{5.6}$$

$$\sigma^2_{\hat{I}_k} \approx s'(\mu_k)^2\sigma_k^2 = [\nabla I_s(f_{\text{reproj}}(\mu_k)) \cdot f'_{\text{reproj}}(\mu_k)]^2\sigma_k^2 \tag{5.7}$$

This direct approach handles the propagation of uncertainty through the whole reconstruction pipeline, accounting for how changes in disparity affect both position and ultimately the sampled color. It is also very efficient since these computations are done analytically.

5.2.3 Loss Distribution and Optimization

For the loss computation, we use the combination of SSIM and L1 distance between the actual target image I_t and the synthesized target image $\hat{I}_t$. SSIM is used since it is robust to small shifts and captures structural information.

In our framework, since we are working with distribution over colors rather than a single value, the error at pixel $\mathbf{p}$ becomes a random variable denoted E. Since both SSIM and L1 operate on the distributed reprojected color, they also become random variables whose weighted combination forms E:

$$E = \lambda_{\text{SSIM}} \cdot (1 - \text{SSIM}) + \lambda_{\text{L1}} \cdot \text{L1} \tag{5.8}$$

For each component k in our mixture model, we compute the mean $\mu_{k,E}$ and variance $\sigma^2_{k,E}$ of this error distribution using the same linear approximation technique described earlier.

We then use a competitive training approach where components specialize based on their relative performance. This approach is inspired by moment matching, where we optimize the predicted distribution parameters such that the mean error approaches zero while the variance aligns with the empirically observed error magnitude.

In order for our two components to specialize, we define an intermediate weight objective $\bar{w}_k$ as the probability that component k has a better (lower) error than the alternative component $1 - k$:

$$\bar{w}_k = P(E_k < E_{1-k}) = P(E_k - E_{1-k} < 0) \tag{5.9}$$

This last probability can be calculated analytically since both E_k and E_{1-k} are assumed Gaussian and that the difference between two Gaussian random variables is also Gaussian. Specifically, if $E_k \sim \mathcal{N}(\mu_{E_k}, \sigma^2_{E_k})$ and $E_{1-k} \sim \mathcal{N}(\mu_{E_{1-k}}, \sigma^2_{E_{1-k}})$, then:

$$E_k - E_{1-k} \sim \mathcal{N}(\mu_{E_k} - \mu_{E_{1-k}}, \sigma^2_{E_k} + \sigma^2_{E_{1-k}}) \tag{5.10}$$

Therefore, $\bar{w}_k$ can be computed using the cumulative distribution function Φ of the normal distribution:

$$\bar{w}_k = \Phi\left(\frac{\mu_{E_{1-k}} - \mu_{E_k}}{\sqrt{\sigma^2_{E_{1-k}} + \sigma^2_{E_k}}}\right) \tag{5.11}$$

We then define our loss function with three parts:

$$\mathcal{L}_\mu = \sum_k \lfloor w_k \rceil \mu_{E_k} \tag{5.12}$$

$$\mathcal{L}_\sigma = \sum_k \lfloor w_k \rceil (\sigma_{E_k} - \mu_{E_k})^2 \tag{5.13}$$

$$\mathcal{L}_w = \sum_k \bar{w}_k \log(w_k) \tag{5.14}$$

where $\lfloor w_k \rceil$ is the rounded w_k, selecting only the most likely component.

This approach encourages component specialization while maintaining reasonable uncertainty estimates. $\mathcal{L}_\mu$ minimizes the error of the selected component, $\mathcal{L}_\sigma$ aligns the predicted uncertainty with actual error magnitude and $\mathcal{L}_w$ uses cross-entropy to ensure mixture weights reflect component performance. We block gradient propagation through $\lfloor w_k \rceil$ to ensure training stability. We also note that μ_{E_k} is equivalent to the default self-supervised loss used in the literature.

Likewise, during inference, we select the most likely disparity according to the mixture weights, avoiding the averaging effect caused by its uncertainty.

$$d^* = \begin{cases} \mu_1 & \text{if } w_1 > 0.5 \\ \mu_2 & \text{otherwise} \end{cases} \tag{5.15}$$

5.2.4 Arbitrary Resolution via Distribution Interpolation

Our mixture distribution representation enables predictions at arbitrary resolutions without blurring edges, similar to [119]. Unlike standard methods that interpolate depth values directly and average across discontinuities, we interpolate the distribution parameters $(\mu_1, \mu_2, \sigma_1^2, \sigma_2^2, w_1)$ independently. At object boundaries, each component mean remains approximately constant on either side (one for foreground, one for background), while the mixture weight w_1 transitions smoothly. Since the hard selection rule from inference depends on whether $w_1 > 0.5$, sharp depth predictions are preserved despite smooth parameter interpolation.

5.2.5 Depth Distribution

While we work in the disparity space z^{-1}, in most applications we need the distribution over depth z. Since depth is a nonlinear transformation $f(x) = \frac{1}{x}$ of disparity, the resulting distribution is no longer Gaussian.

Working only on the most likely component, we can derive the depth distribution $p(z)$ from the disparity $\mathcal{N}(z^{-1}|\mu, \sigma^2)$ using the change of variables formula:

$$p(z) = \mathcal{N}(z^{-1}|\mu, \sigma^2) \left| \frac{dz^{-1}}{dz} \right| = \frac{1}{z^2} \cdot \mathcal{N}(z^{-1}|\mu, \sigma^2) \tag{5.16}$$

While this distribution is not Gaussian, we can still evaluate its likelihood at any depth value z efficiently. Figure 5.6 shows how the $\frac{1}{z^2}$ factor stretches the distribution and increases uncertainty at greater depths.

We can also find closed-form expressions for the median and mode of this distribution. Since $f(x) = \frac{1}{x}$ is a strictly monotonic transformation in the positives, we have $z_{\text{median}} =$

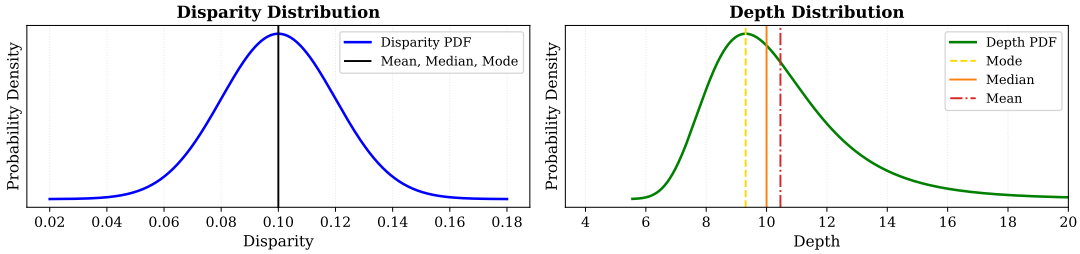


Figure 5.6: Transformation of disparity distribution to depth distribution. Left: Gaussian distribution in disparity space. Right: Resulting reciprocal Gaussian distribution in depth space showing how the nonlinear transformation $f(x) = \frac{1}{x}$ transforms the distribution shape.

$\frac{1}{\mu}$. The mode of $p(z)$ can be found by solving $\frac{\partial \log p(z)}{\partial z} = 0$, yielding $z_{\text{mode}} = \frac{1}{\mu+2\sigma^2}$. The mean can be approximated using a second order Taylor expansion around μ, giving $z_{\text{mean}} \approx \frac{1}{\mu} + \frac{\sigma^2}{\mu^3}$. In practice we use the median as our depth estimate since it doesn't depend on variance, making it more robust.

5.2.6 Prediction of Uncertainty

Variance

Our mixture model provides an estimate of the uncertainty in disparity prediction. To understand what this uncertainty represents, we analyze the variance of the mixture distribution using the Law of Total Variance.

For a mixture of two Gaussians $\mathcal{M}(z^{-1}) = w_1\mathcal{N}(z^{-1}|\mu_1, \sigma_1^2) + w_2\mathcal{N}(z^{-1}|\mu_2, \sigma_2^2)$, where $w_1 + w_2 = 1$, the total variance decomposes into two terms:

$$\text{Var}(Z^{-1}) = \underbrace{\sum_{k=1}^{2} w_k\sigma_k^2}_{\text{Intra-Component Variance}} + \underbrace{\sum_{k=1}^{2} w_k(\mu_k - \mu_{\mathcal{M}})^2}_{\text{Inter-Component Variance}} \tag{5.17}$$

where $\mu_{\mathcal{M}} = w_1\mu_1 + w_2\mu_2$ is the expected disparity.

The first term represents the uncertainty within each component, while the second term captures the disagreement between components. At object boundaries where one component predicts foreground and another predicts background, this inter-component term dominates. This result can be used to compute confidence intervals for the predicted disparity.

Entropy

Although variance quantifies total uncertainty, it is an overestimation when components are well-separated. Entropy, by contrast, offers a more fundamental measure of uncertainty. For our disparity mixture $\mathcal{M}(z^{-1}) = \sum_{i=1}^{2} w_i\mathcal{N}(z^{-1}|\mu_i, \sigma_i^2)$ the differential entropy is $H(Z^{-1}) = -\int p(z^{-1}) \log p(z^{-1})\, dz^{-1}$. While $H(Z^{-1})$ has no general closed form solution, we can establish simple bounds. A lower bound assumes we know which component generated each sample:

$$H(Z^{-1}) \geq H_{\text{lower}} = \sum_{i=1}^{2} w_i H_i \tag{5.18}$$

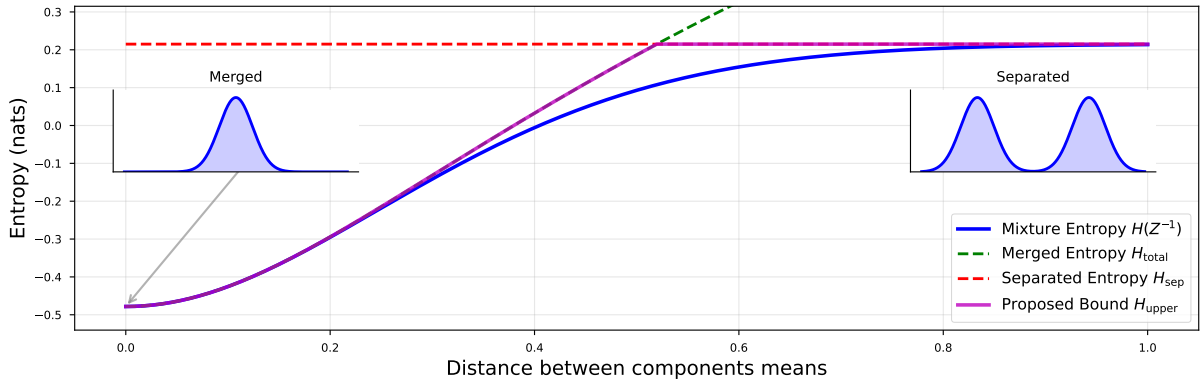


Figure 5.7: Entropy of 1D Gaussian mixture as a function of the distance between component means. The mixture entropy $H(Z^{-1})$ is bounded above by both H_{total} and H_{sep}. The insets show the probability density functions for the merged ($d = 0$) and separated ($d = 1$) extreme cases.

where $H_i = \frac{1}{2}\log(2\pi e\sigma_i^2)$ is the entropy of component i.

Two upper bounds also provide complementary perspectives. The first treats the mixture as a single Gaussian with the total variance:

$$H(Z^{-1}) \leq H_{\text{total}} = \frac{1}{2}\log(2\pi e \cdot \text{Var}(Z^{-1})) \tag{5.19}$$

This bound is exact when both components are fully merged (small inter-component variance). The second considers both components as separated:

$$H(Z^{-1}) \leq H_{\text{sep}} = H_{\text{lower}} + H_w \quad \text{where} \quad H_w = -\sum_{i=1}^{2} w_i \log w_i \tag{5.20}$$

This bound is exact when both components are well-separated (high inter-component variance). The curve in Figure 5.7 suggests taking the minimum of these two upper bounds, which keeps the tightest one in each regime:

$$H(Z^{-1}) \leq H_{\text{upper}} = \min(H_{\text{total}}, H_{\text{sep}}) \tag{5.21}$$

Types of Uncertainty

It is important to clarify what type of uncertainty our mixture model captures. Following the standard taxonomy [56], we distinguish between aleatoric uncertainty (data uncertainty) and epistemic uncertainty (model uncertainty).

Our mixture model primarily captures aleatoric uncertainty, the inherent ambiguity in the depth estimation task. Within this, we can further differentiate between homoscedastic and heteroscedastic uncertainty components:

Homoscedastic uncertainty represents the global, irreducible noise and ambiguity affecting all pixels similarly such as image noise, blur or fundamental limitations of monocular depth estimation.

Heteroscedastic uncertainty on the other hand, represents spatially varying uncertainty that depends on the local image content. This includes ambiguities at object boundaries, transparent or reflective surfaces, or other challenging regions.

With our mixture model, we can separately analyze these two types of aleatoric uncertainty. It is even more clear if we use global variance as described in Section 5.3.5. In this case, components variances captures homoscedastic uncertainty, while the components weights captures heteroscedastic uncertainty.

Regarding epistemic uncertainty, our approach has limited capacity to represent it explicitly. At inference time, the model cannot directly quantify output uncertainty with respect to its own parameters. However, the depth distribution may still hint at epistemic effects: when the network repeatedly produced high error during training, it can learn to keep uncertainty high, thereby indirectly signaling reduced confidence in its prediction. This can also catch issues with the training objective such as higher uncertainty on moving objects.

5.2.7 Edge Sharpness Measure

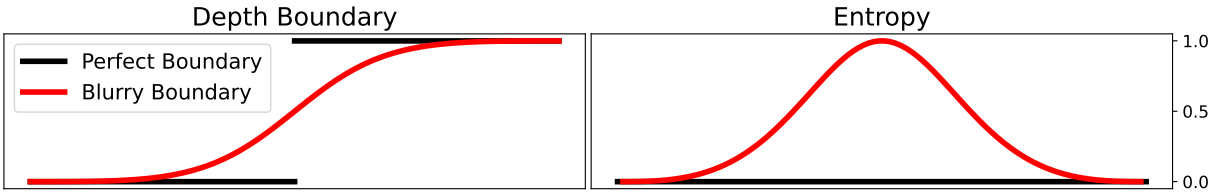


Figure 5.8: Entropy visualization at object borders: low entropy indicates sharp transitions and high entropy indicates blurry ones.

To quantitatively evaluate the sharpness of depth discontinuities, we introduce a border sharpness measure based on the entropy of edge pixels. This measure captures how cleanly the model represents depth discontinuity, with lower values indicating sharper transitions. The measure is computed as follows:

1. Apply a Canny filter to the log depth map to extract edge pixels, forming the set E.
2. For each edge pixel $i \in E$, consider its 3×3 neighborhood $\mathcal{N}_i$ in the depth map.
3. Normalize the depth values d_{ij} in $\mathcal{N}_i$ such that $p_{ij} = \frac{d_{ij} - \min(d_{\mathcal{N}_i})}{\max(d_{\mathcal{N}_i}) - \min(d_{\mathcal{N}_i})}$
4. Compute the Bernoulli entropy for each normalized value:
$$H(p_{ij}) = -p_{ij} \log_2(p_{ij}) - (1 - p_{ij}) \log_2(1 - p_{ij}) \tag{5.22}$$
5. Average the entropy values within each neighborhood to obtain a local sharpness score for each edge pixel:
$$S_i = \frac{1}{|\mathcal{N}_i|} \sum_{j \in \mathcal{N}_i} H(p_{ij}) \tag{5.23}$$
6. The final sharpness measure is the average over all edge pixels $\mathcal{E}$:
$$S = \frac{1}{|\mathcal{E}|} \sum_{i \in \mathcal{E}} S_i \tag{5.24}$$

This entropy-based approach captures the decisiveness of depth transitions. In Figure 5.8, values close to 0 or 1 correspond to low entropy and therefore sharp transitions, while values near 0.5 reach the maximum entropy of 1 and indicate blurry transitions. Using $\log_2$ bounds the entropy between 0 and 1, making the measure interpretable.

5.3 Experiments

5.3.1 Datasets

We train and evaluate our method on the KITTI dataset [35], a standard benchmark for self-supervised depth estimation featuring stereo pairs and video sequences from urban and highway driving. Following [115], we use Eigen Zhou splits [25] and report results on the KITTI Eigen Benchmark test set with improved ground truth annotations from [123]. VKITTIv2 [11, 33] is used for its fine-grained synthetic annotations.

5.3.2 Implementation Details

Our encoder uses a ConvNeXt-Base backbone [76] pretrained on ImageNet [21]. The decoder follows a U-Net-like architecture with 5 output channels: $(\mu_1, \mu_2, \sigma_1, \sigma_2, w_1)$. We use two decoders: Monodepth2 (MD2) [38] and HR-Depth [78].

Each configuration is trained with monocular (M), stereo (S) and both (MS) training data. We train our model for 30 epochs with a batch size of 8 using the Adam optimizer [58], an initial learning rate of 10^{-4} decreasing to 10^{-5} after 15 epochs. For data augmentation, we apply random horizontal flips and color jitter at a resolution of 640×192.

Our final loss is computed as $\mathcal{L} = \mathcal{L}_\mu + \mathcal{L}_\sigma + \lambda_w \mathcal{L}_w + \lambda_s \mathcal{L}_s$ with $\mathcal{L}_s$ the standard smoothness loss [37] applied to both mean predictions, $\lambda_s = 10^{-3}$ and $\lambda_w = 0.1$ to ensure that the gradient of $\mathcal{L}_w$ does not dominate the gradient of $\mathcal{L}_\mu$. We also use the contributions of [38], computing the minimum loss over the different sources and filtering out static pixels.

5.3.3 Quantitative Results

We compare our method on the KITTI Eigen Benchmark test set in Table 5.1 using both image and pointcloud metrics. All methods use the ConvNeXt-Base [76] backbone, and we report results for our method using both MD2 and HR-Depth decoders. We observe modest overall improvements, as expected since edges constitute a small fraction of pixels. We note that when object depth is poorly estimated, sharper boundaries can negatively impact metrics; however, this is mitigated by regions with accurate depth where sharpness is beneficial.

On the edge sharpness measure reported in Table 5.2, which isolates edge quality from overall depth accuracy, our method significantly outperforms baseline methods for both decoders, demonstrating superior performance in predicting discontinuous object boundaries.

We evaluate edge completeness on VKITTIv2 in Table 5.3 using the metric from [115]. Our method shows improvements over most baseline methods, indicating that our method is able to more accurately find edges in the depth maps.

Method	Train Data	Image-based				Pointcloud-based		
		MAE↓	RMSE↓	AbsRel↓	LogSI↓	Chamfer↓	F-Score↑	IoU↑
SfM-Learner [149]	M	1.98	4.57	10.69	15.80	0.73	44.77	30.03
Klodt [60]	M	1.96	4.54	10.49	15.86	0.72	45.26	30.40
Monodepth2 [38]	M	1.84	4.11	8.82	13.10	0.71	46.64	31.62
Johnston [51]	M	1.83	**3.99**	8.85	12.89	0.71	45.72	30.78
HR-Depth [78]	M	1.80	4.04	8.65	**12.75**	0.69	47.35	32.10
Ours MD2	M	1.81	4.14	8.92	13.35	0.68	46.52	31.53
Ours HR-Depth	M	**1.76**	4.08	**8.50**	12.86	**0.66**	**48.46**	**33.11**
Garg [34]	S	1.60	3.75	7.65	11.39	0.60	53.28	37.33
Monodepth [37]	S	1.61	3.72	7.73	11.57	0.64	51.25	35.45
SuperDepth [96]	S	1.64	3.77	7.81	11.63	0.63	52.30	36.40
Ours MD2	S	1.56	3.63	7.37	11.42	0.60	52.32	36.75
Ours HR-Depth	S	**1.52**	**3.55**	**7.17**	**11.03**	**0.58**	**53.57**	**37.85**
Depth-VO-Feat [142]	MS	1.63	3.72	7.70	11.64	0.62	52.01	36.15
Monodepth2 [38]	MS	1.61	3.62	7.90	10.99	0.64	50.50	34.98
FeatDepth [111]	MS	1.60	3.60	7.80	11.01	0.65	49.99	34.51
CADepth [137]	MS	1.63	3.60	8.09	10.84	0.66	49.32	34.06
DiffNet [147]	MS	1.62	3.63	7.97	10.93	0.65	49.63	34.23
HR-Depth [78]	MS	1.58	**3.56**	7.70	**10.68**	0.62	51.49	35.93
Ours MD2	MS	1.57	3.72	7.39	11.37	0.62	51.90	36.29
Ours HR-Depth	MS	**1.54**	3.71	**7.28**	11.26	**0.61**	**52.40**	**36.77**

Table 5.1: Comparison of methods on KITTI Eigen Benchmark test set. Results are taken from [115] so that all models use the same backbone and data pipeline. Training data is either (M) monocular, (S) stereo or (MS) both.

Method	M	MS	S
MD2	0.499	0.520	0.509
Ours MD2	**0.350**	**0.336**	**0.387**
HR-Depth	0.471	0.456	0.516
Ours HR-Depth	**0.373**	**0.357**	**0.385**

Table 5.2: Edge Sharpness Measure for all methods and data types. Lower values indicate sharper edges.

Method	M	MS	S
MD2	3.53	3.70	3.68
Ours MD2	**3.31**	**3.12**	**3.19**
HR-Depth	**2.86**	2.97	3.08
Ours HR-Depth	2.88	**2.72**	**2.79**

Table 5.3: Edge completeness measure on VKITTIv2, representing the average distance in pixels from predicted edges to labeled edges; lower is better.

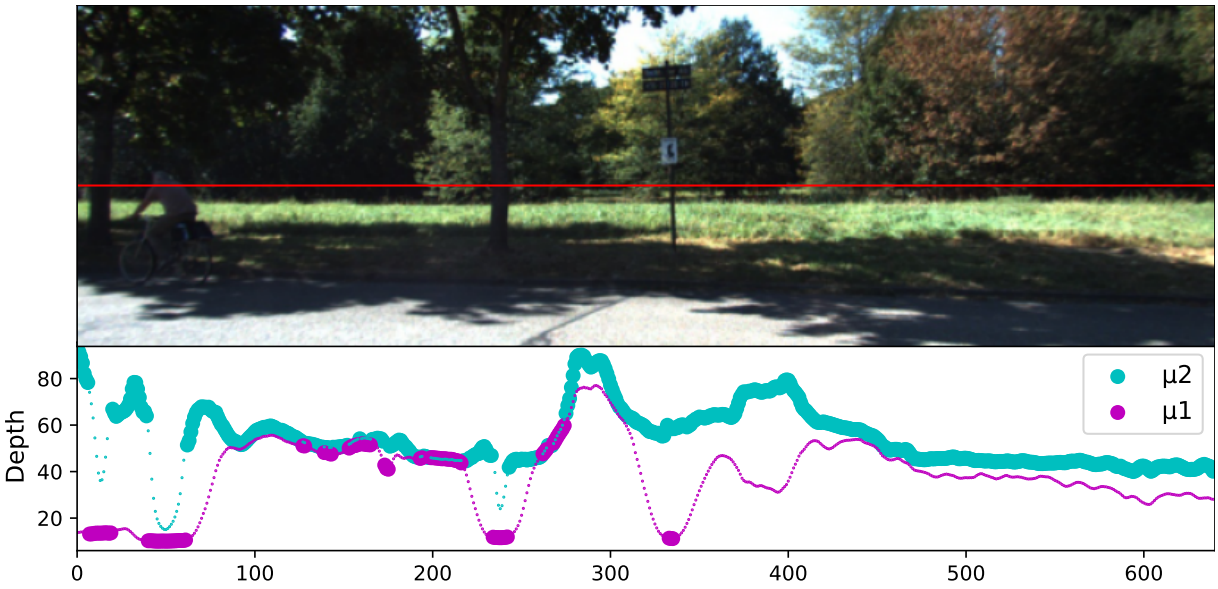


Figure 5.9: Depth distribution along a single row. Bold points indicate the component selected by the mixture weight. We see that both components complementarily over/underestimate obstacle width smoothly and that it's the selection process that creates the discontinuity.

5.3.4 Qualitative Results

The mixture-weight visualization in Figure 5.4(3) shows how different objects are assigned different components at their borders. Figure 5.9 further illustrates how the two components tend to work complementarily: one component typically underestimates object width while the other overestimates it. The mixture weights then create the discontinuity at the optimal position by switching between the two components when one become more likely than the other, which is why Figure 5.10 exhibits sharper depth boundaries once interpolation is removed.

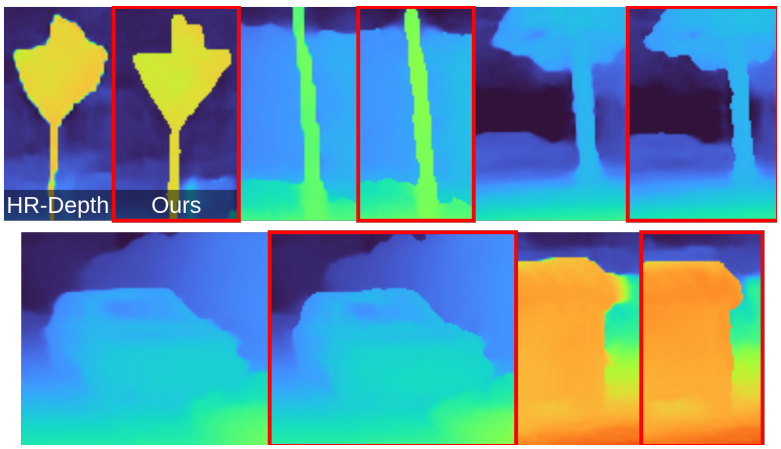


Figure 5.10: Zoomed-in view of objects with sharp borders. Our method produces noticeably sharper boundary predictions compared to the baseline. Full images are available in Figure A.1 of the appendix.

The point cloud comparison in Figure 5.1 shows that other methods typically produce floating artifacts at object boundaries, resulting in spurious points between foreground and background. In contrast, our method generates clean point clouds with sharp object boundaries, effectively eliminating these artifacts.

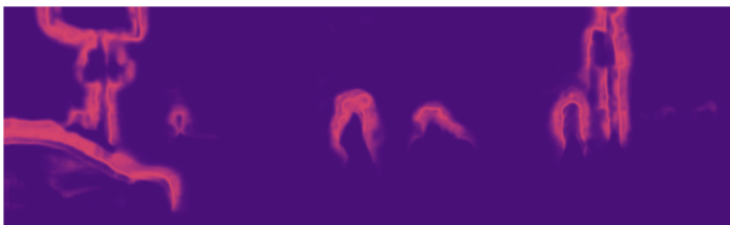
5.3.5 Ablation Studies

In order to understand the contribution of the uncertainty propagation, we perform an ablation study on the KITTI Eigen Benchmark test set with the HR-Depth decoder. Instead of propagating the uncertainty through the loss, we use a single global color variance across the image (with learned scale). In this case we find that the model is not able to separate the two components, underlining the importance of the uncertainty propagation.

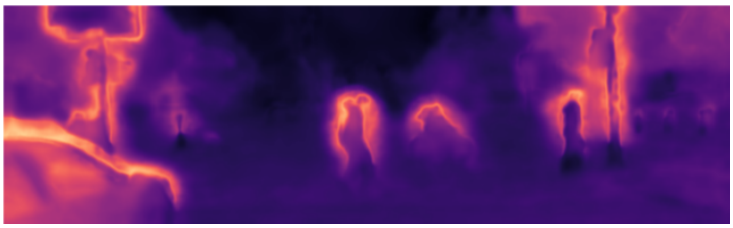
Given these results, we then tried to replace the predicted per pixel disparity variances with a single global disparity variance. Table 5.4 shows that this modification provided slight improvements, suggesting that while variance propagation through the loss is crucial for component separation, the mixture means are the primary drivers of the resulting color variances. We also tested one global variance per component, but in this case we observed that both variances converged to similar values, effectively reducing the model to the single variance setting.

Variance Type	Image-based				Pointcloud-based		
	MAE↓	RMSE↓	AbsRel↓	LogSI↓	Chamfer↓	F-Score↑	IoU↑
Per Pixel	1.54	3.71	7.28	11.26	0.61	52.40	36.77
Global	1.51	3.55	7.10	10.88	0.61	52.77	37.14

Table 5.4: Performance comparison using per pixel or global variance estimation. Global variance yields slightly better results.



(a) Global variance



(b) Per-pixel variance

Figure 5.11: Comparison of entropy upper bound for the two types of variance.

Uncertainty Behavior

Comparing per-pixel and global variance settings, we observe in Figure 5.11 that both configurations have increased entropy at object boundaries coming from the inter-component variance term. However, per-pixel variance prediction allows the model to better capture changes in uncertainty across the image, such as higher uncertainty on objects borders and reduced uncertainty in flat regions such as the ground. Figure 5.12 also provides a top down view of the predicted depth distribution along a row of the image, showing the predicted log-likelihood for every depth values, with the black line being the median depth and blue circle being the LiDAR points. It demonstrates that lidar points fall within high likelihood regions of the predicted distribution and that the uncertainty gets larger with increasing distance.

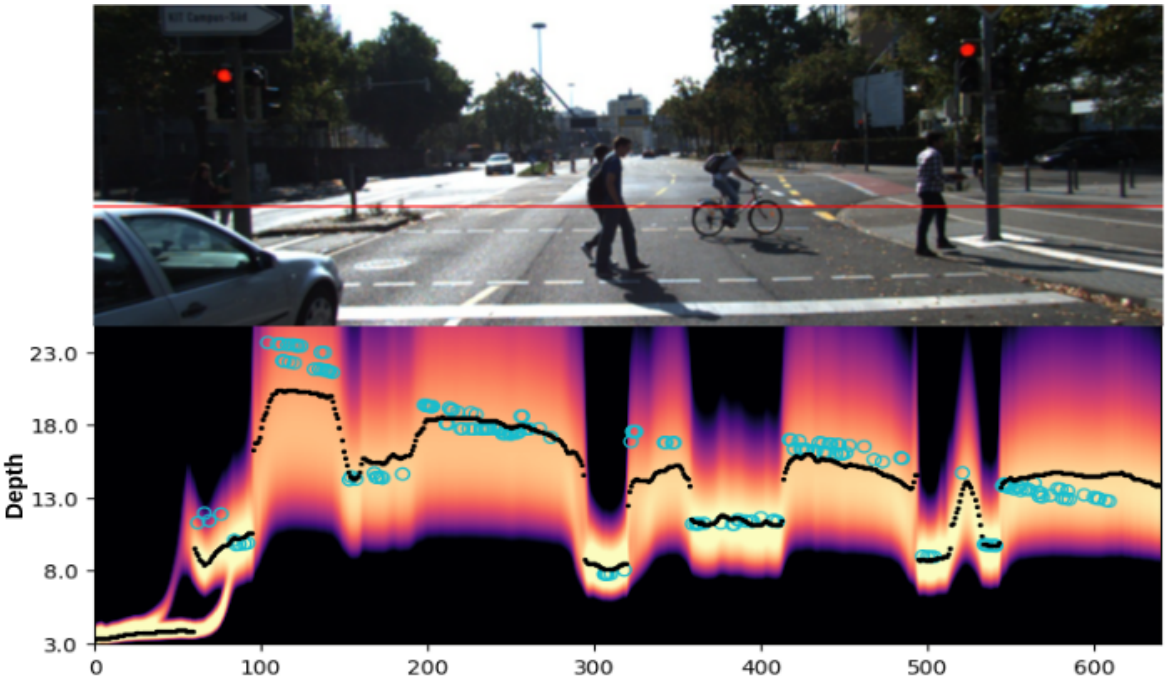


Figure 5.12: Top-down view of the depth distribution along the red row shown in the input image. The color gradient represents the log-likelihood of the predicted distribution, the black line shows the predicted depth, and blue circles indicate LiDAR ground truth points. The probability density follows the expected pattern of Figure 5.2.

Finally, Table 5.5 quantifies the difference, showing the negative log-likelihood (NLL) of LiDAR points under the predicted depth distribution for both variance settings. We report the NLL with respect to the full mixture distribution, the distribution of the best component and for reference, the optimal variance obtained by solving for the global variance that minimizes the NLL on each image (requires ground truth). We observe that per-pixel variance yields better likelihoods across the board thanks to its increased flexibility.

5.3.6 Alternative Approaches and Negative Results

During development of this method, we explored several alternative design choices that ultimately proved unsuccessful. These negative results provide insight into what is critical for the success of our approach.

Variance Type	Negative Log-Likelihoods ↓		
	Mixture	Best Component	Optimal Variance
Per Pixel	0.023	0.024	-0.202
Global	0.149	0.172	-0.219

Table 5.5: Negative log-likelihoods of LiDAR annotations with respect to the predicted distribution for per-pixel and global variance settings. We report results using the likelihood of the full mixture, the best component only, and solving for the optimal global variance on each image for comparison.

We experimented with barycentric interpolation as an alternative to bilinear interpolation. Barycentric interpolation identifies the three nearest pixels to the reprojected position and constructs a plane through these points in the image coordinate and color space, with the reprojected color determined by its position on this plane. While this approach offers a linear relationship with disparity (compared to bilinear interpolation's quadratic relationship), potentially reducing approximation errors in uncertainty propagation, the results were significantly worse. We attribute this to degradation in color averaging that interfered with depth estimation even without the mixture representation.

Given our edge entropy metric, a direct approach to improving boundary sharpness would be to use entropy as an explicit training objective. We tested a loss function based on the squared distance from binary values, which like entropy is maximal at 0.5 and minimal at 0 and 1, but with linear gradients to avoid instability near the extremes. This consistently produced false edges in low-gradient regions where objects are distant, creating staircase artifacts and degrading overall depth quality.

We also investigated alternatives to the SSIM component of our photometric loss, including pure L1 and L2 losses which can be interpreted as negative log-likelihoods of Laplacian and Gaussian distributions respectively. However, SSIM consistently outperformed these alternatives due to its sensitivity to texture and structural information in the image, which proves crucial when comparing synthesized and target views in the self-supervised framework.

5.4 Conclusion

In this chapter, we introduced a self-supervised monocular depth estimation method that models each pixel's depth as a mixture of distributions. Our approach achieves competitive accuracy on standard benchmarks and drastically improves depth and pointcloud discontinuities. The proposed distribution propagation technique and mixture-based representation open new possibilities for robust depth estimation and uncertainty modeling in self-supervised learning. These advances have potential applications in related tasks such as optical flow, and can be extended to more complex distributions as well as the integration of temporal and pose uncertainty.

Furthermore, the mixture weights produced by our model provide a rich, interpretable representation of scene structure. These weights highlight object boundaries and regions with distinct depth characteristics, suggesting a possible direction for self-supervised instance segmentation.

Chapter 6

Conclusions and Perspectives

This thesis addresses two structural limitations of self-supervised monocular depth estimation: the inability to recover metric scale, and the blurring of depth discontinuities at object boundaries. Both problems are motivated by the practical requirements of Camera Monitor Systems (CMS) deployed on buses and trucks, where accurate distance measurements and sharp object localization are needed for safety-critical applications. The two contributions developed in this work address these challenges by incorporating geometric and distributional structure into the self-supervised learning framework.

6.1 Summary of Contributions

Our first contribution integrates ground plane priors into self-supervised depth training through novel loss functions and a learned attention mechanism. The ground constraint loss enforces geometric consistency between the predicted depth and the theoretical flat ground derived from camera extrinsic parameters, while the attention mechanism identifies which image regions correspond to reliable ground. This design addresses both scale ambiguity and camera overfitting simultaneously: the ground plane provides the missing metric reference, and conditioning on extrinsic parameters allows the model to adapt to different camera configurations without retraining. On KITTI, the method achieves competitive metric depth accuracy among self-supervised approaches. More notably, it demonstrates strong robustness to camera pose variations, maintaining reasonable performance under pitch, roll, and yaw perturbations of 5 to 15 degrees where baseline methods degrade significantly. Zero-shot transfer from KITTI to DDAD, a dataset with different camera setups including side and rear views, reduces relative error from 0.403 to 0.154, outperforming even supervised methods with far more parameters. For CMS applications, this means that a single model can serve cameras mounted at different positions, from the middle of a cab door to the top of the cabin at a steep downward angle. The method does have limitations: it requires visible flat ground during training, and the attention regularization threshold must be set per dataset. Performance degrades on prolonged slopes or terrain where flat ground is absent, though the attention mechanism provides a diagnostic signal in such cases by indicating low ground confidence.

Our second contribution represents per-pixel depth as a two-component Gaussian mixture distribution, explicitly modeling the fact that pixels at object boundaries observe both foreground and background. The mixture is propagated through the entire self-supervised pipeline, from reprojection through bilinear interpolation to loss computation, using first-order Taylor approximations to maintain differentiability. A competitive train-

ing scheme ensures that each component specializes, with one capturing the foreground depth and the other the background. At inference, the dominant component is selected, producing sharp depth transitions rather than blurred averages. Experiments show a 30 to 40 percent reduction in edge entropy compared to baselines, and point cloud reconstructions are free of the floating artifacts that typically appear between foreground and background surfaces. The distributional representation also provides subpixel depth accuracy at edges, which matters particularly at long range: for trailer end detection at 12 meters, where a single pixel can correspond to a large distance interval, the mixture enables finer localization than a scalar prediction. Beyond boundary sharpness, the mixture weights and component variances constitute a calibrated uncertainty estimate that can be propagated to downstream modules such as a physics model for collision prediction. The main limitation is that edges represent a small fraction of image pixels, so improvements on global image metrics are modest.

Both contributions share several properties worth noting. They are structure-aware: one exploits the geometric relationship between camera and ground plane, while the other exploits the distributional structure of depth at occlusion boundaries. Both operate within the self-supervised paradigm, requiring no depth annotations and remaining compatible with the data-rich, label-poor setting of CMS deployments. Both are also modular, designed to integrate into existing self-supervised pipelines without requiring fundamental architectural changes.

6.2 Relation to Current Research Trends

The field of monocular depth estimation has evolved considerably during the course of this thesis. One significant development has been the use of self-supervised visual features as backbones for depth estimation. Models such as DINOv2 [91] learn strong semantic representations from large-scale unlabeled image collections, and these features transfer well to depth prediction tasks. When combined with supervision from synthetic data or large scale dataset, this approach has proven effective: Depth Anything [138] and its successors [74, 139] demonstrate that pseudo-labeled real data, generated using teacher models trained on synthetic datasets, can produce monocular depth networks with strong generalization to diverse scenes. MoGe [127] shows that careful selection of training supervision, combined with an affine-invariant representation, enables accurate monocular geometry estimation across open-domain images. These approaches achieve generalization that was previously out of reach for methods trained on lower scale datasets.

A related trend is the emergence of feed-forward models for full scene reconstruction from one or multiple images. DUSt3R [129] and MASt3R [71] reformulate multi-view reconstruction as a direct regression problem, predicting 3D point maps from image pairs using transformer architectures trained on large supervised datasets. More recent models such as VGGT [126] and π^3 [130] extend this idea to handle arbitrary numbers of input views, jointly predicting camera parameters, depth maps, and point correspondences in a single forward pass. These models achieve strong monocular depth performance as a byproduct of their broader reconstruction objective.

Beyond depth estimation itself, 3D scene understanding is increasingly recognized as a valuable prior for other perception tasks. In robotic manipulation, depth and spatial reasoning provide the geometric grounding needed to interact with objects in 3D space [50], and vision-language models benefit from explicit depth information for precise spatial understanding [12]. This broader trend suggests that the methods developed in this thesis,

like the depth distributions or the metric-scale recovery could find applications beyond their original automotive context.

The contributions of this thesis are complementary to these developments. A foundation model backbone does not know the camera's height above the ground, and a model trained on diverse scenes does not inherently guarantee metric scale or sharp depth discontinuities at boundaries. The ground-constraint mechanism is agnostic to the choice of encoder and could be applied on top of features from DINOv2 or any other backbone. Similarly, the mixture representation is independent of the architecture producing the depth estimates, and its distributional output could provide principled uncertainty estimates for downstream reasoning. As monocular depth estimation moves toward larger models and richer pretraining, the structural priors developed here remain relevant as targeted solutions to problems that scale and architecture alone do not solve. Moreover, this work shows that these structural priors can be integrated into the self-supervised learning process, providing an alternative path to improving depth estimation performance without relying solely on scaling supervised data.

6.3 Future Directions

Several extensions of this work present themselves as promising directions for future research.

A natural next step for both contributions is to incorporate temporal information. While we currently estimate depth from individual frames, its metric scale depends on ground visibility in each image. Propagating scale estimates across consecutive frames would allow the model to maintain metric accuracy through sequences where flat ground is temporarily occluded or when traversing bumpy terrain. For shaper boundaries, enforcing temporal coherence of predictions would ensure that depth discontinuities remain sharp and consistent across frames, avoiding flickering artifacts that would be distracting or misleading in a CMS display. Video-level reasoning could also benefit the pose estimation component of the self-supervised pipeline, providing more stable training signals.

Camera Monitor Systems equip vehicles with cameras at multiple positions around the vehicle. While the overlapping fields of view between these cameras are typically narrow, this overlap is useful for cross-camera calibration, allowing the system to verify or update camera extrinsic parameters over time as mounting positions shift due to vibration or mechanical stress. This multi-camera setting also opens the possibility of multi-view self-supervised training, where the model learns to predict consistent depth maps across different viewpoints. We could use our ground-constraint mechanism to provide scale for all views even in challenging conditions where only one camera has a clear view of the ground (e.g front camera providing scale for the side cameras).

The contributions of this thesis could also benefit from integration with recent advances in visual representation learning. Replacing the current CNN-based encoders with features from self-supervised vision models such as DINOv2 [91] could improve the depth network's ability to generalize across scenes and conditions, while the ground constraint and distributional output would continue to address metric scale and boundary precision on top of these richer features.

6.4 Conclusion

This thesis started from a practical question: how to extract reliable three-dimensional information from the cameras of a CMS system, using only the unlabeled video these cameras produce during normal operation. The two methods developed here show that self-supervised depth estimation, when augmented with appropriate structural priors, can achieve metric accuracy and boundary precision that bring it closer to the requirements of safety-critical applications. The ground plane provides a geometric anchor for metric scale, and distributional depth provides both sharper boundaries and uncertainty quantification. These are targeted solutions to specific problems, but they illustrate a broader point: the gap between self-supervised and supervised depth estimation can be narrowed not only by scaling data and models, but by building the right geometric and statistical structure into the learning process.

Appendix A

Mathematical Derivations and Visualizations

This appendix provides the complete mathematical derivations for the formulation presented in Chapter 5. These detailed derivations support the main text and are included here for completeness.

A.1 Reprojection Equation with Disparity

Given the disparity distribution for each pixel in the target image, we need to compute where these pixels project into the source images. Since we have a mixture model, we perform this projection separately for each component.

In standard 3D geometry, a point $\mathbf{P}_t$ in the target camera frame projects to point $\mathbf{P}_s$ in the source camera frame through:

$$\mathbf{P}_s = R\mathbf{P}_t + \mathbf{t} \tag{A.1}$$

where R and $\mathbf{t}$ are the rotation matrix and translation vector between frames.

To work in image coordinates, we relate 3D points to pixel coordinates. For a pixel $\tilde{\mathbf{p}} = (u, v, 1)$ with disparity z^{-1} (inverse depth), the corresponding 3D point is:

$$\mathbf{P} = \frac{1}{z^{-1}} K^{-1}\tilde{\mathbf{p}} \tag{A.2}$$

where K is the camera intrinsic matrix.

Substituting this into our projection equation:

$$\frac{1}{z_s^{-1}} K^{-1}\tilde{\mathbf{p}}_s = R\left(\frac{1}{z_t^{-1}} K^{-1}\tilde{\mathbf{p}}_t\right) + \mathbf{t} \tag{A.3}$$

Rearranging, we get our final expression for the reprojection:

$$\frac{z_t^{-1}}{z_s^{-1}}\tilde{\mathbf{p}}_s = K\left(RK^{-1}\tilde{\mathbf{p}}_t + z_t^{-1}\mathbf{t}\right) \tag{A.4}$$

This defines our reprojection function f_{reproj} that maps a disparity value z_t^{-1} at pixel $\tilde{\mathbf{p}}_t = (u_t, v_t, 1)$ to a position $\tilde{\mathbf{p}}_s = (u_s, v_s)$ in the source image:

$$\mathbf{p}_s = f_{\text{reproj}}(z_t^{-1}, \tilde{\mathbf{p}}_t) = \frac{z_s^{-1}}{z_t^{-1}} K\left(RK^{-1}\tilde{\mathbf{p}}_t + z_t^{-1}\mathbf{t}\right) \tag{A.5}$$

A.2 Error Variance Approximation

A.2.1 SSIM Variance

Here we derive the approximation to the variance of the SSIM (Structural Similarity [133]), which is crucial for our uncertainty propagation framework.

First, the formula of the SSIM for x, y two patches of pixels of size N:

$$\text{SSIM}(x, y) = \frac{(2\bar{x}\bar{y} + C_1)(2\sigma_{xy} + C_2)}{(\bar{x}^2 + \bar{y}^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}$$

For simplicity we use shorthands that follow the order above:

$$\text{SSIM}(x, y) = \frac{AB}{CD} = \frac{P}{Q}$$

In our case we specifically want to compute the distribution of the SSIM with respect to the predicted color distribution at the center of the patch $X = \mathcal{N}(\mu_X, \sigma_X^2)$ and consider the neighborhood of the patch as well as the target y as constants.

Using linear error propagation, the mean is simply $\text{SSIM}(\mu_X, y)$ and we can approximate the variance of the SSIM as follows:

$$\sigma_{\text{SSIM}}^2 = \left(\frac{\partial \text{SSIM}}{\partial x}\right)^2 \cdot \sigma_X^2$$

We can decompose this derivative as a function of all the SSIM input variables that depend on x using the chain rule:

$$\frac{\partial \text{SSIM}}{\partial x} = \frac{\partial \text{SSIM}}{\partial \bar{x}} \cdot \frac{\partial \bar{x}}{\partial x} + \frac{\partial \text{SSIM}}{\partial \sigma_X^2} \cdot \frac{\partial \sigma_X^2}{\partial x} + \frac{\partial \text{SSIM}}{\partial \sigma_{XY}} \cdot \frac{\partial \sigma_{XY}}{\partial x}$$

Computing each of these derivatives independently:

$$\begin{aligned}
\frac{\partial \text{SSIM}}{\partial \bar{x}} &= 2\frac{\bar{y}BQ - \bar{x}DP}{Q^2} & \frac{\partial \bar{x}}{\partial x} &= \frac{1}{N} \\
\frac{\partial \text{SSIM}}{\partial \sigma_X^2} &= -\frac{CP}{Q^2} & \frac{\partial \sigma_X^2}{\partial x} &= \frac{2(x - \bar{x})}{N} \\
\frac{\partial \text{SSIM}}{\partial \sigma_{XY}} &= \frac{2A}{Q} & \frac{\partial \sigma_{XY}}{\partial x} &= \frac{y - \bar{y}}{N}
\end{aligned}$$

Combining all of these and reducing to the same denominator:

$$\frac{\partial \text{SSIM}}{\partial x} = \frac{2}{NQ^2}[\bar{y}BQ - \bar{x}DP + AQ(y - \bar{y}) - CP(x - \bar{x})]$$

Therefore, the SSIM can be modeled as a normal random variable:

$$\text{SSIM} \sim \mathcal{N}(\mu_{\text{SSIM}}, \sigma_{\text{SSIM}}^2)$$

A.2.2 Converting SSIM to a Cost Function

Since SSIM is a similarity metric with range $[-1, 1]$, we convert it to a cost function $\mathcal{C}_{\text{SSIM}}$ with range $[0, 1]$:

$$\mathcal{C}_{\text{SSIM}} = \frac{1 - \text{SSIM}}{2}$$

This means that when two patches are perfectly correlated with an SSIM of 1, the cost will be 0. Alternatively, when they are inversely correlated with an SSIM of -1, the cost will be 1.

The parameters of the cost function are then adapted to match the SSIM distribution:

$$\mathcal{C}_{\text{SSIM}} \sim \mathcal{N}(\mu_{\mathcal{C}}, \sigma_{\mathcal{C}}^2) \quad \text{where} \quad \mu_{\mathcal{C}} = \frac{1 - \mu_{\text{SSIM}}}{2} \quad \text{and} \quad \sigma_{\mathcal{C}} = \frac{\sigma_{\text{SSIM}}}{2}$$

A.2.3 L1 Distribution

In addition to SSIM, we also need to compute the distribution of the L1 loss. The L1 loss for a pixel is defined as the absolute difference between the predicted and target values:

$$\mathcal{L}_{L1}(x, y) = |x - y|$$

The derivative of the L1 loss with respect to x is:

$$\frac{\partial \mathcal{L}_{L1}}{\partial x} = \begin{cases} 1 & \text{if } x > y \\ -1 & \text{if } x < y \end{cases}$$

Since we are evaluating at the mean μ_X, the sign is determined by comparing μ_X with the target y. Following our error propagation framework, the variance of the L1 loss is:

$$\sigma_{L1}^2 = \left(\frac{\partial \mathcal{L}_{L1}}{\partial x} \right)^2 \cdot \sigma_X^2 = \sigma_X^2$$

The mean of the L1 distribution is simply:

$$\mu_{L1} = |\mu_X - y|$$

Similar to SSIM, we model the L1 loss as a normal random variable:

$$\mathcal{L}_{L1} \sim \mathcal{N}(\mu_{L1}, \sigma_{L1}^2)$$

This approximation works well in our uncertainty propagation framework and allows us to effectively combine the L1 and SSIM distributions.

A.2.4 Merging SSIM and L1 Losses

In practice, our loss function is a weighted combination of SSIM and L1 loss:

$$\mathcal{L} = 0.85 \cdot \mathcal{C}_{\text{SSIM}} + 0.15 \cdot \mathcal{L}_{L1}$$

To merge the standard deviations of both losses, we weight them accordingly:

$$\sigma_{\mathcal{L}} = 0.85 \cdot \sigma_{\mathcal{C}} + 0.15 \cdot \sigma_{L1}$$

This combined uncertainty estimate captures the overall uncertainty in our loss function and is propagated back to guide the learning of our mixture model parameters.

While all these equations appear complex, they are all computed analytically and the computational cost is very low.

A.3 Additional Visualization

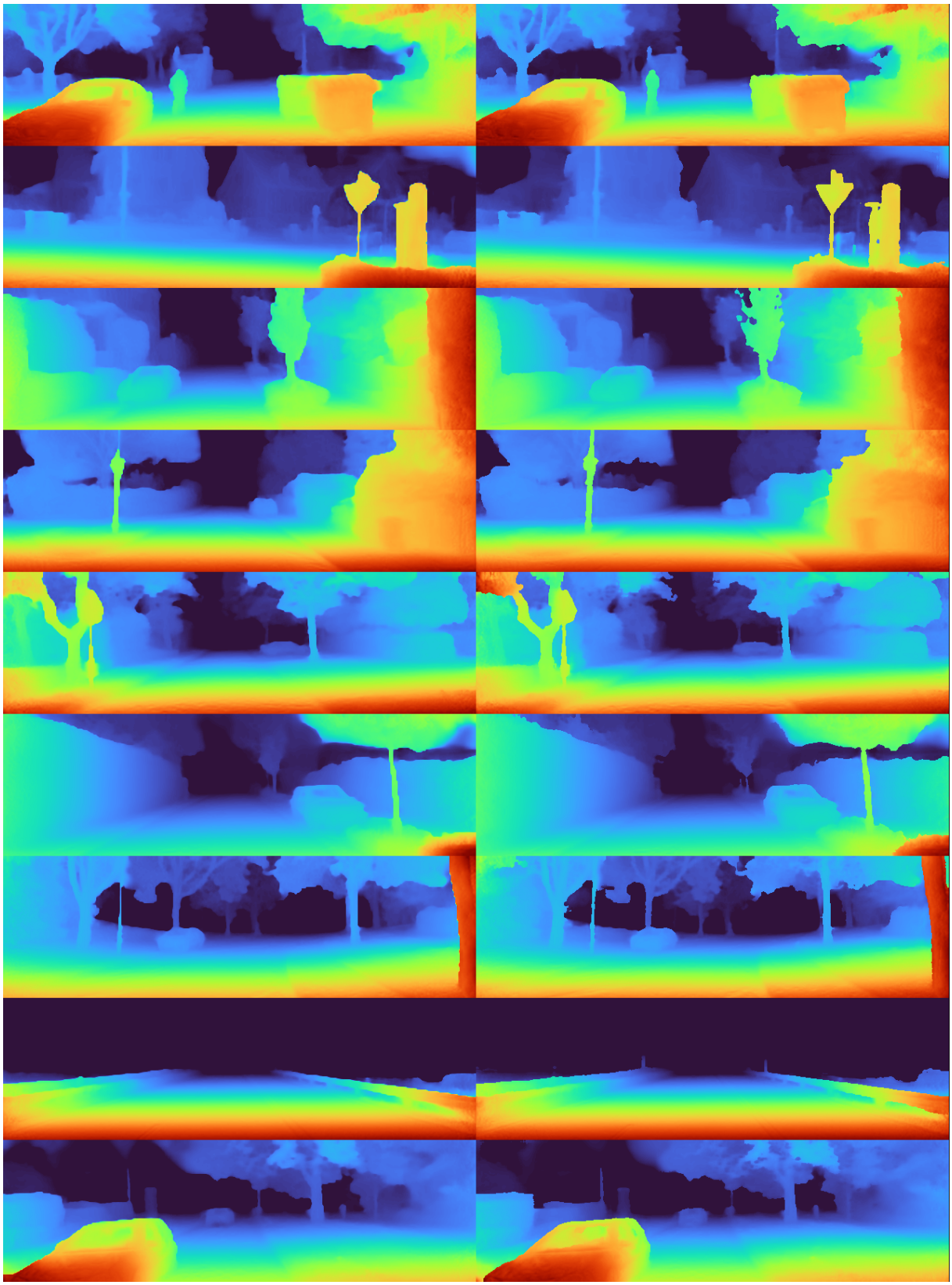


Figure A.1: Visualisations containing objects from Figure 5.10 Left: HR-Depth, Right: Ours. Blurring is visible as a thin halo around edges when zoomed in.

FOLIO ADMINISTRATIF

THESE DE L'INSA LYON, MEMBRE DE L'UNIVERSITE DE LYON

NOM : **CECILLE** DATE de SOUTENANCE : **12 juin 2026**

Prénoms : **Aurélien**

TITRE : **Profondeur Monoculaire : Apprentissage Auto-Supervisé Intégrant la Structure de la Scène**

NATURE : **Doctorat** Numéro d'ordre : **()**

École Doctorale : **École doctorale Informatique et Mathématiques de Lyon**

Spécialité : **Informatique**

RÉSUMÉ :
Cette thèse porte sur l'estimation auto-supervisée de profondeur monoculaire dans le contexte des retroviseurs par caméra pour véhicules lourds. Ce cadre fournit de grandes quantités de vidéos non annotées, mais pose deux limite majeures ont été rencontré pour l'usage en systèmes réels : l'absence d'échelle métrique et le flou des discontinuités de profondeur aux contours des objets.
Une première contribution introduit une contrainte géométrique liée au sol dans l'apprentissage auto-supervisé afin de retrouver une profondeur métrique à partir des seuls paramètres de la caméra, grâce à un a priori analytique de profondeur au sol et à un mécanisme d'attention apprenant où cet a priori est fiable. Les résultats montrent une amélioration de la robustesse aux variations de position de caméra et une bonne capacité de généralisation sur des caméras et jeux de données non vus lors de l'apprentissage.
Une seconde contribution représente la profondeur de chaque pixel par un mélange à deux composantes afin de modéliser explicitement les ambiguïtés aux frontières d'occultation. Cette formulation produit des contours plus nets, réduit les artefacts flottants dans les nuages de points et fournit une estimation d'incertitude exploitable en aval.
Dans son ensemble, ce travail montre que l'intégration d'a priori structurels améliore l'estimation auto-supervisée de profondeur monoculaire au-delà de ce que permettent les seuls objectifs photométriques standards.

MOTS-CLÉS : **Estimation de Profondeur, Caméra Monoculaire, Apprentissage Auto-Supervisé, Échelle Métrique, Bordure Nette**

Laboratoire(s) de recherche : **Laboratoire d'InfoRmatique en Image et Systèmes d'information (LIRIS)**

Directeurs de thèse : **Stefan DUFFNER, Franck DAVOINE**

Président du Jury : **(président, nommé au moment de la soutenance)**

Composition du Jury :

David PICARD, Philippe XU, Véronique CHERFAOUI, Céline TEULIÈRE
Liming CHEN, Franck DAVOINE, Stefan DUFFNER

Bibliography

[1] Panos Achlioptas, Olga Diamanti, Ioannis Mitliagkas, and Leonidas J. Guibas. Learning representations and generative models for 3d point clouds. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 40–49, 2018.

[2] Vasileios Arampatzakis, George Pavlidis, Nikolaos Mitianoudis, and Nikos Papamarkos. Monocular depth estimation: A thorough review. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(4):2396–2414, 2023.

[3] Dylan Auty and Krystian Mikolajczyk. Monocular depth estimation using cues inspired by biological vision systems. In *2022 26th International Conference on Pattern Recognition (ICPR)*, pages 4051–4057. IEEE, 2022.

[4] Sid Yingze Bao, Mohit Bagra, Yu-Wei Chao, and Silvio Savarese. A mixture model for 3d multi-object tracking. In *2011 IEEE Workshop on Challenges and Opportunities in Robot Perception*, pages 599–606, 2011.

[5] Daniel Barath, Jiri Matas, and Jana Noskova. MAGSAC: Marginalizing sample consensus. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10197–10205, 2019.

[6] Herbert Bay, Tinne Tuytelaars, and Luc Van Gool. SURF: Speeded up robust features. In *European Conference on Computer Vision*, pages 404–417. Springer, 2006.

[7] Shariq Farooq Bhat, Ibraheem Alhashim, and Peter Wonka. Adabins: Depth estimation using adaptive bins. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 4009–4018, 2021.

[8] Mario Bijelic, Tobias Gruber, Fahim Mannan, Florian Kraus, Werner Ritter, Klaus Dietmayer, and Felix Heide. Seeing through fog without seeing fog: Deep multimodal sensor fusion in unseen adverse weather. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11682–11692, 2020.

[9] Christopher M Bishop. Mixture density networks. In *Technical Report NCRG/94/004, Neural Computing Research Group, Aston University*, 1994.

[10] Aleksei Bochkovskii, Amaël Delaunoy, Hugo Germain, Marcel Santos, Yichao Zhou, Stephan R Richter, and Vladlen Koltun. Depth pro: Sharp monocular metric depth in less than a second. *arXiv preprint arXiv:2410.02073*, 2024.

[11] Yohann Cabon, Naila Murray, and Martin Humenberger. Virtual KITTI 2, 2020.

[12] Wenxiao Cai, Iaroslav Ponomarenko, Jianhao Yuan, Xiaoqi Li, Wankou Yang, Hao Dong, and Bo Zhao. SpatialBot: Precise spatial understanding with vision language models. *arXiv preprint arXiv:2406.13642*, 2024.

[13] Aurélien Cecille, Stefan Duffner, Franck Davoine, Thibault Neveu, and Rémi Agier. Groco: Ground constraint for metric self-supervised monocular depth. In *European Conference on Computer Vision*, pages 54–69. Springer, 2024.

[14] Aurélien Cecille, Stefan Duffner, Franck Davoine, Rémi Agier, and Thibault Neveu. Towards sharper object boundaries in self-supervised depth estimation. In *36th British Machine Vision Conference 2025, BMVC 2025, Sheffield, UK, November 24-27, 2025*. BMVA, 2025.

[15] Xiaozhi Chen, Huimin Ma, Ji Wan, Bo Li, and Tian Xia. Multi-view 3d object detection network for autonomous driving. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1907–1915, 2017.

[16] Xingyu Chen, Ruonan Zhang, Ji Jiang, Yan Wang, Ge Li, and Thomas H Li. Self-supervised monocular depth estimation: Solving the edge-fattening problem. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pages 5776–5786, 2023.

[17] Javier Civera, Andrew J Davison, and JM Martinez Montiel. Inverse depth parametrization for monocular slam. *IEEE transactions on robotics*, 24(5):932–945, 2008.

[18] Antonio Criminisi, Ian Reid, and Andrew Zisserman. Single view metrology. In *Proceedings of the Seventh IEEE International Conference on Computer Vision*, volume 1, pages 434–441. IEEE, 1999.

[19] James E Cutting and Peter M Vishton. Perceiving layout and knowing distances. In *Perception of Space and Motion*, pages 69–117. Academic Press, 1995.

[20] Arthur P Dempster, Nan M Laird, and Donald B Rubin. Maximum likelihood from incomplete data via the em algorithm. *Journal of the royal statistical society: series B (methodological)*, 39(1):1–22, 1977.

[21] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255, 2009.

[22] Daniel DeTone, Tomasz Malisiewicz, and Andrew Rabinovich. SuperPoint: Self-supervised interest point detection and description. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops*, pages 224–236, 2018.

[23] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. *International Conference on Learning Representations*, 2021.

[24] David Eigen and Rob Fergus. Predicting depth, surface normals and semantic labels with a common multi-scale convolutional architecture. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 2650–2658, 2015.

[25] David Eigen, Christian Puhrsch, and Rob Fergus. Depth map prediction from a single image using a multi-scale deep network. In *Advances in neural information processing systems*, pages 2366–2374, 2014.

[26] Jakob Engel, Thomas Schöps, and Daniel Cremers. Lsd-slam: Large-scale direct monocular slam. In *European conference on computer vision*, pages 834–849. Springer, 2014.

[27] William Epstein. The known-size-apparent-distance hypothesis. *The American Journal of Psychology*, 74(3):333–346, 1961.

[28] Haoqiang Fan, Hao Su, and Leonidas J. Guibas. A point set generation network for 3d object reconstruction from a single image. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 605–613, 2017.

[29] Y Fan et al. Foundationstereo: Zero-shot stereo matching. *arXiv preprint arXiv:2501.09898*, 2025.

[30] Martin A Fischler and Robert C Bolles. Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography. *Communications of the ACM*, 24(6):381–395, 1981.

[31] Salva Foix, Guillem Alenya, and Carme Torras. Lock-in time-of-flight (ToF) cameras: A survey. *IEEE Sensors Journal*, 11(9):1917–1926, 2011.

[32] Huan Fu, Mingming Gong, Chaohui Wang, Kayhan Batmanghelich, and Dacheng Tao. Deep ordinal regression network for monocular depth estimation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2002–2011, 2018.

[33] Adrien Gaidon, Qiao Wang, Yohann Cabon, and Eleonora Vig. Virtual worlds as proxy for multi-object tracking analysis. In *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition*, pages 4340–4349, 2016.

[34] Ravi Garg, Vijay Kumar Bg, Gustavo Carneiro, and Ian Reid. Unsupervised cnn for single view depth estimation: Geometry to the rescue. In *Computer Vision–ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11-14, 2016, Proceedings, Part VIII 14*, pages 740–756. Springer, 2016.

[35] Andreas Geiger, Philip Lenz, Christoph Stiller, and Raquel Urtasun. Vision meets robotics: The KITTI dataset. In *The International Journal of Robotics Research*, volume 32, pages 1231–1237, 2013.

[36] James Jerome Gibson. *The Perception of the Visual World*. Houghton Mifflin, 1950.

[37] Clément Godard, Oisin Mac Aodha, and Gabriel J Brostow. Unsupervised monocular depth estimation with left-right consistency. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 270–279, 2017.

[38] Clément Godard, Oisin Mac Aodha, Michael Firman, and Gabriel J Brostow. Digging into self-supervised monocular depth estimation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 3828–3838, 2019.

[39] Ariel Gordon, Hanhan Li, Rico Jonschkowski, and Anelia Angelova. Depth from videos in the wild: Unsupervised monocular depth learning from unknown cameras. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 8977–8986, 2019.

[40] Vitor Guizilini, Rares Ambrus, Sudeep Pillai, Allan Raventos, and Adrien Gaidon. 3d packing for self-supervised monocular depth estimation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 2485–2494, 2020.

[41] Mohit Gupta, Qi Yin, and Shree K. Nayar. Structured light in sunlight. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, December 2013.

[42] Miles Hansard, Seungkyu Lee, Ouk Choi, and Radu Horaud. *Time-of-flight cameras: principles, methods and applications*. Springer Science & Business Media, 2012.

[43] Chris Harris and Mike Stephens. A combined corner and edge detector. In *Alvey Vision Conference*, pages 147–151, 1988.

[44] R. Hartley and A. Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge University Press, 2004.

[45] Richard Hartley and Andrew Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge University Press, 2nd edition, 2003.

[46] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

[47] Heiko Hirschmuller. Stereo processing by semiglobal matching and mutual information. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 30(2):328–341, 2008.

[48] Derek Hoiem, Andrew N Stein, Alexei A Efros, and Martial Hebert. Recovering occlusion boundaries from a single image. In *2007 IEEE 11th International Conference on Computer Vision*, pages 1–8. IEEE, 2007.

[49] Ian P Howard and Brian J Rogers. Perceiving in depth, volume 1: Basic mechanisms. 2012. A modern exhaustive reference on binocular and monocular cues used in human and machine vision.

[50] Yueru Jia, Jiaming Liu, Sixiang Chen, Chenyang Gu, Zhilue Wang, Longzan Luo, Lily Lee, Pengwei Wang, Zhongyuan Wang, Renrui Zhang, and Shanghang Zhang. Lift3d foundation policy: Lifting 2D large-scale pretrained models for robust 3D robotic manipulation. *arXiv preprint arXiv:2411.18623*, 2024.

[51] Adrian Johnston and Gustavo Carneiro. Self-supervised monocular trained depth estimation using self-attention and discrete disparity volume. In *Proceedings of the ieee/cvf conference on computer vision and pattern recognition*, pages 4756–4765, 2020.

[52] Hyunyoung Jung, Eunhyeok Park, and Sungjoo Yoo. Fine-grained semantics-aware representation enhancement for self-supervised monocular depth estimation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 12642–12652, 2021.

[53] Achuta Kadambi and Ramesh Raskar. Rethinking resolution: The case for compressive time-of-flight imaging. *IEEE Transactions on Computational Imaging*, 2017. Discusses the physical fabrication limits that lead to lower spatial resolution in ToF sensors compared to RGB sensors.

[54] Rudolph Emil Kalman. A new approach to linear filtering and prediction problems. 1960.

[55] Juho Kannala and Sami S. Brandt. A generic camera model and calibration method for conventional, wide-angle, and fish-eye lenses. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 28(8):1335–1340, 2006.

[56] Alex Kendall and Yarin Gal. What uncertainties do we need in bayesian deep learning for computer vision? In *Advances in neural information processing systems*, pages 5574–5584, 2017.

[57] Fayaz Ali Khan, Waleed Ahmed, Mohammed Al-Mutairi, et al. A survey on deep learning techniques for monocular depth estimation. *IEEE Access*, 8:52322–52344, 2020.

[58] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2014.

[59] Genki Kinoshita and Ko Nishino. Camera height doesn't change: Unsupervised monocular scale-aware road-scene depth estimation, 2023.

[60] Maria Klodt and Andrea Vedaldi. Supervising the new with the old: learning SfM from SfM. In *Proceedings of the European conference on computer vision (ECCV)*, pages 698–713, 2018.

[61] Arno Knapitsch, Jaesik Park, Qian-Yi Zhou, and Vladlen Koltun. Tanks and temples: Benchmarking large-scale scene reconstruction. *ACM Transactions on Graphics*, 36(4), 2017.

[62] Tobias Koch, Lukas Liebel, Friedrich Fraundorfer, and Korner Marco. Evaluation of optical online 3d reconstruction methods: Lidar vs. realsense vs. kinect. *International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, 2018. A technical comparison of how different sensor technologies vary in precision and noise, affecting GT quality.

[63] Karlo Koledić, Luka Petrović, Ivan Marković, and Ivan Petrović. Gvdepth: Zero-shot monocular depth estimation for ground vehicles based on probabilistic cue fusion. *arXiv preprint arXiv:2412.06080*, 2024.

[64] Karlo Koledić, Luka Petrović, Ivan Petrović, and Ivan Marković. GenDepth: Generalizing monocular depth estimation for arbitrary camera parameters via ground plane embedding, 2023.

[65] Y Kondo et al. Multi-path interference mitigation for indirect time-of-flight camera. *Kyushu University*, 2021.

[66] Jason Ku, Melissa Mozifian, Jungwook Lee, Ali Harakeh, and Steven L Waslander. Joint 3d proposal generation and object detection from view point transformations. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1–8. IEEE, 2018.

[67] Iro Laina, Christian Rupprecht, Vasileios Belagiannis, Federico Tombari, and Nassir Navab. Deeper depth prediction with fully convolutional residual networks. In *2016 Fourth International Conference on 3D Vision (3DV)*, pages 239–248. IEEE, 2016.

[68] Jin Han Lee, Myung-Kyu Han, Dong Wook Ko, and Il Hong Suh. Big to small: Multi-scale local planar guidance for monocular depth estimation. In *arXiv preprint arXiv:1907.10326*, 2019.

[69] Jin Han Lee, Myung-Kyu Han, Dong Wook Ko, and Il Hong Suh. From big to small: Multi-scale local planar guidance for monocular depth estimation. In *IEEE Transactions on Pattern Analysis and Machine Intelligence*, volume 43, pages 490–506. IEEE, 2021.

[70] Vincent Lepetit, Francesc Moreno-Noguer, and Pascal Fua. EPnP: An accurate O(n) solution to the PnP problem. volume 81, pages 155–166. Springer, 2009.

[71] Vincent Leroy, Yohann Cabon, and Jérôme Revaud. Grounding image matching in 3D with MASt3R. *arXiv preprint arXiv:2406.09756*, 2024.

[72] You Li and Javier Ibanez-Guzman. Lidar for autonomous driving: The state of the art and the future. *IEEE Signal Processing Magazine*, 37(4):56–71, 2020. Excellent overview of hardware complexity, mechanical rotation, and the sparse nature of point clouds in automotive contexts.

[73] Zhenyu Li, Zehui Chen, Xianming Liu, and Junjun Jiang. Depthformer: Exploiting long-range correlation and local information for accurate monocular depth estimation. 2022.

[74] Haotong Lin, Sili Chen, Jun Hao Liew, Donny Y. Chen, Zhenyu Li, Guang Shi, Jiashi Feng, and Bingyi Kang. Depth anything 3: recovering the visual space from any views. *arXiv preprint arXiv:2511.10647*, 2025.

[75] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 10012–10022, 2021.

[76] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A convnet for the 2020s. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11976–11986, 2022.

[77] David G Lowe. Distinctive image features from scale-invariant keypoints. *International Journal of Computer Vision*, 60(2):91–110, 2004.

[78] Xiaoyang Lyu, Liang Liu, Mengmeng Wang, Xin Kong, Lina Liu, Yong Liu, Xinxin Chen, and Yi Yuan. Hr-depth: High resolution self-supervised monocular depth estimation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 2294–2301, 2021.

[79] Fangchang Ma, Guilherme V Cavalheiro, and Sertac Karaman. Self-supervised sparse-to-dense: Self-supervised depth completion from LiDAR and monocular camera. In *2019 International Conference on Robotics and Automation (ICRA)*, pages 3288–3295. IEEE, 2019.

[80] Yi Ma, Jana Kosecka, Stefano Soatto, and S Shankar Sastry. *An Invitation to 3-D Vision: From Images to Geometric Models*. Springer, 2004. Provides a rigorous derivation of how depth maps are lifted into 3D Euclidean space.

[81] Andrew Maimone and Henry Fuchs. Reducing interference between multiple structured light depth sensors using motion. In *2012 IEEE Virtual Reality Workshops (VRW)*, pages 51–54, 2012.

[82] Jitendra Malik and Ruth Rosenholtz. Computing local surface orientation and shape from texture for 3-d scene analysis. *International Journal of Computer Vision*, 23(2):149–168, 1997.

[83] Pascal Mamassian, David C Knill, and Daniel Kersten. The perception of cast shadows. *Trends in Cognitive Sciences*, 2(8):288–295, 1998.

[84] Leonard A McGee and Stanley F Schmidt. Discovery of the kalman filter as a practical tool for aerospace and industry. Technical report, 1985.

[85] Yue Ming, Xiaohuan Meng, Chunlei Fan, and Hui Yu. Deep learning for monocular depth estimation: A review. *Neurocomputing*, 438:14–33, 2021.

[86] Nikhil Naik, Achuta Kadambi, Christoph Rhemann, Shahram Izadi, Ramesh Raskar, and Sing Bing Kang. A light transport model for mitigating multipath interference in time-of-flight sensors. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 73–81, 2015.

[87] Srinivasa G Narasimhan and Shree K Nayar. Vision and the atmosphere. *International Journal of Computer Vision*, 48(3):233–254, 2002.

[88] Pushmeet Kohli Nathan Silberman, Derek Hoiem and Rob Fergus. Indoor segmentation and support inference from rgbd images. In *ECCV*, 2012.

[89] Richard A Newcombe, Shahram Izadi, Otmar Hilliges, David Molyneaux, David Kim, Andrew J Davison, Pushmeet Kohi, Jamie Shotton, Steve Hodges, and Andrew Fitzgibbon. KinectFusion: Real-time 3d reconstruction and interaction using a moving depth camera. In *ACM Symposium on User Interface Software and Technology (UIST)*, pages 559–568, 2011.

[90] Teng Leng Ooi, Bin Wu, and Zijiang J He. Distance determined by the angular declination below the horizon. *Nature*, 414(6860):197–200, 2001.

[91] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, et al. DINOv2: Learning robust visual features without supervision. *arXiv preprint arXiv:2304.07193*, 2023.

[92] Stephen E Palmer. *Vision Science: Photons to Phenomenology*. MIT Press, 1999.

[93] Byeonghoon Park, Yongchan Keh, Donghi Lee, Yongkwan Kim, Sungsoon Kim, Kisuk Sung, Jungkee Lee, Donghoon Jang, and Youngkwon Yoon. Outdoor operation of structured light in mobile phone. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV) Workshops*, Oct 2017.

[94] Andra Petrovai and Sergiu Nedevschi. Monodvps: A self-supervised monocular depth estimation approach to depth-aware video panoptic segmentation. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pages 3077–3086, 2023.

[95] Luigi Piccinelli, Yung-Hsu Yang, Christos Sakaridis, Mattia Segu, Siyuan Li, Luc Van Gool, and Fisher Yu. UniDepth: Universal monocular metric depth estimation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10371–10381, 2024.

[96] Sudeep Pillai, Rareş Ambruş, and Adrien Gaidon. Superdepth: Self-supervised, super-resolved monocular depth estimation. In *2019 International Conference on Robotics and Automation (ICRA)*, pages 9250–9256. IEEE, 2019.

[97] Matteo Poggi, Filippo Aleotti, Fabio Tosi, and Stefano Mattoccia. On the uncertainty of self-supervised monocular depth estimation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 3227–3237, 2020.

[98] Matteo Poggi, Fabio Tosi, Konstantinos Batsos, Philippos Mordohai, and Stefano Mattoccia. On the synergies between machine learning and binocular stereo for depth estimation from images: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(9):5314–5334, 2021.

[99] René Ranftl, Alexey Bochkovskiy, and Vladlen Koltun. Vision transformers for dense prediction. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 12179–12188, 2021.

[100] René Ranftl, Katrin Lasinger, David Hafner, Konrad Schindler, and Vladlen Koltun. Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(3):1623–1637, 2022.

[101] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *International Conference on Medical image computing and computer-assisted intervention*, pages 234–241. Springer, 2015.

[102] Santiago Royo and Marc Ballesta-Garcia. An overview of LiDAR imaging systems for autonomous vehicles. *Applied Sciences*, 9(19):4093, 2019. Details the "Time-of-Flight" (ToF) principle, the centimeter-level precision, and the mechanical limitations of scanning mirrors.

[103] Ethan Rublee, Vincent Rabaud, Kurt Konolige, and Gary Bradski. ORB: An efficient alternative to SIFT or SURF. In *2011 International Conference on Computer Vision*, pages 2564–2571. IEEE, 2011.

[104] Radu Bogdan Rusu and Steve Cousins. Point cloud library (pcl). 2011. The standard reference for defining and processing *point clouds* as a data structure in 3D vision.

[105] Ashutosh Saxena, Min Sun, and Andrew Y Ng. Make3d: Learning 3d scene structure from a single still image. In *IEEE transactions on pattern analysis and machine intelligence*, volume 31, pages 824–840, 2009.

[106] Daniel Scharstein and Richard Szeliski. A taxonomy and evaluation of dense two-frame stereo correspondence algorithms. *International Journal of Computer Vision*, 47(1-3):7–42, 2002.

[107] Johannes L Schönberger and Jan-Michael Frahm. Structure-from-motion revisited. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 4104–4113, 2016.

[108] Brent Schwarz. LiDAR: Mapping the world in 3d. *Nature Photonics*, 4(7):429–430, 2010.

[109] Maximilian Seitzer, Arash Tavakoli, Dimitrije Antic, and Georg Martius. On the pitfalls of heteroscedastic uncertainty estimation with probabilistic neural networks. In *International Conference on Learning Representations*, 2022.

[110] Jamie Shotton, Andrew Fitzgibbon, Mat Cook, Toby Sharp, Mark Finocchio, Richard Moore, Alex Kipman, and Andrew Blake. Real-time human pose recognition in parts from single Depth images. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2011. The seminal paper for the Microsoft Kinect, explaining the use of structured light for dense indoor depth sensing.

[111] Chang Shu, Kun Yu, Zhixiang Duan, and Kuiyuan Yang. Feature-metric loss for self-supervised learning of depth and egomotion. In *European Conference on Computer Vision*, pages 572–588. Springer, 2020.

[112] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.

[113] Nicki Skafte, Martin Søgaard, Soren Zec, and Daniel Petersen. Reliable training and estimation of variance networks. In *Advances in Neural Information Processing Systems*, pages 6323–6333, 2019.

[114] Ludo Sluijterman, Kamil Ciosek, Shimon Whiteson, Yoram Bachrach, and Yee Whye Teh. Optimal and efficient mean-variance estimation for deep learning. *arXiv preprint arXiv:2402.02033*, 2024.

[115] Jaime Spencer, Chris Russell, Simon Hadfield, and Richard Bowden. Deconstructing self-supervised monocular reconstruction: The design decisions that matter. *Transactions on Machine Learning Research*, 2022. Reproducibility Certification.

[116] Jiaming Sun, Zehong Shen, Yuang Wang, Hujun Bao, and Xiaowei Zhou. LoFTR: Detector-free local feature matching with transformers. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 8922–8931, 2021.

[117] Richard Szeliski. *Computer vision: algorithms and applications*. Springer Nature, 2022.

[118] A. Torralba, P. Isola, and W.T. Freeman. *Foundations of Computer Vision*. Adaptive Computation and Machine Learning series. MIT Press, 2024.

[119] Fabio Tosi, Filippo Aleotti, Matteo Poggi, and Stefano Mattoccia. Smd-nets: Stereo mixture density networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 8942–8952, 2021.

[120] Bill Triggs, Philip F McLauchlan, Richard I Hartley, and Andrew W Fitzgibbon. Bundle adjustment — a modern synthesis. *Vision Algorithms: Theory and Practice*, pages 298–372, 2000.

[121] Shubham Tulsiani, Richard Tucker, and Noah Snavely. Layer-structured 3d scene inference via view synthesis. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 302–317, 2018.

[122] Jonas Uhrig, Nick Schneider, Lukas Schneider, Uwe Franke, Thomas Brox, and Andreas Geiger. Sparsity invariant cnns. In *2017 International Conference on 3D Vision (3DV)*, pages 11–20. IEEE, 2017.

[123] Jonas Uhrig, Nick Schneider, Lukas Schneider, Uwe Franke, Thomas Brox, and Andreas Geiger. Sparsity invariant cnns. In *International Conference on 3D Vision (3DV)*, 2017.

[124] Tom Van Dijk and Guido De Croon. How do neural networks see depth in single images? In *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 2183–2191. IEEE, 2019.

[125] Brandon Wagstaff and Jonathan Kelly. Self-supervised scale recovery for monocular depth and egomotion estimation. In *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 2620–2627, 2021.

[126] Jianyuan Wang, Minghao Chen, Nikita Karaev, Andrea Vedaldi, Christian Rupprecht, and David Novotny. VGGT: Visual geometry grounded transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025.

[127] Ruicheng Wang, Sicheng Xu, Cassie Dai, Jianfeng Xiang, Yu Deng, Xin Tong, and Jiaolong Yang. Moge: Unlocking accurate monocular geometry estimation for open-domain images with optimal training supervision. *arXiv preprint arXiv:2410.19115*, 2024.

[128] Ruoyu Wang, Zehao Yu, and Shenghua Gao. Planedepth: Self-supervised depth estimation via orthogonal planes. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 21425–21434, 2023.

[129] Shuzhe Wang, Vincent Leroy, Yohann Cabon, Boris Chidlovskii, and Jerome Revaud. DUSt3R: Geometric 3D vision made easy. *arXiv preprint arXiv:2312.14132*, 2024.

[130] Yifan Wang, Jianjun Zhou, Haoyi Zhu, Wenzheng Chang, Yang Zhou, Zizun Li, Junyi Chen, Jiangmiao Pang, Chunhua Shen, and Tong He. π^3: Permutation-equivariant visual geometry learning. *arXiv preprint arXiv:2507.13347*, 2025.

[131] Yihan Wang, Lahav Lipson, and Jia Deng. Sea-raft: Simple, efficient, accurate raft for optical flow, 2024.

[132] Youhong Wang, Yunji Liang, Hao Xu, Shaohui Jiao, and Hongkai Yu. Sqldepth: Generalizable self-supervised fine-structured monocular depth estimation. In *Proceedings of the AAAI conference on artificial intelligence*, volume 38, pages 5713–5721, 2024.

[133] Zhou Wang, Alan C Bovik, Hamid R Sheikh, and Eero P Simoncelli. Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, 2004.

[134] Juyang Weng, Paul Cohen, and Michel Herniou. Camera calibration with distortion models and accuracy evaluation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 14(10):965–980, 1992.

[135] Jie Xiang, Yun Wang, Lifeng An, Haiyang Liu, Zijun Wang, and Jian Liu. Visual attention-based self-supervised absolute depth estimation using geometric priors in autonomous driving. 7(4):11998–12005, 2022.

[136] Gangwei Xu, Haotong Lin, Hongcheng Luo, Xianqi Wang, Jingfeng Yao, Lianghui Zhu, Yuechuan Pu, Cheng Chi, Haiyang Sun, Bing Wang, et al. Pixel-perfect depth with semantics-prompted diffusion transformers. *arXiv preprint arXiv:2510.07316*, 2025.

[137] Guanglei Yan, Zhichao Li, Qiuyu Xie, Mingyu Cheng, Zhen Wang, Siu-Ming Lee, Xueming Liu, and Shang Gao. Channel-wise attention-based network for self-supervised monocular depth estimation. In *2021 International Conference on 3D Vision (3DV)*, pages 464–473, 2021.

[138] Lihe Yang, Bingyi Kang, Zilong Huang, Xiaogang Xu, Jiashi Feng, and Hengshuang Zhao. Depth anything: Unleashing the power of large-scale unlabeled data. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10371–10381, 2024.

[139] Lihe Yang, Bingyi Kang, Zilong Huang, Zhen Zhao, Xiaogang Xu, Jiashi Feng, and Hengshuang Zhao. Depth anything V2. In *Advances in Neural Information Processing Systems*, 2024.

[140] Xiaodong Yang, Zhuang Ma, Zhiyu Ji, and Zhe Ren. GEDepth: Ground embedding for monocular depth estimation. In *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 12673–12681. IEEE, 2023.

[141] Wei Yin, Chi Zhang, Hao Chen, Zhipeng Cai, Gang Yu, Kaixuan Wang, Xiaozhi Chen, and Chunhua Shen. Metric3d: Towards zero-shot metric 3d prediction from a single image. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 9043–9053, 2023.

[142] Huangying Zhan, Ravi Garg, Chamara Saroj Weerasekera, Kejie Li, Harsh Agarwal, and Ian Reid. Unsupervised learning of monocular depth estimation and visual odometry with deep feature reconstruction. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 340–349, 2018.

[143] Jiuling Zhang, Yurong Wu, and Huilong Jiang. Survey on monocular metric depth estimation. *Computers*, 14(11):502, 2025.

[144] Zhengyou Zhang. A flexible new technique for camera calibration. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 22(11):1330–1334, 2000.

[145] Zhengyou Zhang. Microsoft Kinect sensor and its effect. *IEEE multimedia*, 19(2):4–10, 2012.

[146] Chaoqiang Zhao, Youmin Zhang, Matteo Poggi, Fabio Tosi, Xianda Guo, Zheng Zhu, Guan Huang, Yang Tang, and Stefano Mattoccia. Monovit: Self-supervised monocular depth estimation with a vision transformer. In *2022 international conference on 3D vision (3DV)*, pages 668–678. IEEE, 2022.

[147] Hang Zhou, David Greenwood, and Sarah Taylor. Self-supervised monocular depth estimation with internal feature fusion. *arXiv preprint arXiv:2110.09482*, 2021.

[148] Qian-Yi Zhou, Jaesik Park, and Vladlen Koltun. Open3D: A modern library for 3D data processing. *arXiv:1801.05934*, 2018. The definitive reference for Open3D, focusing on its efficiency and Python-friendly API for 3D reconstruction.

[149] Tinghui Zhou, Matthew Brown, Noah Snavely, and David G Lowe. Unsupervised learning of depth and ego-motion from video. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1851–1858, 2017.

[150] Daniel Zoran, Phillip Isola, Dilip Krishnan, and William T Freeman. Learning ordinal relationships for Mid-level Vision. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2015. A key CV paper that formalizes *ordinal depth* (closer-than relationships) as a learning task for monocular images.